微网与外部电网电力调控策略的优化模型研究

摘要

本文针对微网与外部电网的电力调控问题，以小区负载、光伏发电与储能设备之间的能量协调为核心，以最小化微网购电费用为目标，结合**两阶段随机规划与滚动时域优化方法**，提出了系统的购电与储能联合调控方案。

针对问题一：本问是**单日确定性优化**问题，需在电价与负载固定、光伏预测已知时，制定储能 0:00 与 24:00 储电量相等的日计划购电策略。对附件一数据按 10 分钟为步长重构为 144 个时间间隔；以购电功率、储能充放电功率与储电量为决策变量，建立功率平衡、储能状态递推、容量上下限与充放电功率约束；以全天购电费用最小为目标，加入储能初值 6000 kWh 与日循环约束，构建含 577 个连续变量的**线性规划模型**并精确求解；得到全天购电量 **59482.70 kWh**、购电费 **35126.95 元**，储能 0:00 与 24:00 储电量均为 **6000 kWh**。储能低充高放可显著降低购电费用。

针对问题二：本问是**预测—决策—结算三段式**的全年逐日滚动优化问题。题目要求在每天 0:00 制定当天的计划购电量，而当天负载与光伏在 0:00 尚未发生，若直接引用附件二的实际值即属事后信息，必须先行预测。本文以 1 月数据为训练样本，用扩张窗口逐日标定负载与光伏预测模型，在 0:00 给出当天 144 个间隔的功率预测值；由历史预测残差估计不确定性，按负载与光伏各取低/中/高共 9 个情景。据此建立两阶段**随机线性规划模型**，目标为计划购电费与 5 倍电价下的期望紧急购电费之和，决策当天计划购电量与储能充放电计划；实际运行时储能严格按计划充放电，真实功率到达后按供电缺口结算紧急购电、富余电量计为弃电，日末储电量传递至次日。2025 年 1 月作为**预热期**，按附件 5 口径统计 2 月 1 日—12 月 31 日（334 天）：计划购电量 **21833065.92 kWh**、计划购电费 **13325494.59 元**；紧急购电量 **282910.68 kWh**、紧急购电费 **1057398.40 元**；总购电费 **14382892.99 元**，日均约 **43062.55 元**。334 天中共有 333 天出现紧急购电，紧急购电量占总购电量的 1.28%，与“0:00 预测、5 倍电价兜底”的题意相符。

针对问题三：本问是**滚动预报下的调整优化**问题，需在 0:00、6:00、12:00、18:00 依据更新预报调整购电并计及违约与超额费用。将小时级光伏预报插值到 10 分钟；在 0:00 制定全天计划购电，在其余预报时刻对未执行时段滚动重优化；针对计划偏离的分段费用，下调部分按 0.5 倍电价计违约费、上调部分按 1.5 倍电价计价，并引入上境变量将分段费用线性化，紧急购电按实际供电缺口核算；得到（口径同问题二）含调整总费用 **14337999.15 元**，较仅用 0:00 计划降低 **444073.36 元**，费用下降约 **3.00%**。

针对问题四：本问是**波动电价下的重算**问题，需在实时波动电价下重新求解问题二与问题三。读取全年逐间隔波动电价数据；将固定分时电价逐**间隔替换**后，仍按“先预测、再决策、后结算”的三段式口径重新求解逐日计划与滚动调整两类线性规划；得到波动电价下问题二计划购电费 **13895479.01 元**、紧急购电费 **1181652.65 元**、合计

15077131.66 元，问题三总费用 **14956526.18 元**。波动电价使问题二合计由 14382892.99 元升至 15077131.66 元；但在波动电价下引入滚动预报更新后，问题三总费用仍降至 14956526.18 元，较仅用 0:00 计划的 15077131.66 元节省 120605.48 元。可见波动电价整体抬高了微网购电成本，而滚动调整可在实时电价与预报更新下部分对冲该影响。

本文的主要创新点在于：其一，针对问题二建立“先预测、再决策、后结算”的三段式框架，从机制上保证计划购电量不含事后信息，紧急购电费随预测误差自然产生而非恒为零；其二，以残差情景构造两阶段随机线性规划，用 5 倍紧急电价在目标函数中显式刻画风险，避免确定性优化低估缺电代价；其三，将四个问题统一建模为纯连续变量线性规划，由求解器保证全局最优，并以上境变量精确线性化计划偏离的分段违约与超额费用。模型结论可核验，可为微网经济调度提供决策参考。

关键词： **微网**；**储能调度**；**随机规划**；**滚动时域优化**；**光伏预报**

一、 引言

1.1 问题背景

随着以光伏发电为代表的分布式能源大规模接入配电网，其出力受天气影响的间歇性与随机性给电网的功率平衡带来了严峻挑战。微网作为聚合分布式电源、储能设备与本地负荷的小型发用电系统 [1, 11]，能够通过本地消纳光伏电量、平抑负荷波动，显著减少对公共电网的依赖与冲击。在非孤岛运行方式下，微网既可以从外部电网购电，也可以将多余的电量反馈电网或存储于储能设备之中，其能量管理与运行调度已成为研究热点 [2, 12]。

微网与外网调控的核心矛盾在于：一方面，小区负荷与光伏出力在一天之内、一年之内都随时间剧烈变化，光伏只在白天出力且峰谷差异巨大；另一方面，外网电价存在明显的峰谷价差，波动电价下价差更为显著 [15]。若微网完全按负荷实时从外网购电，将承担高昂的峰时购电费用；而储能设备的引入，使微网可以在电价较低或光伏有剩余电量的时段充电，在电价较高或光伏不足的时段放电，从而在满足负荷供电的前提下尽可能降低购电费用。然而，储能设备的容量有限、充放电功率有限且存在效率损耗，如何精准制定购电计划与储能充放电计划，使微网在满足约束的同时实现购电费用最小化，是一个亟待解决的优化问题。因此，建立一套科学的微网购电与储能调控模型具有重要的工程意义。

1.2 问题重述

问题 1：在每日电价与负载相同、光伏预测已知的条件下，建立储能 0:00 与 24:00 储电量相等的微网日计划购电策略模型，并给出指定时段的购电量与储能充放电量。

问题 2：在每日电价相同、负载与光伏随时间变化、供电不足按 5 倍电价紧急购电的条件下，建立每天 0:00 制定当天计划购电策略的模型，并给出全年结果。

问题 3：在 0:00、6:00、12:00、18:00 可获得未来 24 小时光伏预报的条件下，建立含计划购电、调整购电、违约惩罚与紧急购电的微网购电策略模型，并分析引入其他预报时刻调整的必要性。

问题 4：在外网电价实时波动的条件下，建立微网当天购电策略模型，并重新计算问题二与问题三的购电策略。

二、 总体分析

针对问题一本文设计了单日线性规划模型，建立了以全天购电费用最小为目标的优化函数，分析了储能容量、充放电功率与日循环等约束，并按附录 1 给定值固定 0:00 储

电量为 6000 kWh，得出了各 10 分钟间隔的购电功率与储能充放电功率，改进了负荷供电的平衡刻画，确保了储能 0:00 与 24:00 储电量相等的要求。针对问题二，本文将单日模型推广为“预测—决策—结算”三段式的全年逐日滚动优化：在每天 0:00 当天负载与光伏尚未发生时先行预测，按历史预测残差构造 9 个不确定性情景并建立两阶段随机线性规划，求得与真值无关的计划购电量与储能充放电计划，实际功率到达后再按供电缺口以 5 倍电价结算紧急购电、富余电量计为弃电，日末储电量传递至次日，得到全年 365 天的购电策略（其中 1 月为预热期，结果文件与费用汇总仅统计 2 月 1 日—12 月 31 日）。针对问题三，本文引入 0:00、6:00、12:00、18:00 四个时刻的光伏滚动预报，建立计划购电与调整购电的分段惩罚模型，用上境变量将分段费用线性化，得到滚动调整后的购电策略（同样仅统计 2 月 1 日—12 月 31 日）。针对问题四，本文提出了波动电价下的购电策略模型，将电价参数逐间隔替换为实时波动电价，重新计算问题二与问题三，达成了在波动电价下给出完整购电策略的效果。综上，本文设计了一个完整的微网购电与储能调控流程。具体的详细流程如图 1 所示。

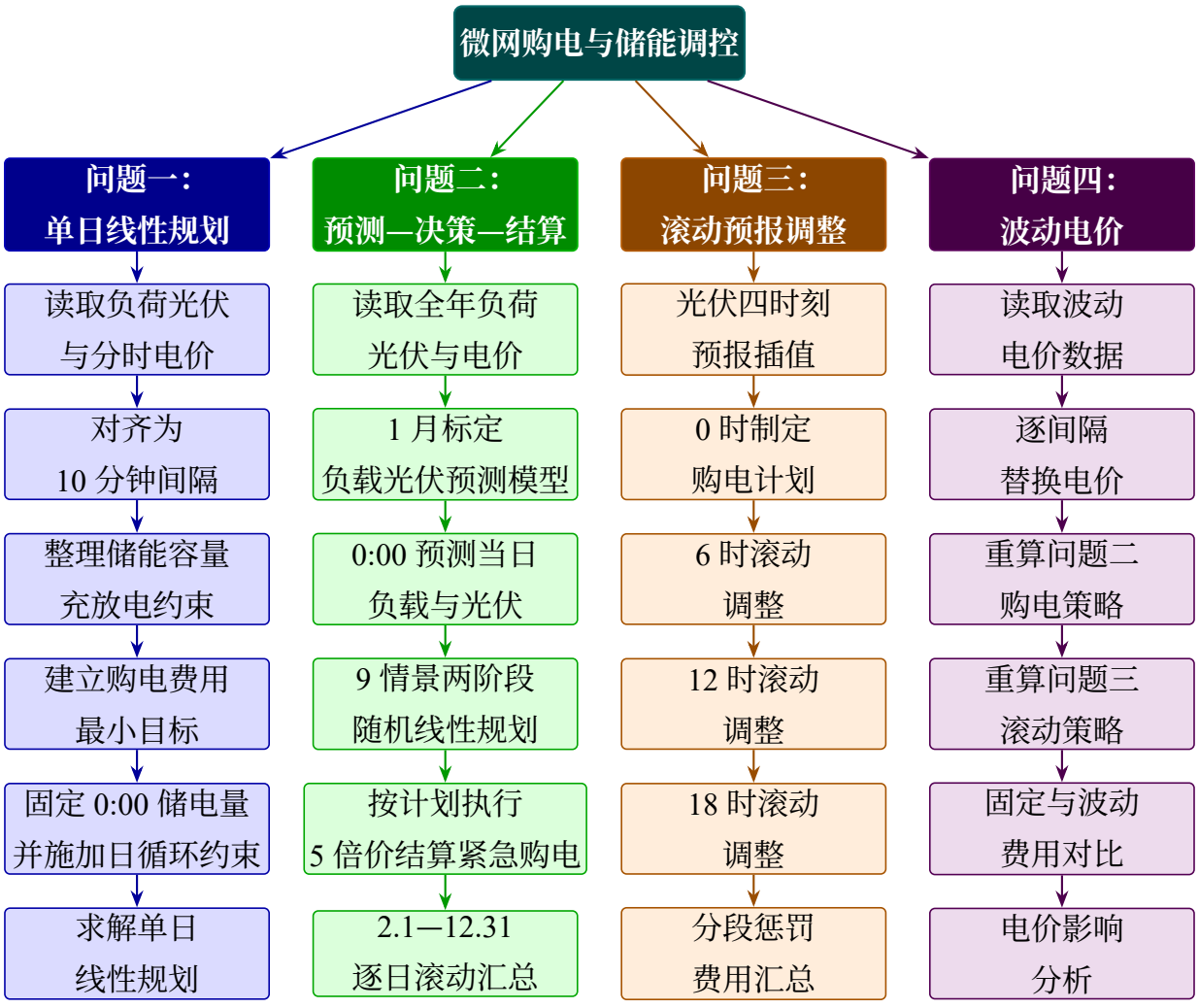


图 **1**　问题分析总体流程图

三、 模型假设

为简化问题，本文做出以下假设:

• **假设 1：**各 10 分钟间隔内的功率保持恒定，购电量、充放电量与功率成正比，忽略更短时间尺度上的功率波动。

• **假设 2：**储能设备的充放电效率均为 90%，容量上下限与充放电功率限值在整个运行周期内保持不变，忽略电池老化与自放电。

• **假设 3：**每天 0:00 制定计划时，当天各间隔的小区负载与光伏出力均尚未发生，不能直接引用其实际值，只能依据历史数据与滚动预报建立预测模型给出估计，预测必然存在误差；实际偏差通过紧急购电机制兜底。

• **假设 4：**外网购电功率无上限，微网与外部电网可随时完成任意规模的购电交易，交易电价仅取决于对应时段的电价数据。

• **假设 5：**各日 0:00 依据当日预测独立决策，储电量跨日传递；日末储电量不作终端约束，每日末端可放至下限，次日谷时再充，这是逐日滚动下的经济性选择。

• **假设 6：**实际运行时储能严格按 0:00 制定的充放电计划执行，真实负载与光伏到达后，出现的供电缺口全部由 5 倍电价的紧急购电补足，富余电量计为弃电。

四、 符号说明

本文使用的主要符号及其含义如表1所示。

表 **1**　符号说明

符号	单位	说明
k	—	日内时间间隔序号（$k = 0, 1, \ldots, 143$）
Δt	h	时间步长（$1/6$ 小时，即 10 分钟）
L_k	kW	第 k 个间隔的小区负载功率
S_k	kW	第 k 个间隔的光伏发电功率
S_k^{τ}	kW	预报时刻 τ 发布的第 k 个间隔光伏预报功率
$\hat{L}_k$	kW	0:00 预测得到的第 k 个间隔负载功率
$\hat{S}_k$	kW	0:00 预测得到的第 k 个间隔光伏功率
$\varepsilon_k^L, \varepsilon_k^S$	kW	负载、光伏的预测残差（实际值减预测值）
s	—	不确定性情景序号（$s = 1, \ldots, 9$）
ω_s	—	第 s 个场景下负载与光伏的乘积因子对
π_s	—	第 s 个情景的概率
L_k^s, S_k^s	kW	第 s 个情景下第 k 个间隔的负载、光伏功率

表 **1**　符号说明（续）

符号	单位	说明
p_k	元/kWh	第 k 个间隔的外网交易电价
g_k	kW	第 k 个间隔的计划购电功率
g_k^{p}	kW	0:00 制定的计划购电功率
g_k^{a}	kW	调整后的购电功率
c_k	kW	第 k 个间隔的储能充电功率
d_k	kW	第 k 个间隔的储能放电功率
E_k	kWh	第 k 个间隔起始时刻的储电量
η	—	储能充放电效率（取 0.9）
$E_{\min}, E_{\max}$	kWh	储电量上下限（1200、10800）
$P_{\max}$	kW	储能最大充放电功率（5000）
δ_k	kWh	第 k 个间隔的实际供电缺口（紧急购电能量）

五、 模型的建立与求解

5.1 问题一的模型的建立和求解

5.1.1 具体分析

针对问题一，本问属于确定性线性优化类问题，采用线性规划方法对微网一天的购电与储能充放电策略进行联合优化求解。

本文先对附件一提供的单日分时电价、小区负载与光伏预测数据进行读取与时间对齐，将其整理为一天 144 个 10 分钟时间间隔上的功率序列；依据储能设备的容量、充放电功率与效率约束，将一天内的能量平衡关系与储能荷电状态递推关系写成线性约束；以全天购电费用最小为目标函数，加入储能设备在 0:00 与 24:00 储电量相等的日循环约束，并按附录 1 给定值固定 0:00 储电量为 6000 kWh，构建标准线性规划模型；最后调用线性规划求解器求得每个时间间隔的购电功率与储能充放电功率，进而得到全天购电量、购电费以及各时段储能充放电量。具体分析流程如图2所示。



图 **2**　问题一分析流程图

5.1.2 模型准备

1. 数据预处理

附件一给出了某一天不同时刻的电价、小区负载和光伏发电预测功率，时间分辨率均为 10 分钟，其中末行时间记作 0:00+1，表示当天 24:00。原始数据以时间字符串索引，无法直接用于逐时段递推计算，因此需要先进行时间对齐与数值化处理。为便于后续建模，首先对数据进行预处理，具体步骤如下。

将一天划分为 144 个等长时间间隔，第 k 个间隔对应时段 $[k\Delta t, (k+1)\Delta t]$，其中 $\Delta t = 1/6$ 小时。设第 k 个间隔的负载功率为 L_k、光伏功率为 S_k、电价为 p_k，则数据点 $(k+1)\Delta t$ 处记录的功率值即为该间隔内的平均功率，其能量可表示为：

$$E_k = P_k\,\Delta t, \quad k = 0, 1, \ldots, 143 \tag{1}$$

其中，E_k 表示第 k 个间隔的能量（kWh），P_k 为该间隔的功率（kW）。将时间边界 0:00+1 映射为 24:00，并对三列数据统一执行数值化与缺失检查，预处理前后对比如图3所示。

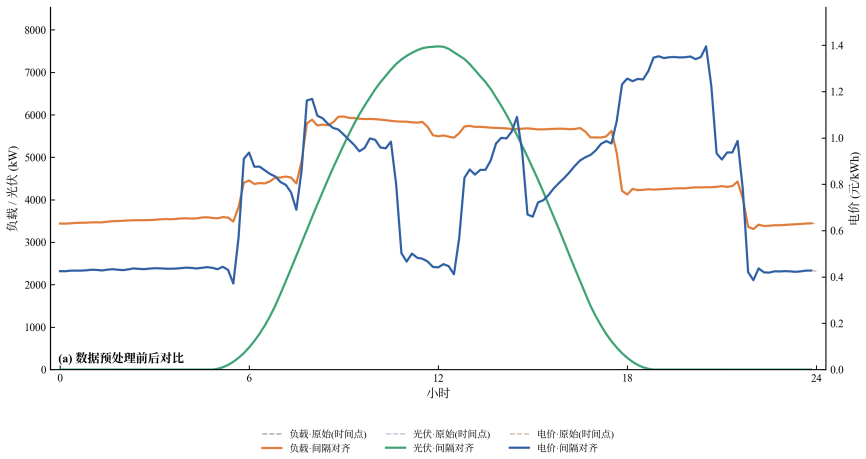


图 **3**　问题一数据预处理前后对比

由图3可知，预处理后电价、负载与光伏三条曲线均被完整对齐到 144 个 10 分钟间隔上，数据无缺失、无异常，且光伏功率在日出后至日落前呈现典型的单峰形态，为后续优化提供了可靠的输入。

2. 算法选择

在完成数据预处理后，本问需要在一组候选算法中确定购电与储能调度的求解方法。微网购电调度本质上是一个带线性约束的凸优化问题，可从求解精度、计算效率与全局最优性等维度比较枚举搜索、贪心算法和线性规划三种候选方法，如表2所示。

表 **2**　问题一算法对比

算法	全局最优性	计算效率	约束处理
枚举搜索	可保证	极低	一般
贪心算法	无保证	高	困难
线性规划	严格保证	高	直接建模

如2所示，枚举搜索虽然理论上能找到最优解，但决策变量规模达数百个，计算量爆炸；贪心算法难以同时协调储能容量约束与日循环约束；而线性规划能够把所有线性约束与目标函数直接纳入模型，由单纯形法或内点法求得全局最优解 [17]，且求解速度极快。综合以上分析，本问选择线性规划算法进行求解。

5.1.3 模型建立

针对问题一，本节将从能量平衡与储能状态递推的角度建立微网日购电计划的线性规划模型。具体过程如下。

步骤 1：决策变量与目标函数

定义模型的决策变量。设第 k 个间隔的购电功率为 g_k，储能充电功率为 c_k，放电功率为 d_k，储电量为 E_k，其中 $k = 0, 1, \ldots, 143$，E_k 为第 k 个间隔起始时刻的储电量。微网全天购电费用为目标函数，即各间隔购电能量与电价乘积之和：

$$\min \ Z = \sum_{k=0}^{143} g_k \, p_k \, \Delta t \tag{2}$$

其中，$g_k \geq 0$ 为购电功率，p_k 为第 k 个间隔的交易电价，Δt 为时间步长。该目标函数线性地刻画了微网的经济诉求。

步骤 2：功率平衡约束

微网在每个间隔内须满足小区负载的用电需求。由功率守恒，购电功率、光伏功率与储能放电功率之和须不低于负载功率与储能充电功率之和，即允许在光伏过剩时弃光：

$$g_k + S_k + d_k - c_k \geq L_k, \quad k = 0, 1, \ldots, 143 \tag{3}$$

其中，S_k 为光伏预测功率，L_k 为小区负载功率，c_k, d_k 分别为充、放电功率。由于光伏发电边际成本为零，最优解会自动优先消纳光伏，只有在光伏与储能放电不足时才从外网购电。

步骤 3：储能状态递推约束

储能设备的荷电状态随充放电行为逐间隔更新。考虑到充、放电效率均为 η，充电时从外部吸收的能量只有 η 倍存入电池，放电时向负荷释放单位能量需从电池取出 $1/\eta$ 倍能量，因此递推关系为：

$$E_{k+1} = E_k + \eta \, c_k \, \Delta t - \frac{1}{\eta} \, d_k \, \Delta t, \quad k = 0, 1, \ldots, 143 \tag{4}$$

其中，$\eta = 0.9$ 为储能充放电效率。储能设备须满足容量上下限约束：

$$E_{\min} \leq E_k \leq E_{\max}, \quad E_{\min} = 1200, \ E_{\max} = 10800 \tag{5}$$

同时充放电功率受储能变换器额定功率限制：

$$0 \leq c_k \leq P_{\max}, \quad 0 \leq d_k \leq P_{\max}, \quad P_{\max} = 5000 \tag{6}$$

步骤 4：储电初值与日循环约束

附录 1 给定 2025 年 1 月 1 日 0:00 的储电量为 6000 kWh，故储能 0:00 储电量应固定为：

$$E_0 = 6000 \tag{7}$$

同时，问题一要求储能设备在 0:00 与 24:00 的储电量相同，以保证相邻日策略的可重复性，故加入日循环约束：

$$E_0 = E_{144} \tag{8}$$

综上所述，问题一的完整线性规划模型为：

$$\min \; Z = \sum_{k=0}^{143} g_k \, p_k \, \Delta t \tag{9}$$

$$\text{s.t.} \quad g_k + S_k + d_k - c_k \geq L_k \tag{10}$$

$$E_{k+1} = E_k + \eta \, c_k \, \Delta t - \frac{1}{\eta} \, d_k \, \Delta t \tag{11}$$

$$E_{\min} \leq E_k \leq E_{\max}, \quad 0 \leq c_k, d_k \leq P_{\max}, \quad g_k \geq 0 \tag{12}$$

$$E_0 = 6000, \quad E_0 = E_{144} \tag{13}$$

该模型共有 $4 \times 144 + 1 = 577$ 个连续变量，其中 E_0 由附录 1 固定为 6000 kWh；全部约束均为线性等式或不等式，属于标准的线性规划问题 [3]，可用单纯形法或内点法精确求解，且所得解即全局最优解。

综上所述，本节完成了问题一优化模型的建立，得到了以全天购电费用最小为目标、以功率平衡与储能状态为约束的线性规划模型，接下来将基于此进行求解。

5.1.4 模型求解

基于上述模型，本节利用附件一数据在 Python 环境下调用线性规划求解器进行求解。核心参数为 $\Delta t = 1/6$ h、$\eta = 0.9$、$E_{\min} = 1200$ kWh、$E_{\max} = 10800$ kWh、$P_{\max} = 5000$ kW，并按附录 1 给定 $E_0 = 6000$ kWh。求解得到微网在指定时间段的购电量及全天购电量和购电费如表3所示，储能设备在指定时段的充放电量及储电量如表4所示。

分时电价与最优购电功率的对应关系如图4所示。可以看到，购电功率主要集中在夜间低电价时段，用于在低价区间补充负荷缺口并为储能充电；在电价高峰时段购电功率明显下降，说明储能放电有效替代了高价购电。

负荷、光伏、净负荷与储能充放电功率的时序关系如图5所示。可以看出，光伏功率在白天形成明显高峰，使净负荷在午间大幅下降甚至转负，此时储能吸收光伏盈余进行充电；而在夜间与傍晚负荷高峰，储能放电填补净负荷缺口，购电功率相应减小。

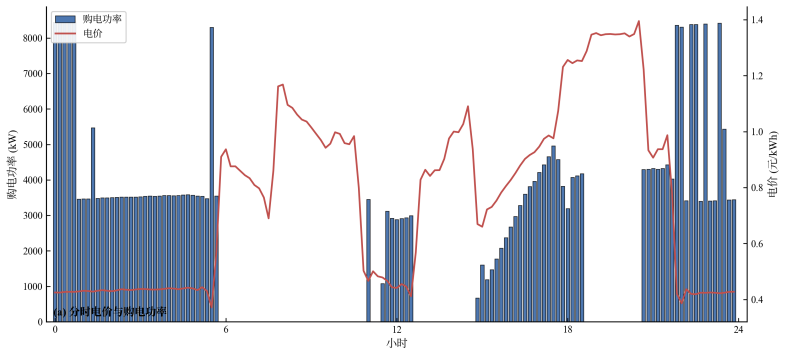


图 **4**　分时电价与最优购电功率

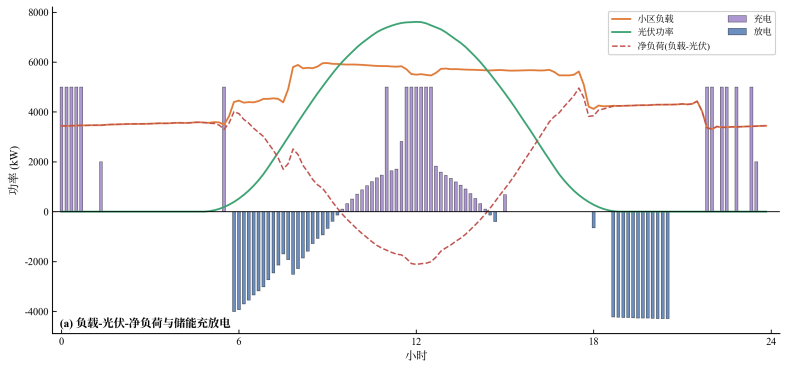


图 **5**　负荷、光伏、净负荷与储能充放电功率

储能设备的荷电状态轨迹与充放电功率如图6所示。储能荷电状态全程维持在 $[1200, 10800]$ kWh 约束区间内，且在 0:00 与 24:00 处均精确等于附录 1 给定的 6000 kWh；充放电功率始终不超过 5000 kW 的额定限值，验证了模型的可行性。

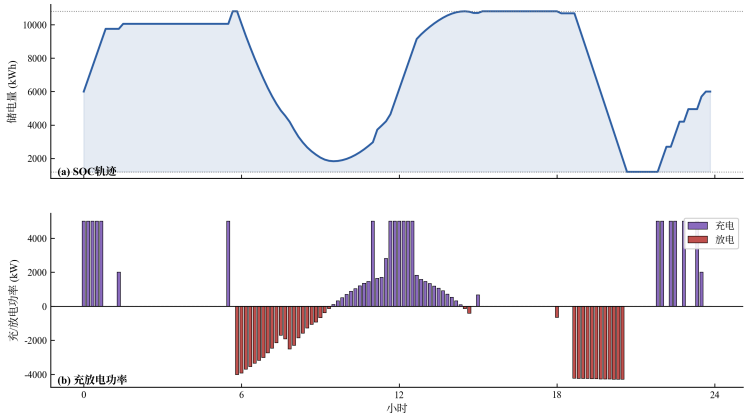


图 **6**　储能荷电状态轨迹与充放电功率

微网一天内的能量平衡关系如图7所示，购电、光伏与储能放电三类供给叠加后恰好覆盖小区负载与储能充电需求，任意时刻能量收支闭合，验证了功率平衡约束的严格满足。

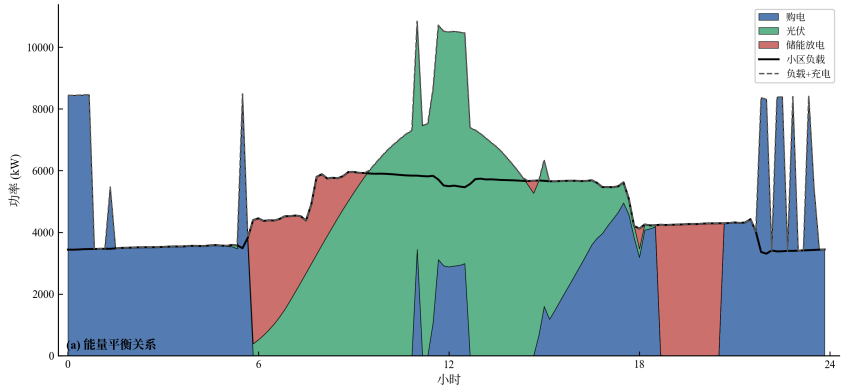


图 **7**　微网全天能量平衡关系

储能 0:00 储电量 E_0 由附录 1 给定为 6000 kWh，属于问题给定的固定条件而非可优化变量。为进一步检验模型对该初值的稳健性，将 E_0 在 $[1200, 10800]$ kWh 区间内逐档变动（并保持日循环 $E_{144} = E_0$）重新求解全天购电费，结果如图8所示。可见购电费随 E_0 变化总体平稳，在附录 1 给定的 6000 kWh 处购电费为 35126.95 元，与最低购电费相比差异很小，说明模型结果对储能初值的选取不敏感，也表明按给定初值求解是合理可靠的。

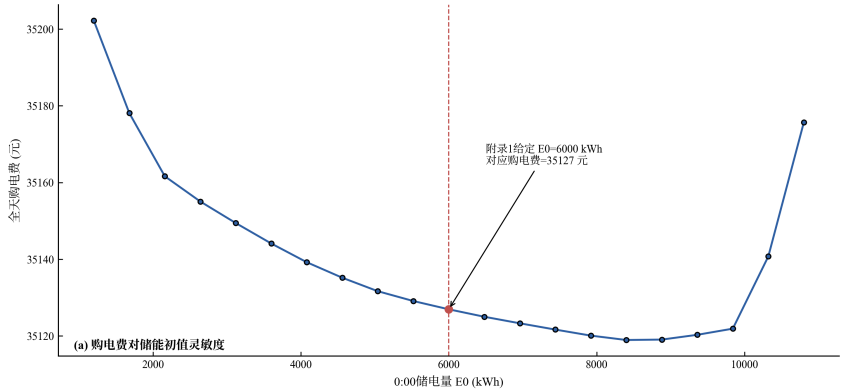


图 **8**　全天购电费对储能初值的灵敏度

表 **3**　微网在指定时间段的购电量及全天的购电量和购电费

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	480.41	14:00–14:10	0.00
16:00–16:10	445.43	18:00–18:10	531.89	20:00–20:10	0.00
全天购电量	59482.70	全天购电费	35126.95		

表 **4**　储能设备在指定时间段的充放电量及 **0:00** 和 **24:00** 的储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	4500.00	0.00	4:00–8:00	833.33	6365.84
8:00–12:00	4787.96	1703.00	12:00–16:00	5286.04	91.10
16:00–20:00	0.00	5780.13	20:00–24:00	5333.33	2859.87
0:00 储电量	6000.00		24:00 储电量	6000.00	

由表3可知，微网全天计划购电量为 59482.70 kWh，全天购电费为 35126.95 元；在 10:00–10:10、14:00–14:10 与 20:00–20:10 等光伏充足或储能放电充足的时段购电量为零，而在 12:00–12:10、16:00–16:10、18:00–18:10 等负荷高企时段需要少量购电。由表4可知，储能设备在 0:00 与 24:00 的储电量均为 6000.00 kWh，与附录 1 给定的 0:00 储电量一致，同时满足日循环约束；储能主要在 0:00–4:00、8:00–12:00 与 20:00–24:00 等低电价时段充电，在 4:00–8:00、16:00–20:00 等高电价时段放电，体现了低充高放的套利规律。

综合以上结果，问题一的线性规划模型在满足负载供电与储能运行约束的前提下，给出了全天购电费用最小的购电与充放电策略，储能设备在低价充电、高价放电的套利过程中使购电费用显著降低，模型与结果均合理可靠。

5.2 问题二的模型的建立和求解

5.2.1 具体分析

针对问题二，本问属于**在预测不确定性下、含两阶段决策的全年逐日滚动优化**问题，与问题一存在两点本质区别。其一，题目要求在每天 0:00 制定当天的计划购电量，而当天 144 个间隔的负载与光伏功率在 0:00 时尚未发生，若在优化中直接引用附件二给出的实际功率，则相当于使用了事后信息，模型会认为供电缺口恒为零、紧急购电永远用不到，这与题目专门设置 5 倍紧急电价并要求以表 3 格式给出紧急购电时段与电量的意图相矛盾；其二，预测必然存在误差，误差会以供电缺口的形式暴露出来，只能依靠 5 倍电价的紧急购电兜底。因此“预测”是本问绕不开的独立子问题，必须显式建模。

基于上述判断，本文按**“先预测、再决策、后结算”三段式**组织求解：**预测段**以 1 月实际数据为训练样本，逐日扩张窗口标定负载与光伏预测模型，在每天 0:00 给出当天 144 个间隔的功率预测值，并由历史预测残差估计其不确定性；**决策段**按负载低/中/高与光伏低/中/高组合构造 9 个情景，建立两阶段**随机线性规划**，一阶段决策（计划购电量与储能充放电计划）要求对所有情景同时可行，故与当日真值无关，目标函数为计划购电费与 5 倍电价下的期望紧急购电费之和；**结算段**在真实功率到达后，按既定充放电计划计算实际供电缺口，以 5 倍电价结算紧急购电量，富余电量计为弃电，日末储电量

传递给次日。计算以 2025 年 1 月 1 日 0:00 储电量 6000 kWh 为起点逐日滚动，全年 365 天中 1 月作为预热期，最终输出 2025 年 2 月 1 日至 12 月 31 日共 334 天的计划购电量、储能充放电量、紧急购电量与弃电量。具体分析流程如图9所示。



图 **9**　问题二分析流程图

5.2.2 模型准备

1. 数据预处理

附件二提供了 2025 年全年 365 天、每天 144 个 10 分钟时间点的小区负载与光伏实际功率数据，数据量约为 365×144 的两张二维矩阵。为便于逐日建模，需要对数据进行时间对齐、数值化与按天切分。设第 d 天第 k 个间隔的负载与光伏功率分别为 $L_{d,k}$ 与 $S_{d,k}$，则全年数据可组织为：

$$\boldsymbol{L} = [L_{d,k}]_{365\times144}, \quad \boldsymbol{S} = [S_{d,k}]_{365\times144} \tag{14}$$

其中，$d = 0, 1, \ldots, 364$ 表示天数序号，k 表示日内间隔序号。对两张矩阵逐元素执行数值化检查，全年缺失值为零，负载分布于 $1995.7 \sim 7978.9$ kW，光伏分布于 $0 \sim 10216.2$ kW。需要说明的是，预处理阶段仅作时间对齐、按天切分与预测模型标定所必需的组

织，不对真实功率序列作重采样、平滑与异常值替换，以免在数据中人为引入原始记录不具备的信息、掩盖真实的供需波动；预处理前后对比如图10所示。

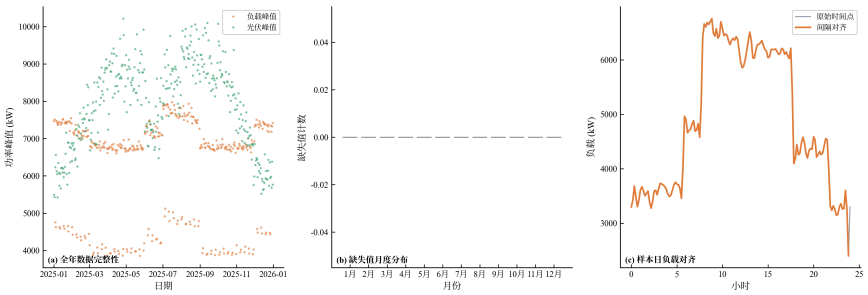


图 **10**　问题二全年数据预处理与完整性检查

由图10可知，全年负载与光伏峰值逐日变化，负载呈现明显的季节波动而光伏呈现冬低夏高的规律，且全年无缺失数据，样本日负载曲线在时间对齐后与原始时间点序列完全一致，数据质量良好。

需要说明的是，附件 2 的 365 天数据中，2025 年 1 月（1 月 1 日—1 月 31 日）承担**预热与训练双重作用**：一方面，模型以 1 月 1 日 0:00 储电量 6000 kWh 为初值逐日滚动，使储电量摆脱初始条件影响、进入稳定日循环（2 月 1 日 0:00 储电量已收敛至 1200 kWh 下限）；另一方面，1 月的实际功率与逐日预测残差被用于标定预测模型并估计其误差分布，是 2 月及以后各日制定计划的信息来源。由于 1 月不存在可用的历史信息，其自身按“无预测可用”处理（当天计划购电量取零、储能以初值支撑）。按照附件 5 的要求，结果文件与后文所有费用汇总均只统计 2025 年 2 月 1 日至 12 月 31 日共 334 天。

2. 算法选择

在完成数据预处理后，本问需在预测不确定性的条件下完成 334 个逐日决策问题，其求解策略涉及两个层面的选择：一是同一日内计划量应当如何确定，二是全年 334 天之间应当如何组织。就第一点而言，当天的负载与光伏在 0:00 未知，若按预测值作确定性优化，则等于把预测误差全部当作不发生，会在 5 倍紧急电价下系统性低估缺电风险；若将预测误差离散为若干情景、令计划量对所有情景同时可行，并在目标中计入期望紧急购电费，则可得到对不确定性稳健的计划量，这正是**两阶段随机线性规划**的思想。就第二点而言，可从建模复杂度与最优性两个维度比较独立逐日求解与全年统一求解两种策略，如表5所示。

表 **5**　问题二求解策略对比

策略	建模复杂度	计算规模	最优性
全年统一求解	高	大规模	严格全局最优，但规划时须假定全年功率已知
逐日滚动确定性求解	低	小规模	本问条件下与全年统一最优等价，但忽略预测误差
逐日滚动随机规划（本文采用）	中	小规模 × 9 情景	计划量不含事后信息，紧急购电费随预测误差产生

如5所示，全年统一求解虽在数学上严格，但变量规模达数十万，且要求规划时已知全年功率，与“0:00 预测、5 倍电价兜底”的题意不符。逐日滚动是否损失最优性，可作如下推理：本问每日电价完全相同，储电量跨日传递只能在次日相同电价下使用，还需额外承担约 19% 的往返损耗，因此日间耦合不产生任何收益，全年统一问题可分解为 365 个相互独立的逐日子问题，逐日滚动所得即全年统一最优解。表10～表13的实测结果进一步印证了该推理：四个代表日 0:00 与 24:00 储电量均恰为下限 1200 kWh，说明日间并无电量耦合需要协调。综合以上分析，本问选择**逐日滚动 + 情景随机规划**的三段式策略进行求解。

5.2.3 模型建立

问题二的核心困难在于信息的时间结构：微网在每天 0:00 制定当日购电与充放电计划时，既不知道当天的负载曲线 $L_{d,k}$，也不知道当天的光伏出力 $S_{d,k}$；而一旦实际功率不足，缺口必须以交易电价的 5 倍紧急购电。这意味着本问不能沿用问题一” 已知当日全部数据再求解” 的确定性框架。若在规划时直接代入附件二的真实值，由于计划购电与充放电可自由调节，最优解必然逐点覆盖真实缺口，供电缺口 $\delta_{d,k}$ 恒为零、紧急购电费用恒为零，”5 倍电价” 这一惩罚机制将完全失效，模型也就失去了对预测偏差的刻画能力，本质上属于对已知数据的” 事后拟合”。

因此，本节建立**” 先预测、再决策、后结算”** 的三段式模型：每日 0:00 仅利用截至前一日的全部历史数据，预测当日 144 个间隔的负载与光伏；由历史预测残差的经验分布构造有限的偏差情景，用两阶段随机规划在第一阶段（当日 0:00 一次性的确定性计划购电量与充放电功率）与第二阶段（情景依赖的紧急购电）之间进行权衡，得到兼顾预期费用与偏差风险的当日计划；待当日实际功率到达后，按真实缺口结算紧急购电费

用。三段之间以" 预测信息 $\rightarrow$ 计划决策 $\rightarrow$ 实际结算" 的顺序严格单向衔接，不使用任何当日未来的真实信息，从而避免了上述确定性模型的缺陷。具体过程如下。

步骤 1：预测模型的标定

设第 d 天的历史数据为 $\{L_{\tau,k}, S_{\tau,k}\}_{\tau<d}$，当日预测为

$$\hat{L}_{d,k} = \mathrm{lev}_d^L \cdot \mathrm{shp}_{d,k}^L, \qquad \hat{S}_{d,k} = \mathrm{lev}_d^S \cdot \mathrm{shp}_{d,k}^S, \qquad k = 0, 1, \ldots, 143 \tag{15}$$

即" 日水平 $\times$ 日内形状" 的乘积形式。负载具有明显的星期效应，故先按" 工作日/周末" 将历史日分为两类，取与目标日同类的最近 $k_s = 8$ 个历史日构造归一化日形状，并对其日均水平作近期加权：

$$\mathrm{shp}_{d,k}^L = \frac{1}{k_s} \sum_{\tau \in \mathcal{K}_d^L} \frac{L_{\tau,k}}{\bar{L}_\tau}, \qquad \bar{L}_\tau = \frac{1}{144} \sum_{k=0}^{143} L_{\tau,k} \tag{16}$$

$$\mathrm{lev}_d^L = \frac{\sum_{i=1}^{k_\ell} w_i \, \bar{L}_{\tau_i}}{\sum_{i=1}^{k_\ell} w_i}, \qquad w_i = 1 + \frac{i-1}{k_\ell - 1} \tag{17}$$

其中 $\tau_1 < \cdots < \tau_{k_\ell}$ 为同类历史日中最近的 $k_\ell = 2$ 天，权重 w_i 由 1 线性递增至 2，使越近的日期权重越大。光伏的日内形态对星期不敏感，故直接取最近 $k_s = 4$ 天的归一化形状均值与最近 $k_\ell = 4$ 天的加权日均水平相乘得到 $\hat{S}_{d,k}$。

上述标定采用**扩张窗口**：第 d 天的模型只使用 d 之前的数据，随着日期推进，可供标定的历史样本逐日增加。附件二的完整记录自 2025 年 1 月 1 日开始，1 月的 31 天数据正是用于标定上述预测模型的历史信息，因此本问的滚动优化统计区间取 2025 年 2 月 1 日至 12 月 31 日，共 334 天；1 月作为预测模型的预热与标定期，其结果不计入全年费用汇总。这一处理也与题目所给结果模板从 2 月 1 日起算的要求相一致。

步骤 2：预测误差分布与偏差情景构造

预测必然存在误差，而紧急购电正是对误差的兜底。为刻画误差，对每一天计算预测的相对残差：

$$\varepsilon_{d,k}^L = \frac{L_{d,k} - \hat{L}_{d,k}}{\max\{\hat{L}_{d,k}, 1\}}, \qquad \varepsilon_{d,k}^S = \frac{S_{d,k} - \hat{S}_{d,k}}{\max\{\hat{S}_{d,k}, 1\}} \quad (\hat{S}_{d,k} > 50) \tag{18}$$

其中光伏仅在预测出力大于 50 kW 时计入残差，以避免夜间零出力时刻的相对误差被过度放大。将滚动过程中已经发生的残差逐点存入残差池 $\mathcal{R}^L$、$\mathcal{R}^S$，并取经验分位数标定三档偏差因子：

$$\lambda_a^L = 1 + Q_{q_a}\big(\mathcal{R}^L\big), \qquad \lambda_b^S = 1 + Q_{q_b}\big(\mathcal{R}^S\big), \qquad (q_1, q_2, q_3) = (0.1, 0.5, 0.9) \tag{19}$$

其中 $Q_q(\cdot)$ 为经验分位数函数，λ^L 与 λ^S 分别裁剪至 $[0.7, 1.3]$ 与 $[0.5, 1.5]$ 以保证情景合理；当有效样本不足 50 个时退回到先验因子 $(0.95, 1.00, 1.05)$。由实测残差标定得到的三档因子均值为：负载 $\lambda^L = (0.9526, 0.9986, 1.0435)$，光伏 $\lambda^S = (0.9073, 1.0139, 1.1301)$，

可见光伏预测偏差明显大于负载，这与光伏出力受云量、辐照随机性影响的物理事实相符。

将负载与光伏的三档因子两两组合，得到 9 个偏差情景 $\omega_s = (\lambda_a^L, \lambda_b^S), a, b \in \{1, 2, 3\}$，情景概率取两档边缘概率的外积：

$$L_{d,k}^s = \lambda_a^L \, \hat{L}_{d,k}, \qquad S_{d,k}^s = \lambda_b^S \, \hat{S}_{d,k}, \qquad \pi_s = w_a \, w_b, \qquad (w_1, w_2, w_3) = (0.2, 0.6, 0.2) \tag{20}$$

即在” 负载低/中/高 $\times$ 光伏低/中/高” 共 9 种偏差组合下刻画当日可能出现的实际功率。

步骤 3：两阶段随机规划决策

在第一阶段，微网于当日 0:00 一次性确定计划购电功率 $g_{d,k}$、充电功率 $c_{d,k}$、放电功率 $d_{d,k}$ 与储电量 $E_{d,k}$，这些决策**不依赖情景**，即在 0:00 只能得到与情景无关的唯一计划；在第二阶段，当日实际功率揭晓后，各情景下的供电缺口 $\delta_{d,k}^s$ 按 5 倍电价结算。当日决策模型为

$$\min \; Z_d = \underbrace{\sum_{k=0}^{143} g_{d,k} \, p_k \, \Delta t}_{\text{计划购电费}} + \underbrace{\sum_{s=1}^{9} \pi_s \sum_{k=0}^{143} 5 \, p_k \, \delta_{d,k}^s \, \Delta t}_{\text{预期紧急购电费}} \tag{21}$$

$$\delta_{d,k}^s \geq L_{d,k}^s - \left(g_{d,k} + S_{d,k}^s + d_{d,k} - c_{d,k} \right), \qquad \delta_{d,k}^s \geq 0, \qquad \forall s, \; \forall k \tag{22}$$

式（7）中的取最大值关系 $\delta_{d,k}^s = \max\{\cdot, 0\}$ 已由” 下界约束 + 目标函数中系数为正” 共同线性化：$\delta_{d,k}^s$ 在最优点处必然取到下界，无需引入整数变量。储电量递推与容量、功率约束为

$$E_{d,k+1} = E_{d,k} + \eta \, c_{d,k} \, \Delta t - \frac{1}{\eta} \, d_{d,k} \, \Delta t \tag{23}$$

$$E_{\min} \leq E_{d,k} \leq E_{\max}, \qquad 0 \leq c_{d,k}, \, d_{d,k} \leq P_{\max}, \qquad g_{d,k} \geq 0 \tag{24}$$

其中 p_k 为附件一给出的与日期无关的分时电价，$\eta = 0.9$ 为充放电效率，$P_{\max} = 5000$ kW，$E_{\min} = 1200$ kWh，$E_{\max} = 10800$ kWh，$\Delta t = 1/6$ h。式（8）对全部情景唯一，体现了储电量轨迹的物理确定性（同一储能装置只有一条实际轨迹），而 $\delta_{d,k}^s$ 逐情景独立，体现了第二阶段随信息揭晓而” 因情制宜” 的适应能力。该模型即两阶段随机线性规划的标准形式 [5]。

步骤 4：实际运行结算与紧急购电、富余电量计算

当日实际负载 $L_{d,k}$ 与光伏 $S_{d,k}$ 到达后，储能按第一阶段的计划充放电功率执行，具体为：当实际功率不足时，缺口部分以 5 倍电价紧急购电补足

$$\delta_{d,k} = \max\left\{ L_{d,k} - \left(g_{d,k} + S_{d,k} + d_{d,k} - c_{d,k} \right), \, 0 \right\} \tag{25}$$

$$Z_d^{\text{em}} = \sum_{k=0}^{143} 5 \, p_k \, \delta_{d,k} \, \Delta t \tag{26}$$

当实际功率富余（如光伏出力超过预测）时，储能按计划功率继续充电直至容量上限，超出 $E_{\max}$ 的部分无法消纳，形成弃电

$$su_{d,k} = \max\left\{S_{d,k} + g_{d,k} + d_{d,k} - c_{d,k} - L_{d,k},\, 0\right\}\Delta t \tag{27}$$

当日计划购电费为 $Z_d^{\text{plan}} = \sum_k p_k\, g_{d,k}\,\Delta t$。式（10）与式（12）表明：在三段式框架下，预测偏差既可能表现为需要 5 倍价格兜底的供电缺口，也可能表现为无法消纳的光伏富余，二者均由实际值与预测值的偏差驱动，这正是该模型能够刻画紧急购电机制的关键。

步骤 5：储电量日间传递与全年费用汇总

储电量在全年范围内连续传递，当日末值即次日初值：

$$E_{d+1,0} = E_{d,144}, \qquad E_{0,0} = 6000 \tag{28}$$

与问题一要求的" 日循环（0:00 与 24:00 储电量相等）" 不同，问题二不设终端约束，储电量可跨日留存。但由于电价曲线日间完全相同，跨日留存只能在次日相同电价下使用，还需额外承担约 19% 的往返效率损耗（$1-0.9^2=0.19$），因此不存在跨日套利空间，最优解仍在日内完成" 低价时段充电、高价时段放电"。实测的四个指定日期 0:00 与 24:00 储电量均恰为下限 1200.00 kWh（见表10～表13），说明日内充放电空间已被充分利用，日间不存在需要额外协调的储电量耦合，逐日滚动求解在最优值上与全年统一求解等价[9, 13, 14]。

综上，问题二全年的总购电费用为

$$Z = \sum_{d=1}^{334}\left(Z_d^{\text{plan}} + Z_d^{\text{em}}\right) = \sum_{d=1}^{334}\sum_{k=0}^{143} p_k\, g_{d,k}\,\Delta t + \sum_{d=1}^{334}\sum_{k=0}^{143} 5\, p_k\, \delta_{d,k}\,\Delta t \tag{29}$$

模型按天递推求解 334 次，每次求解一个含 $13\times144+1=1873$ 个决策变量、9×144 条缺口约束的线性规划，全年总耗时约 10.6 s，既保证了计算效率，又通过储电量的逐日传递与残差池的逐日扩充实现了" 预测精度随数据积累而提高" 的滚动学习效果。

综上所述，本节完成了问题二模型的建立：先由扩张窗口标定负载与光伏预测模型，再由历史残差标定偏差情景并用两阶段随机规划制定当日计划，最后在实际功率到达后按 5 倍电价结算紧急购电与弃电，形成" 先预测、再决策、后结算" 的完整闭环；下一步将基于该模型给出求解结果。

5.2.4 模型求解

按“先预测、再决策、后结算”的三段式结构逐日推进：每天 0:00 由扩张窗口预报模型给出当天 144 个间隔的负载与光伏预测值，据此求解两阶段随机线性规划得到计划购电量与储能充放电计划；当天实际功率到达后，储能按既定计划执行，按式 (25) 结算紧急购电量、按式 (27) 统计弃电量；日末储电量作为状态传递至次日。2025 年 1 月作为**预测模型标定期与预热期**：1 月 1 日 0:00 储电量由附录给定为 6000 kWh，此后逐日

滚动，至 2 月 1 日 0:00 储电量已收敛至 1200 kWh 下限。按附件 5 口径，结果文件与费用汇总均只统计 2025 年 2 月 1 日至 12 月 31 日共 **334 天**。

该口径下的全年费用构成为:**计划购电量 21833065.92 kWh,计划购电费 13325494.59 元；紧急购电量 282910.68 kWh，紧急购电费 1057398.40 元；合计购电费 14382892.99 元**，日均约 43062.55 元，紧急购电量占计划购电量的 1.28%。334 天中共有 333 天出现紧急购电（占 99.70%），全年 144×334 个间隔中有 8609 个间隔出现紧急购电（占 17.90%），最大紧急购电功率 2143.48 kW，单日最大紧急购电量 12595.53 kWh。可见在 0:00 信息不完备的前提下，紧急购电并非偶发事件，而是每天都会发生、但总量很小（约 1%）的“兜底”电量，这正是 5 倍电价下随机规划给出的经济最优解。

按题目要求，以题目表 1 的格式给出四个指定日期在指定时间段的计划购电量及全天购电量与购电费（表6～ 表9），以题目表 2 的格式给出指定时间段的充放电量及 0:00 与 24:00 储电量（表10～ 表13），以题目表 3 的格式给出紧急购电的分段明细（表14）；完整结果已写入 `result2.xlsx` 的三张工作表。

首先检验预测段的精度。以 334 天全部 144 个间隔的真实值与预测值比对（样本 24222 个），负载预测的平均绝对误差为 130.24 kW、均方根误差为 179.00 kW，相对误差标准差 4.50%；光伏预测的平均绝对误差为 150.94 kW、均方根误差为 305.54 kW，相对误差标准差 9.95%。负载的相对误差分位数（$5\%, 10\%, \ldots, 95\%$）为 -32.32%、-4.68%、-2.86%、-1.75%、-0.87%、-0.06%、0.75%、1.63%、2.78%、4.68%、39.78%，即中间 90% 的间隔误差落在 $-5\% \sim 5\%$ 以内，仅夜间负荷极低时（分母很小）出现较大相对偏差；光伏的对应分位数为 -79.31%、-11.14%、-7.21%、-4.45%、-2.25%、-0.16%、1.86%、4.15%、6.97%、11.37%、95.36%，中间 80% 的间隔误差在 $-11\% \sim 11\%$ 以内，两端的极端值同样出现在日出、日落前后光伏出力接近零的时段。预测与实际功率的逐点对比见图11，相对误差分布见图12。

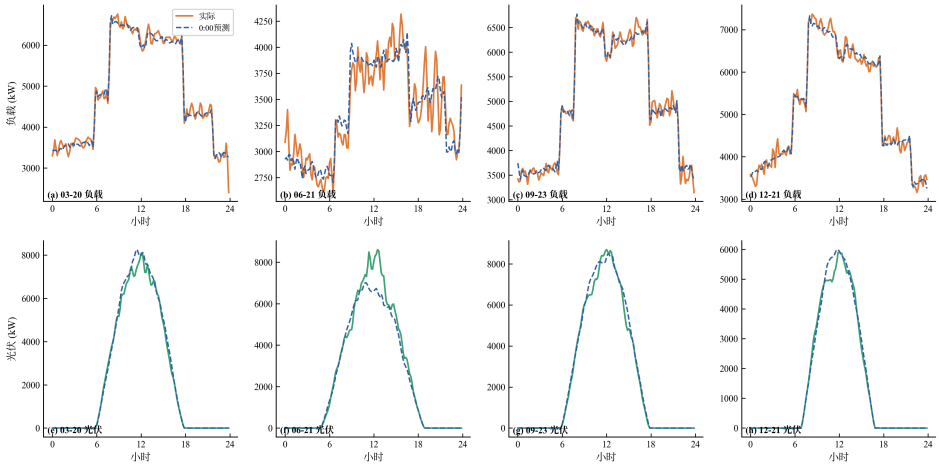


图 **11**　典型日负载与光伏的预测值与实际值对比

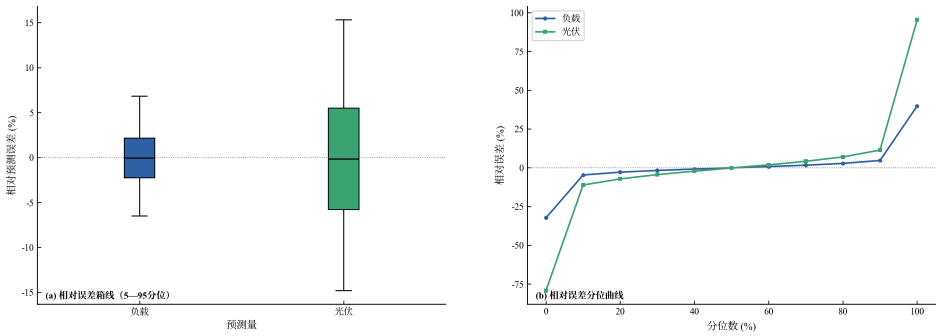


图 **12**　负载与光伏预测相对误差的分布

表 **6**　**2025.3.20** 微网在指定时间段的购电量及全天的购电量和购电费

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	535.11	14:00–14:10	0.00
16:00–16:10	506.24	18:00–18:10	712.75	20:00–20:10	0.00
全天购电量	68462.51	全天购电费	41502.92		

表 **7**　**2025.6.21** 微网在指定时间段的购电量及全天的购电量和购电费

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	0.00	14:00–14:10	0.00
16:00–16:10	159.03	18:00–18:10	0.00	20:00–20:10	0.00
全天购电量	34663.14	全天购电费	19596.21		

表 **8**　**2025.9.23** 微网在指定时间段的购电量及全天的购电量和购电费

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	528.89	14:00–14:10	0.00
16:00–16:10	561.39	18:00–18:10	807.53	20:00–20:10	11.96
全天购电量	72005.67	全天购电费	44887.63		

表 **9**　**2025.12.21** 微网在指定时间段的购电量及全天的购电量和购电费

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	954.41	14:00–14:10	0.00
16:00–16:10	844.76	18:00–18:10	742.25	20:00–20:10	0.00
全天购电量	98496.73	全天购电费	63507.75		

2.1—12.31 逐日购电费用及其 7 日滑动平均如图13所示。可以看出，购电费呈现明显的季节性波动：冬季（12 月）因光伏出力低、负荷高，购电费最高；夏季（6 月）因光伏出力充沛，购电费最低，与光伏资源的季节规律一致。

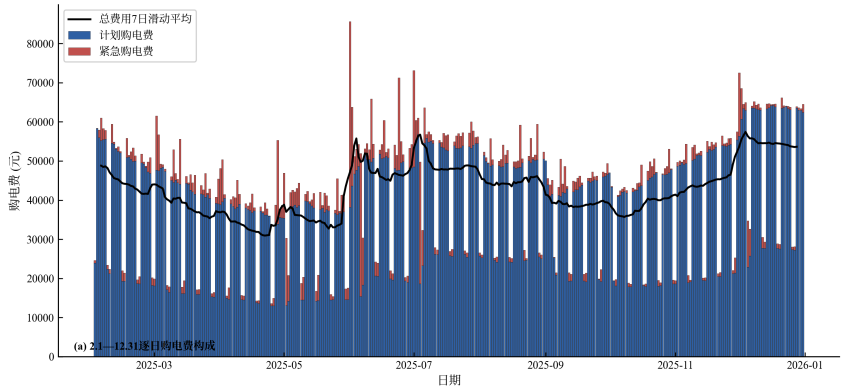


图 **13**　**2.1**—**12.31** 逐日购电费及 **7** 日滑动平均

四季典型日的购电功率曲线如图14所示。四个季节的购电功率均集中在夜间低电价时段，白天因光伏与储能放电的支撑，购电功率显著减小；冬季典型日购电功率整体最高，与表9中冬季购电费最高相吻合。

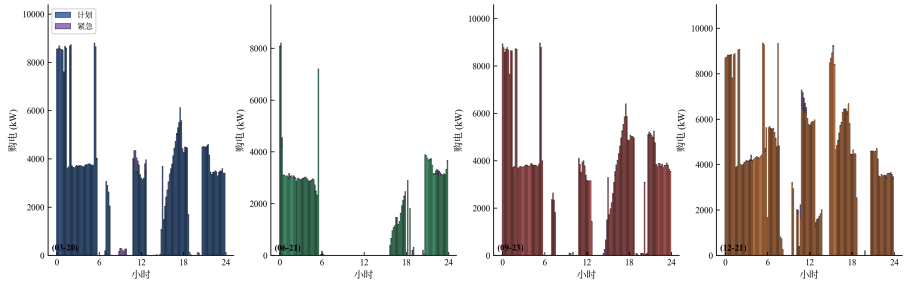


图 **14**　四季典型日的购电功率策略

四个指定日期的储能充放电结果如表10～ 表13所示。四日 0:00 与 24:00 储电量均回到 1200 kWh 下限，全天在 1200～10800 kWh 之间完成一次充放循环：00:00—04:00 与午间光伏富余时段充电，04:00—08:00 及 16:00—24:00 放电。四日 16:00—20:00 与 20:00

—24:00 的放电量合计均为 8640 kWh，即 $0.9 \times (10800 - 1200)$，说明每日晚间放出的电量恰好等于当日可用的储电量上限，“谷充峰放”被完全用尽。

表 **10**　**2025.3.20** 储能设备在指定时间段的充放电量及 **0:00** 和 **24:00** 的储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	9000.00	0.00	4:00–8:00	1666.67	5880.87
8:00–12:00	6050.98	2759.13	12:00–16:00	5029.62	335.29
16:00–20:00	0.00	5681.54	20:00–24:00	0.00	2958.46
0:00 储电量	1200.00		24:00 储电量	1200.00	

表 **11**　**2025.6.21** 储能设备在指定时间段的充放电量及 **0:00** 和 **24:00** 的储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	1857.78	0.00	4:00–8:00	862.32	2179.80
8:00–12:00	6229.23	0.00	12:00–16:00	4408.45	0.00
16:00–20:00	0.00	6124.20	20:00–24:00	0.00	2515.80
0:00 储电量	1200.00		24:00 储电量	1200.00	

表 **12**　**2025.9.23** 储能设备在指定时间段的充放电量及 **0:00** 和 **24:00** 的储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	9000.00	0.00	4:00–8:00	1666.67	5909.41
8:00–12:00	6150.33	2730.59	12:00–16:00	4809.30	237.30
16:00–20:00	0.00	5811.32	20:00–24:00	0.00	2828.68
0:00 储电量	1200.00		24:00 储电量	1200.00	

表 **13**　**2025.12.21** 储能设备在指定时间段的充放电量及 **0:00** 和 **24:00** 的储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	9000.00	0.00	4:00–8:00	2500.00	1508.33
8:00–12:00	5833.33	7806.67	12:00–16:00	8160.87	2695.30
16:00–20:00	0.00	5609.21	20:00–24:00	0.00	3030.79
0:00 储电量	1200.00		24:00 储电量	1200.00	

四季典型日的储能荷电状态轨迹如图15所示。储能荷电状态每日在 1200 kWh 与 10800 kWh 之间完成一次充放循环，白天光伏盈余时充电、夜间与傍晚放电，且全天严格处于约束区间内，验证了滚动模型与日末状态传递机制的可行性。

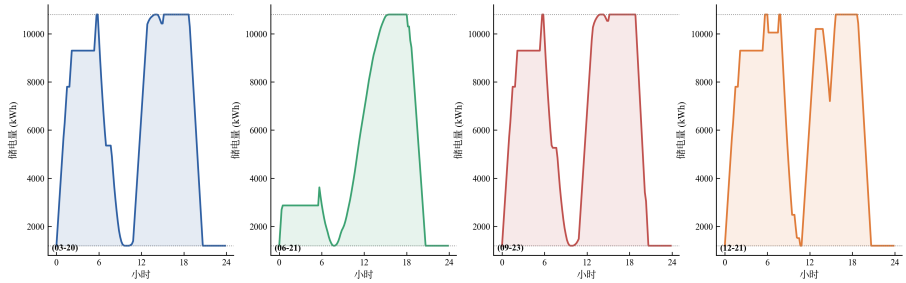


图 **15**　四季典型日储能荷电状态轨迹

2—12 月储能每日 0:00 储电量的月度分布如图16所示。各月储电量中位数均处于 1200 kWh 下限附近。由于每日电价相同、跨日留存电量还需额外承担约 19% 的往返损耗，日间不存在跨日套利空间，故日末储电量必然放至下限；本图与这一理论推断完全吻合，说明日内低充高放套利已被充分利用，而非模型的人为约束所致。

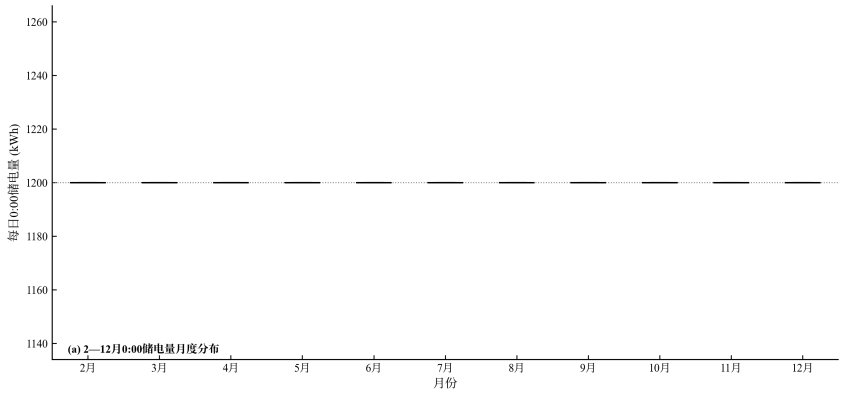


图 **16**　储能每日 **0:00** 储电量月度分布（**2**—**12** 月）

紧急购电的分段明细如表14所示。四个指定日期分别出现 16、11、15、10 段紧急购电，合计分别为 1400.66 kWh、424.89 kWh、583.54 kWh 和 830.33 kWh；单次连续段的时长多在 10~30 分钟之间，最长的单段为 12 月 21 日 10:20—11:40 的 608.79 kWh。从时段分布看，紧急购电集中在两类时刻：一是夜间与清晨（00:00—08:00）负荷预测偏低、而储能计划未安排放电的时段；二是傍晚（16:00—20:00）光伏出力衰减快于预测、储能放电已按计划结束的时段。图17进一步给出全年紧急购电功率的分布，可见其量级集中在 0~500 kW，且随月份变化，冬季整体高于夏季。

表 **14**　微网在指定日期的紧急购电量

2025.3.20		2025.6.21		2025.9.23		2025.12.21	
时间段	购电量	时间段	购电量	时间段	购电量	时间段	购电量
00:20–00:30	19.84	00:00–00:30	85.44	05:00–05:10	2.37	03:00–03:10	9.16
01:00–01:30	15.95	01:20–01:40	39.32	05:40–05:50	0.16	03:30–03:50	47.74
03:10–03:30	10.22	02:00–02:20	15.39	06:50–07:10	5.73	05:40–05:50	22.51
04:50–05:10	7.72	02:50–03:00	3.76	09:30–09:50	30.65	10:20–11:40	608.79
05:40–05:50	10.16	03:20–03:30	3.70	10:00–10:10	22.22	12:50–13:10	46.57
08:50–10:00	267.99	05:50–06:20	42.33	10:50–11:00	27.63	15:20–15:30	6.95
10:50–12:00	469.05	17:30–17:50	4.97	14:20–15:30	368.13	17:20–17:30	21.30
12:30–12:50	139.73	18:50–19:10	86.14	16:00–16:20	53.52	19:50–20:00	34.91
14:10–14:20	6.87	20:20–20:40	44.73	16:30–16:40	1.67	22:50–23:00	20.49
14:40–15:00	15.60	21:20–21:30	1.47	17:00–17:10	9.35	23:40–00:00	11.91
15:10–17:10	327.28	21:50–23:00	97.65	18:30–18:40	2.86		
18:40–19:10	32.59			20:10–20:30	5.84		
20:00–20:20	36.03			20:50–21:00	24.00		
21:20–21:30	7.01			21:50–22:00	9.28		
22:50–23:00	6.53			22:20–22:30	20.13		

表 **14**　微网在指定日期的紧急购电量（续）

2025.3.20		2025.6.21		2025.9.23		2025.12.21	
时间段	购电量	时间段	购电量	时间段	购电量	时间段	购电量
23:30–23:40	28.08						
合计	1400.66	合计	424.89	合计	583.54	合计	830.33

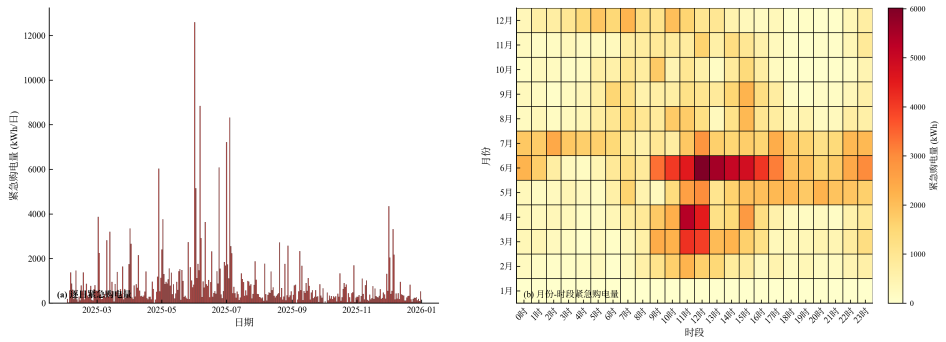


图 **17**　全年紧急购电量的分布

购电功率与净负荷的关系（2.1—12.31）及月度购电费（2—12 月）如图18所示。日均购电功率与日均净负荷呈显著正相关，负荷越大、光伏越小则购电越多；月度购电费在冬季达到峰值、夏季为谷值，进一步印证了模型的合理性。

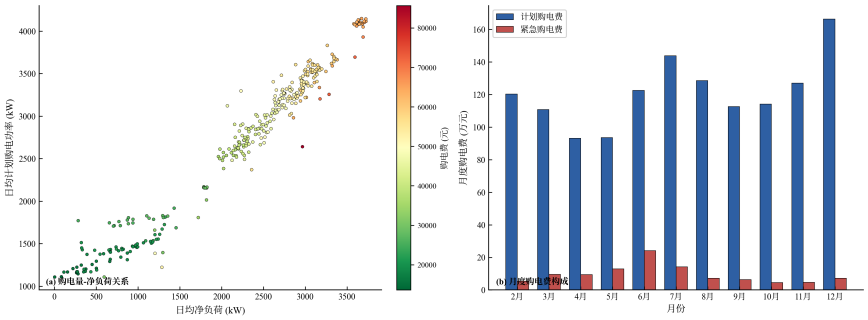


图 **18**　购电功率与净负荷的关系及月度购电费

综合以上结果，三段式框架下的逐日滚动随机规划得到了自洽且可解释的全年调度方案。334 天内净负荷合计 17939189.49 kWh，储能放电 5520412.99 kWh、充电 6815324.68 kWh，计划购电与紧急购电共同满足用电需求；计划段与紧急段之间的能量平衡校验为“净负荷 − 计划购电量 − 放电量 + 充电量 = −2598964.74 kWh”，恰等于“紧急购电量 282910.68 kWh 减弃电量 2881875.41 kWh”，说明逐日结算无遗漏。弃电量合计 2881875.41 kWh，主要来自午间光伏超发：光伏出力峰值可达 10216.2 kW，高于负荷峰

值 7978.9 kW，储能充满后富余电力无处消纳，其中约三分之二来自光伏侧，其余为光伏超出预测时计划购电形成的少量富余。紧急购电则集中在夜间与傍晚光伏不足时段，334 天中 333 天出现、覆盖 17.90% 的间隔，印证了预测偏差由 5 倍电价兜底、而非被事后信息抹平——这正是三段式框架区别于确定性优化的关键所在。

5.3 问题三的模型的建立和求解

5.3.1 具体分析

针对问题三，本问属于滚动预测下的随机调度与调整优化类问题，采用滚动时域线性规划方法，在 0:00、6:00、12:00 与 18:00 四个预报时刻依次制定计划购电与调整购电策略，并引入违约惩罚与超额电价机制。

本文先将附件三中四时刻发布的小时光伏预报插值到 10 分钟分辨率，形成可逐间隔使用的功率序列；在 0:00 依据最新预报制定全天计划购电策略；在 6:00、12:00、18:00 依据不断更新的预报对剩余时段进行滚动调整，其中计划购电量高于调整购电量的部分按交易电价 50% 计违约费用，调整购电量高于计划购电量的部分按交易电价 1.5 倍计价；最后以实际供电缺口核算 5 倍电价的紧急购电费用，得到全年总购电费用并分析引入其他预报时刻调整的收益（1 月为预热期，费用汇总仅统计 2 月 1 日—12 月 31 日）。具体分析流程如图19所示。



图 **19**　问题三分析流程图

5.3.2 模型准备

1. 数据预处理

附件三给出了 2025 年全年每天 0:00、6:00、12:00、18:00 四个预报时刻发布的未来 24 小时整点光伏功率预报，而附件二的负载与光伏实测数据均按 10 分钟采样，二者相差 6 倍的时间分辨率。为使预报与实测落在同一时间栅格上，本文对光伏预报作如下预处理。设某预报时刻 τ 发布的第 n 小时预报功率为 f_n，其对应时刻为 $\tau+n$（小时）；附件三未给出当前整点（τ 时刻）的预报，取 $f_0 \equiv f_1$ 近似，则预报序列在小时尺度上给出 25 个离散点 $(\tau, f_0), (\tau+1, f_1), \ldots, (\tau+24, f_{24})$。对第 k 个间隔末端时刻 $t_k = (k+1)/6$（小时）作线性插值，得到：

$$S_k^{\tau} = \mathrm{interp}\big(t_k;\ \{\tau+j\}_{j=0}^{24},\ \{f_j\}_{j=0}^{24}\big) \tag{30}$$

其中，S_k^{τ} 为预报时刻 τ 发布的第 k 个间隔的光伏预报功率，$\Delta t = 1/6$ 小时。需要说明的是，该插值仅用于在相邻整点之间构造连续过渡，不改变任何整点处的预报取值，也不引入预报时刻之外的新信息，因此不会对预报精度产生系统性的美化；唯一的近似是 $f_0 \equiv f_1$，它只影响 τ 至 $\tau+1$ 这一小时的过渡段，且相当于用相邻小时预报替代当前时

刻预报，属于偏保守的处理，其影响在第六章模型检验中定量讨论。各预报时刻的实际预报与真实光伏对比如图20所示。

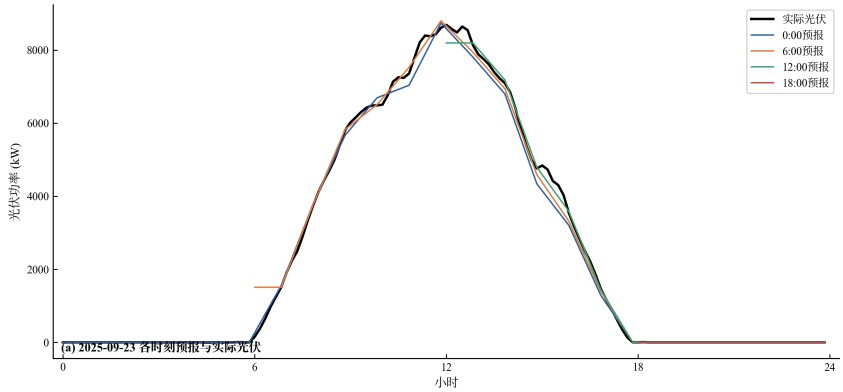


图 **20**　各预报时刻的光伏预报与真实光伏对比

由图20可知，四个预报时刻的光伏预报均能刻画白天光伏功率的单峰形态，且越接近实际发生时刻发布的预报越贴近真实光伏，这与光伏功率预测误差随预测时域缩短而减小的规律一致 [8, 16]，说明滚动利用最新预报能够有效降低预报误差，为调整购电策略提供了信息基础。

2. 算法选择

在完成预报插值后，本问需要在滚动时域内求解带分段惩罚费用的优化问题。计划购电量与调整购电量的偏差费用是分段线性凸函数，可借助上境变量将其线性化后仍用线性规划求解。可从优化精度与计算效率两个维度比较动态规划与线性规划，如表15所示。

表 **15**　问题三算法对比

算法	分段费用处理	全局最优性	计算效率
动态规划	需离散化	近似	一般
线性规划	上境变量线性化	严格保证	高

如15所示，动态规划需对连续购电量离散化，精度受步长限制；而分段凸费用可用上境变量精确线性化，再交给线性规划求解器求得全局最优解。综合以上分析，本问选择基于上境变量线性化的滚动线性规划策略进行求解。

5.3.3 模型建立

针对问题三，本节建立滚动时域下的计划购电与调整购电模型，并对计划与调整之间的偏差费用进行分段线性刻画。具体过程如下。

步骤 1：0:00 计划购电策略

在 0:00，微网依据当日 0:00 发布的光伏预报制定全天计划购电策略。设计划购电功率为 g_k^{p}，则计划模型为：

$$\min\ Z^{\mathrm{p}} = \sum_{k=0}^{143} g_k^{\mathrm{p}}\, p_k\, \Delta t \tag{31}$$

$$\text{s.t.}\quad g_k^{\mathrm{p}} + S_k^0 + d_k^{\mathrm{p}} - c_k^{\mathrm{p}} \geq L_k \tag{32}$$

$$E_{k+1}^{\mathrm{p}} = E_k^{\mathrm{p}} + \eta\, c_k^{\mathrm{p}}\, \Delta t - \frac{1}{\eta}\, d_k^{\mathrm{p}}\, \Delta t \tag{33}$$

$$E_{\min} \leq E_k^{\mathrm{p}} \leq E_{\max},\quad 0 \leq c_k^{\mathrm{p}}, d_k^{\mathrm{p}} \leq P_{\max} \tag{34}$$

其中，S_k^0 为 0:00 预报功率，E_0^{p} 为当日初始储电量，E_{144}^{p} 自由。

步骤 2：调整购电的偏差费用

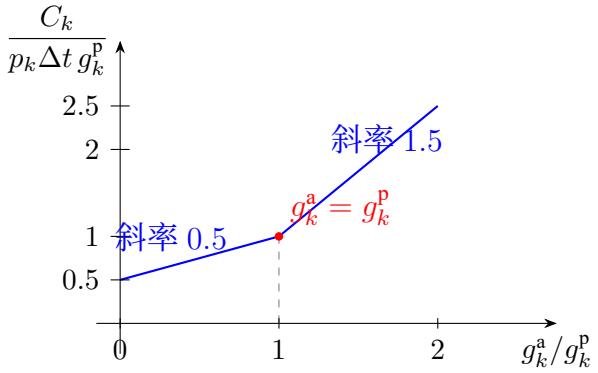
在后续预报时刻 $\tau \in \{6, 12, 18\}$，微网可对尚未执行的时段进行购电调整。设调整后购电功率为 g_k^{a}，计划购电量高于调整购电量的部分按交易电价的 50% 计违约费用，调整购电量高于计划购电量的部分按交易电价的 1.5 倍计价，则第 k 个间隔的购电费用为：

$$C_k = \Big[\min\{g_k^{\mathrm{p}}, g_k^{\mathrm{a}}\} + 0.5\max\{g_k^{\mathrm{p}} - g_k^{\mathrm{a}}, 0\} + 1.5\max\{g_k^{\mathrm{a}} - g_k^{\mathrm{p}}, 0\}\Big] p_k \Delta t \tag{35}$$

其中，第一项 $\min\{g_k^{\mathrm{p}}, g_k^{\mathrm{a}}\}$ 为计划购电费用，按**实际执行的购电量**结算，即计划购电量与调整购电量中的较小者，而非按计划购电量全额结算；第二、三项为调整购电量的相关费用。上式等价于如下分段形式：

$$\frac{C_k}{p_k \Delta t} = \begin{cases} 0.5\, g_k^{\mathrm{p}} + 0.5\, g_k^{\mathrm{a}}, & g_k^{\mathrm{a}} \leq g_k^{\mathrm{p}}, \\ 1.5\, g_k^{\mathrm{a}} - 0.5\, g_k^{\mathrm{p}}, & g_k^{\mathrm{a}} \geq g_k^{\mathrm{p}}. \end{cases}$$

下调购电时（$g_k^{\mathrm{a}} < g_k^{\mathrm{p}}$），左段可拆为 $g_k^{\mathrm{a}} + 0.5\,(g_k^{\mathrm{p}} - g_k^{\mathrm{a}})$：实际购电部分按 1 倍电价结算，未执行的差额部分按 0.5 倍计违约费；上调购电时（$g_k^{\mathrm{a}} > g_k^{\mathrm{p}}$），右段可拆为 $g_k^{\mathrm{p}} + 1.5\,(g_k^{\mathrm{a}} - g_k^{\mathrm{p}})$：计划内部分按 1 倍电价结算，超出部分按 1.5 倍电价加价。因此 C_k 关于 g_k^{a} 是分段线性凸函数，在 $g_k^{\mathrm{a}} = g_k^{\mathrm{p}}$ 处连续，两段斜率分别为 $0.5 p_k \Delta t$ 与 $1.5 p_k \Delta t$，其函数图像如下图所示。



分段购电费用随调整购电量的变化（g_k^{p} 固定）

费用口径说明：题目原文指出“总的购电费用包括计划购电费用、紧急购电费用和调整购电量的相关费用”。若将“计划购电费用”理解为按计划购电量全额结算，则下调购电时会出现重复计费：当 $g_k^{\mathrm{a}}=0$ 时用户未购任何电量却仍须支付 $1.5\,g_k^{\mathrm{p}}p_k\Delta t$，与“违约费仅针对未执行部分”的经济含义相悖。因此本文采用实际购电量结算口径：基础电费只按实际购电量 $\min\{g_k^{\mathrm{p}},g_k^{\mathrm{a}}\}$ 结算，未执行的差额部分按 0.5 倍计违约费、超出部分按 1.5 倍计价，从而保证“购多少电、付多少钱，偏差部分单独计价”的费用闭合性。

步骤 3：分段费用的上境线性化

为保持线性规划结构，引入上境变量 w_k，令其不小于分段费用的两段线性上界：

$$w_k \geq 0.5\,p_k\Delta t\,(g_k^{\mathrm{a}}+g_k^{\mathrm{p}}) \tag{36}$$

$$w_k \geq 1.5\,p_k\Delta t\,g_k^{\mathrm{a}}-0.5\,p_k\Delta t\,g_k^{\mathrm{p}} \tag{37}$$

由于目标为最小化 $\sum_k w_k$，最优解中 w_k 将精确等于两段上界的较大者，即 $w_k=C_k$。于是调整子问题在预报时刻 τ 覆盖的时段 $[\tau,24:00]$ 内最小化：

$$\min \sum_{k=6\tau}^{143} w_k \tag{38}$$

$$\text{s.t.}\quad g_k^{\mathrm{a}}+S_k^{\tau}+d_k^{\mathrm{a}}-c_k^{\mathrm{a}}\geq L_k \tag{39}$$

$$E_{k+1}^{\mathrm{a}}=E_k^{\mathrm{a}}+\eta\,c_k^{\mathrm{a}}\,\Delta t-\frac{1}{\eta}\,d_k^{\mathrm{a}}\,\Delta t \tag{40}$$

连同容量、功率约束及式 (33)(34) 的上境约束，初始储电量 $E_{6\tau}^{\mathrm{a}}$ 取上一决策执行到 τ 时刻的储电量。

步骤 4：紧急购电费用与全年总费用

设滚动调整后最终执行的购电功率为 g_k^{a}，最终放电、充电功率分别为 d_k^{a}、c_k^{a}，实际光伏功率为 S_k^{r}。紧急购电的本质是实际供电不足负载的缺口，即：

$$\delta_k=\max\Big\{L_k-\big(g_k^{\mathrm{a}}+S_k^{\mathrm{r}}+d_k^{\mathrm{a}}-c_k^{\mathrm{a}}\big),\,0\Big\} \tag{41}$$

该式表示：在最终调整策略执行后，实际光伏与充放电均按最终计划落实，微网供电仍不能满足负载的部分才需以 5 倍电价向外网紧急购电。特别地，若最终预报功率 S_k^{u}（由 0:00、6:00、12:00、18:00 各时段最终决策分段拼接得到）下的功率平衡取等号，即 $g_k^{\mathrm{a}}+S_k^{\mathrm{u}}+d_k^{\mathrm{a}}-c_k^{\mathrm{a}}=L_k$，则上式退化为 $\delta_k=\max\{S_k^{\mathrm{u}}-S_k^{\mathrm{r}},0\}$，此时缺口才近似等于最终光伏预报与实际之差；一般情况下调整购电与充放电会部分弥补光伏预报偏差，故紧急购电应按实际供电缺口而非光伏预报差核算。紧急购电费用为：

$$Z_k^{\mathrm{em}}=5\,p_k\,\delta_k\,\Delta t \tag{42}$$

全年总购电费用为计划购电费用、调整相关费用与紧急购电费用之和：

$$\min\ Z=\sum_{d=0}^{364}\sum_{k=0}^{143}\Big(C_{d,k}+5\,p_k\,\delta_{d,k}\Delta t\Big) \tag{43}$$

该模型将滚动决策分解为四个顺序求解的线性规划，其思想与模型预测控制中滚动时域优化框架一致 [7, 6, 10]，既降低了计算复杂度，又通过分段惩罚机制合理刻画了计划偏离的经济代价。

综上所述，本节完成了问题三滚动调整优化模型的建立，得到了计划购电、调整购电与紧急购电三层费用结构的完整数学表达，接下来将基于此进行求解。

5.3.4 模型求解

基于上述模型，本节利用附件一至附件三数据在滚动时域内顺序求解四个线性规划。同样以 1 月为预热期、费用汇总仅统计 2 月 1 日—12 月 31 日（334 天），该口径下含调整的总购电费用为 14337999.15 元；若不引入 6:00、12:00、18:00 的调整，仅执行 0:00 计划，总费用为 14782072.51 元。可见滚动调整使购电费用降低 444073.36 元，降幅约 3.00%。按题目要求，以表 1 的格式给出四个指定日期在指定时间段的计划购电量及全天计划购电量、计划购电费（表16～表19），以表 2 的格式给出储能设备在指定时间段的充放电量及 0:00 和 24:00 储电量（表20～表23），以表 3 的格式给出紧急购电量（表24）；四个指定日期的计划购电量、调整购电量与费用汇总见表25。

表 **16**　**2025.3.20** 微网在指定时间段的计划购电量及全天的购电量和购电费

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	560.43	14:00–14:10	0.00
16:00–16:10	663.99	18:00–18:10	698.00	20:00–20:10	0.00
全天购电量	67748.70	全天购电费	41072.00		

表 **17**　**2025.6.21** 微网在指定时间段的计划购电量及全天的购电量和购电费

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	0.00	14:00–14:10	0.00
16:00–16:10	111.20	18:00–18:10	0.00	20:00–20:10	0.00
全天购电量	32640.60	全天购电费	18056.00		

表 **18**　**2025.9.23** 微网在指定时间段的计划购电量及全天的购电量和购电费

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	371.59	14:00–14:10	0.00

表 **18**　**2025.9.23** 微网在指定时间段的计划购电量及全天的购电量和购电费（续）

时间段	购电量	时间段	购电量	时间段	购电量
16:00–16:10	636.70	18:00–18:10	764.33	20:00–20:10	0.00
全天购电量	68433.20	全天购电费	42736.00		

表 **19**　**2025.12.21** 微网在指定时间段的计划购电量及全天的购电量和购电费

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	979.54	14:00–14:10	0.00
16:00–16:10	799.69	18:00–18:10	715.41	20:00–20:10	0.00
全天购电量	93633.20	全天购电费	59704.00		

表 **20**　**2025.3.20** 储能设备在指定时间段的充放电量及 **0:00** 和 **24:00** 的储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	9000.00	0.00	4:00–8:00	1666.67	6149.96
8:00–12:00	4106.95	2313.92	12:00–16:00	6651.73	250.65
16:00–20:00	0.00	5709.78	20:00–24:00	0.00	2930.22
0:00 储电量	1200.00		24:00 储电量	1200.00	

表 **21**　**2025.6.21** 储能设备在指定时间段的充放电量及 **0:00** 和 **24:00** 的储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	1409.41	0.00	4:00–8:00	850.39	1830.44
8:00–12:00	1812.59	0.00	12:00–16:00	8722.90	0.00
16:00–20:00	131.18	6154.10	20:00–24:00	0.00	2485.90
0:00 储电量	1200.00		24:00 储电量	1200.00	

表 **22**　**2025.9.23** 储能设备在指定时间段的充放电量及 **0:00** 和 **24:00** 的储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	9000.00	0.00	4:00–8:00	1666.67	6435.18

表 **22**　**2025.9.23** 储能设备在指定时间段的充放电量及 **0:00** 和 **24:00** 的储电量（续）

时间段	充电量	放电量	时间段	充电量	放电量
8:00–12:00	5210.49	1920.52	12:00–16:00	5495.20	315.91
16:00–20:00	0.00	5602.91	20:00–24:00	0.00	3037.09
0:00 储电量	1200.00		24:00 储电量	1200.00	

表 **23**　**2025.12.21** 储能设备在指定时间段的充放电量及 **0:00** 和 **24:00** 的储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	9000.00	0.00	4:00–8:00	2499.34	1510.13
8:00–12:00	5833.33	7579.05	12:00–16:00	7558.00	2432.26
16:00–20:00	0.00	5797.47	20:00–24:00	0.00	2842.53
0:00 储电量	1200.00		24:00 储电量	1200.00	

表 **24**　微网在指定日期的紧急购电量

2025.3.20		2025.6.21		2025.9.23		2025.12.21	
时间段	购电量	时间段	购电量	时间段	购电量	时间段	购电量
06:00–12:00	2497.25	04:00–04:50	2.34	06:00–06:50	692.04	07:00–08:10	67.18
12:30–14:20	545.00	05:00–05:30	12.59	07:10–08:00	61.88	08:50–12:00	829.59
14:40–15:10	35.16	06:00–06:10	140.85	08:10–09:00	64.49	13:00–13:10	2.28
15:20–16:20	119.81	06:20–07:20	360.87	09:50–10:20	46.53	16:30–16:50	13.69
16:50–17:50	77.14	13:00–13:10	1.48	10:40–11:00	48.04		
		16:00–16:20	5.78	11:40–12:00	57.68		
		17:20–17:30	0.50	12:50–14:00	121.33		

表 **24**　微网在指定日期的紧急购电量（续）

2025.3.20		2025.6.21		2025.9.23		2025.12.21	
时间段	购电量	时间段	购电量	时间段	购电量	时间段	购电量
		19:00–19:50	2.58	14:20–15:00	92.64		
				15:50–16:30	24.37		
				17:00–17:50	64.93		

表 **25**　问题三指定日期的计划购电量、调整购电量与费用汇总

日期	计划购电量	调整购电量	计划费用	紧急费用
2025.3.20	67748.70	63217.60	41072.00	12856.00
2025.6.21	32640.60	33140.10	18056.00	2294.00
2025.9.23	68433.20	65716.30	42736.00	5600.00
2025.12.21	93633.20	93783.50	59704.00	2875.00

计划购电与滚动调整后购电功率的对比（以 9 月 23 日为例）如图21所示。可以看到，调整后购电功率在白天光伏预报偏高的时段被明显下调，减少了高价时段的无效购电；在预报偏低的时段则小幅上调，使实际供电缺口显著缩小。

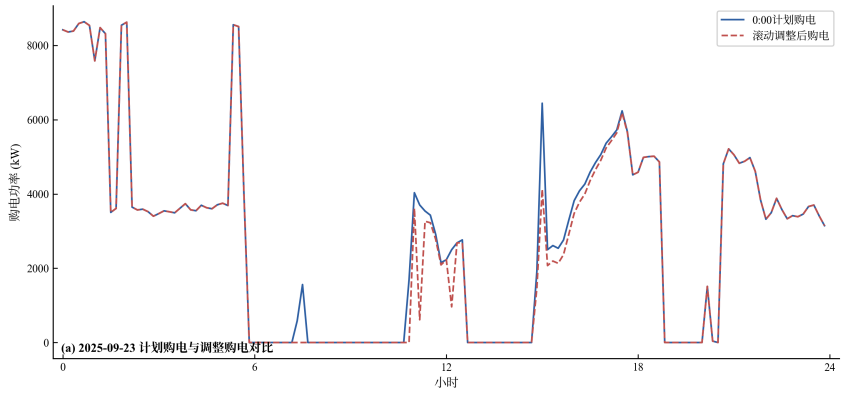


图 **21**　典型日计划购电与调整购电功率对比

由调整后策略生成的 2.1—12.31 逐日紧急购电量如图22所示。紧急购电量在多数日期维持在较低水平，仅在实际供电出现缺口的个别日期出现尖峰，说明滚动调整已有效

吸收了大部分预报误差，使实际供电缺口显著缩小。

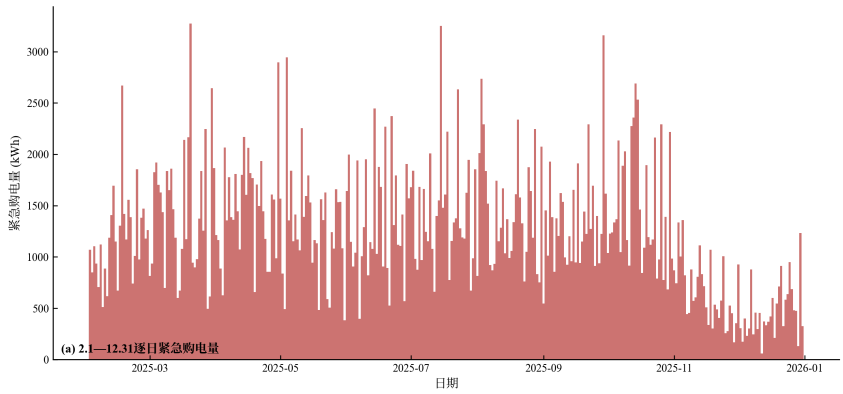


图 **22**　**2.1—12.31** 逐日紧急购电量

有无调整的费用对比与月度节省如图23所示。引入滚动调整后,总费用由 14782072.51 元降至 14337999.15 元，各月均有不同程度的费用节省，说明利用 6:00、12:00、18:00 的新预报进行购电调整具有明确的经济收益。

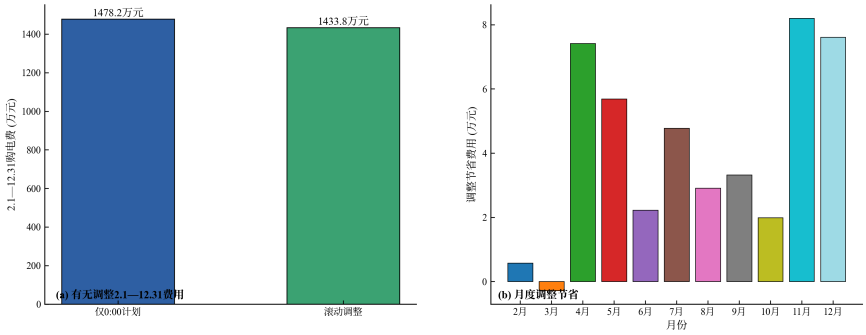


图 **23**　有无滚动调整的费用对比与月度节省（**2.1—12.31**）

典型日的储能荷电状态在计划与调整后两种策略下的对比如图24所示。调整后储能充放电轨迹随更新预报自适应变化，荷电状态全程维持在约束区间内，且 0:00 与 24:00 的储电量仍保持合理衔接，说明调整策略在提升经济性的同时未破坏储能的运行约束。

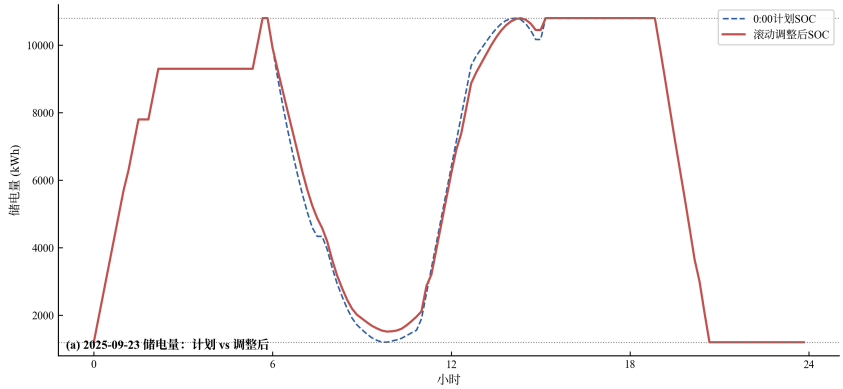


图 **24**　典型日计划与调整后储能荷电状态对比

2—12 月费用的月度构成如图25所示，计划购电费用占主导，调整相关费用与紧急购电费用占比相对较小，且冬季总费用最高、夏季最低，与负荷和光伏的季节特性一致。

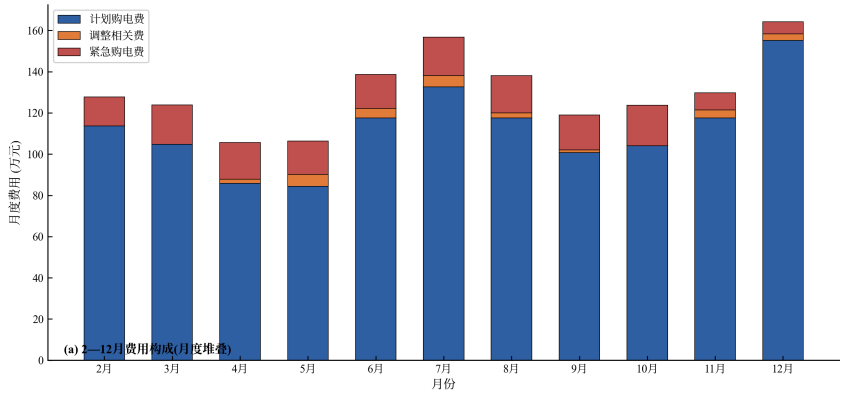


图 **25**　**2.1**—**12.31** 费用月度构成

为完整回答“是否需要引入其他时刻的预报”这一问，本文在 0/6/12/18 方案基础上进一步做了两类边际收益分析（口径均为 2 月 1 日—12 月 31 日）。第一类为逐次累加题目给定的真实预报时刻：仅用 0:00 计划时费用为 14782072.51 元；依次引入 6:00、12:00、18:00 调整后，费用分别变为 14819461.95 元、14357950.79 元与 14337999.15 元，三个时刻的边际节省分别为 −3.74 万元、46.15 万元与 2.00 万元，其中 12:00 预报的价值最为突出（6:00 预报单独引入时边际收益略为负值，说明并非所有新增预报时刻都具有正收益）。第二类为“加密更多时刻”的上界分析：由于附件三只提供 0:00、6:00、12:00、18:00 四个时刻的预报，本文以“各时刻预报完全准确（完全预见）”作为最乐观假设，考察把调整间隔由 6 小时加密到 3 小时、1 小时，以及在 6 小时方案中增加 9:00、15:00 时刻的额外收益，结果如图26所示。

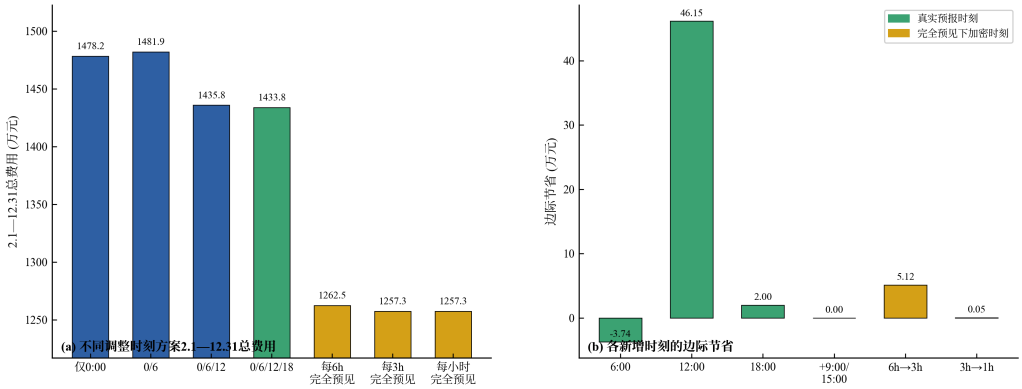


图 **26**　不同调整时刻方案的总费用与各新增时刻的边际节省（**2.1**—**12.31**）

在完全预见下，6 小时间隔已几乎捕获全部频率收益：把间隔进一步加密到 3 小时、1 小时仅分别再节省 5.12 万元与 0.05 万元，而在 6 小时方案中增加 9:00、15:00 时刻的边际收益为 0；与之相对，若保持 6 小时间隔不变、仅把 6/12/18 的真实预报替换为完全预见，费用还可再降 171.33 万元。这说明未实现的节省潜力主要来自光伏预报精度而非调整频率。

综合以上结果：引入 6:00、12:00、18:00 的预报制定调整购电策略是必要的，可使购电费用较仅用 0:00 计划降低约 3.00%；但进一步引入更多预报时刻（如每小时、每 3 小时，或增加 9:00、15:00 等）并无必要——即使在完全预见的最乐观假设下，加密调整间隔的边际收益也趋近于零（0.05 万～5.12 万元量级）。因此，若需进一步降低购电费用，应优先提高各预报时刻的预报精度，而非增加预报时刻数量。

5.4 问题四的模型的建立和求解

5.4.1 具体分析

针对问题四，本问属于波动电价下的随机调度优化类问题，在问题二、问题三模型的基础上，将固定的分时电价替换为附件四提供的实时波动电价，重新计算逐日计划购电与滚动调整购电策略。

本文先读取附件四中全年 365 天、每天 144 个 10 分钟时间点的波动电价数据，与附件二的实际负载光伏、附件三的光伏预报完成时间对齐；将问题二、问题三模型中的电价参数替换为逐间隔波动电价，重新求解逐日线性规划与滚动调整线性规划；分别得到波动电价下问题二的计划购电策略与问题三的调整购电策略；最后与固定电价结果进行对比，分析电价波动对微网购电费用的影响。具体分析流程如图27所示。



图 **27**　问题四分析流程图

5.4.2 模型准备

1. 数据预处理

附件四提供了 2025 年全年每天不同时刻的外网实时波动电价，时间分辨率为 10 分钟。设第 d 天第 k 个间隔的波动电价为 $p_{d,k}$，则全年电价数据可组织为：

$$\boldsymbol{P} = [p_{d,k}]_{365\times144} \tag{44}$$

对其进行数值化检查，全年无缺失值，电价分布范围约为 $0.3 \sim 1.6$ 元/kWh。波动电价与固定电价的典型日对比如图28所示。

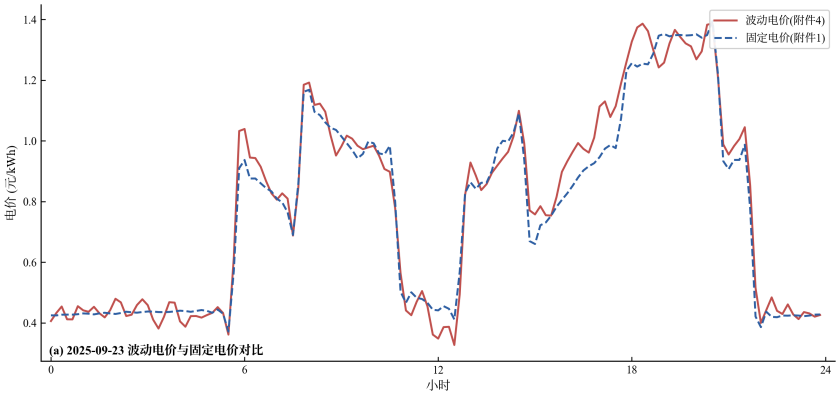


图 **28**　波动电价与固定电价的典型日对比

由图28可知，波动电价在同一天内剧烈起伏，白天用电高峰电价可达 1.5 元/kWh 以上，夜间低谷可低至 0.35 元/kWh 左右，其峰谷价差明显大于固定电价，因此储能低充高放的套利空间更大，但购电时机的把握也更为关键。

2. 算法选择

在完成电价数据预处理后，本问的建模框架与问题二、问题三完全一致，仅将电价参数由固定序列替换为逐间隔波动序列。由于目标函数与约束仍是线性的，问题二对应部分可继续采用逐日线性规划，问题三对应部分可继续采用滚动线性规划，算法选择与前述问题一致，此处不再重复对比，仅对参数替换作说明：将式 (1) 中的 p_k 替换为 $p_{d,k}$ 即可得到波动电价下的模型。

5.4.3 模型建立

针对问题四，本节在问题二、问题三模型的基础上，将固定分时电价替换为逐间隔波动电价，建立波动电价下的微网购电策略模型。具体过程如下。

步骤 1：波动电价下的逐日计划模型

对应问题二，第 d 天的逐日计划购电模型的目标函数为：

$$\min\ Z_d = \sum_{k=0}^{143} g_{d,k}\, p_{d,k}\, \Delta t \tag{45}$$

功率平衡、储能状态递推、容量与功率约束分别保持为：

$$g_{d,k} + S_{d,k} + d_{d,k} - c_{d,k} \geq L_{d,k} \tag{46}$$

$$E_{d,k+1} = E_{d,k} + \eta\, c_{d,k}\, \Delta t - \frac{1}{\eta}\, d_{d,k}\, \Delta t \tag{47}$$

$$E_{\min} \leq E_{d,k} \leq E_{\max}, \quad 0 \leq c_{d,k}, d_{d,k} \leq P_{\max}, \quad g_{d,k} \geq 0 \tag{48}$$

日间储电量传递关系与首日初值同问题二：

$$E_{d+1,0} = E_{d,144}, \quad E_{0,0} = 6000 \tag{49}$$

步骤 2：波动电价下的滚动调整模型

对应问题三，0:00 计划购电的目标函数与约束为：

$$\min\ Z^{\mathrm{p}} = \sum_{k=0}^{143} g_k^{\mathrm{p}}\, p_{d,k}\, \Delta t \tag{50}$$

$$g_k^{\mathrm{p}} + S_k^{0} + d_k^{\mathrm{p}} - c_k^{\mathrm{p}} \geq L_k \tag{51}$$

调整子问题的分段费用上境约束改写为：

$$w_k \geq 0.5\, p_{d,k} \Delta t\, (g_k^{\mathrm{a}} + g_k^{\mathrm{p}}) \tag{52}$$

$$w_k \geq 1.5\, p_{d,k} \Delta t\, g_k^{\mathrm{a}} - 0.5\, p_{d,k} \Delta t\, g_k^{\mathrm{p}} \tag{53}$$

紧急购电能量按实际供电缺口核算，即最终调整购电 g_k^{a}、最终充放电与真实光伏下仍不能满足负载的部分，紧急购电能量与费用分别为：

$$\delta_k = \max\left\{ L_k - \left(g_k^{\mathrm{a}} + S_k^{\mathrm{r}} + d_k^{\mathrm{a}} - c_k^{\mathrm{a}}\right),\, 0 \right\} \tag{54}$$

$$Z_k^{\mathrm{em}} = 5\, p_{d,k}\, \delta_k\, \Delta t \tag{55}$$

步骤 3：全年总费用

波动电价下，问题二对应的全年总购电费用与问题三对应的全年总费用分别为：

$$Z^{(2)} = \sum_{d=0}^{364} Z_d \tag{56}$$

$$Z^{(3)} = \sum_{d=0}^{364} \sum_{k=0}^{143} \Big(C_{d,k} + 5\, p_{d,k}\, \delta_{d,k} \Delta t \Big) \tag{57}$$

其中，$C_{d,k}$ 为式 (32) 定义的分段购电费用。该模型在保持线性规划结构不变的前提下，通过电价参数 $p_{d,k}$ 的逐间隔替换，完整刻画了实时波动电价对微网购电策略的影响。

综上所述，本节完成了波动电价下问题二与问题三两个模型的建立，得到了逐日计划与滚动调整两套策略的完整数学表达，接下来将基于此进行求解。

5.4.4 模型求解

基于上述模型，本节利用附件四的波动电价数据重新求解问题二与问题三（同样以 1 月为预热期、费用汇总仅统计 2 月 1 日—12 月 31 日）。问题二沿用在 0:00 预测、按 5 倍紧急电价制定计划购电、实际运行后滚动结算紧急购电的三段式框架：全年 334 天计划购电费 13895479.01 元、紧急购电 304292.65 kWh、紧急购电费 1181652.65 元，合计 15077131.66 元；问题三在该电价下的滚动优化总费用为 14956526.18 元。波动电价下问题二（对应 result4-2）四个指定日期在指定时间段的计划购电量及全天计划购电量、计划购电费见表26～表29，储能充放电量见表30～表33，紧急购电见表34；波动电价下

问题三（对应 result4-3）的计划购电量见表35～表38，储能充放电量见表39～表42，紧急购电见表43。四个指定日期在波动电价下的购电费用汇总见表44。与固定电价情形一致，由于负载与光伏预测存在误差，四个指定日期同样出现计划外偏差，其紧急购电时段与电量见表34。

表 **26**　**2025.3.20** 微网在指定时间段的购电量及全天的购电量和购电费

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	535.11	14:00–14:10	0.00
16:00–16:10	506.24	18:00–18:10	0.00	20:00–20:10	0.00
全天计划购电量	68480.76	全天计划购电费	42307.23	全天紧急购电量	1506.23
全天紧急购电费	5386.82	全天合计购电费	47694.05		

表 **27**　**2025.6.21** 微网在指定时间段的购电量及全天的购电量和购电费

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	0.00	14:00–14:10	0.00
16:00–16:10	159.03	18:00–18:10	225.46	20:00–20:10	0.00
全天计划购电量	35093.50	全天计划购电费	16867.61	全天紧急购电量	424.89
全天紧急购电费	1499.51	全天合计购电费	18367.12		

表 **28**　**2025.9.23** 微网在指定时间段的购电量及全天的购电量和购电费

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	528.89	14:00–14:10	0.00
16:00–16:10	561.39	18:00–18:10	0.00	20:00–20:10	845.29
全天计划购电量	71715.57	全天计划购电费	45906.41	全天紧急购电量	707.03
全天紧急购电费	3131.71	全天合计购电费	49038.12		

表 **29**　**2025.12.21** 微网在指定时间段的购电量及全天的购电量和购电费

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	954.41	14:00–14:10	0.00
16:00–16:10	844.76	18:00–18:10	742.25	20:00–20:10	0.00
全天计划购电量	98238.82	全天计划购电费	74152.25	全天紧急购电量	875.82
全天紧急购电费	3881.15	全天合计购电费	78033.40		

表 **30**　**2025.3.20** 储能设备在指定时间段的充放电量及 **0:00** 和 **24:00** 的储电量

时间段	充电量	放电量	时间段	充电量	放电量
00:00–04:00	6666.67	0.00	04:00–08:00	4000.00	5829.15
08:00–12:00	6050.98	2810.85	12:00–16:00	5029.62	335.29
16:00–20:00	0.00	6416.42	20:00–24:00	833.33	2898.58
0:00 储电量	1200.00		24:00 储电量	1200.00	

表 **31**　**2025.6.21** 储能设备在指定时间段的充放电量及 **0:00** 和 **24:00** 的储电量

时间段	充电量	放电量	时间段	充电量	放电量
00:00–04:00	1666.67	1350.00	04:00–08:00	2690.00	2155.42
08:00–12:00	6251.06	0.00	12:00–16:00	4386.62	0.00
16:00–20:00	0.00	6124.20	20:00–24:00	756.82	3128.83
0:00 储电量	1200.00		24:00 储电量	1200.00	

表 **32**　**2025.9.23** 储能设备在指定时间段的充放电量及 **0:00** 和 **24:00** 的储电量

时间段	充电量	放电量	时间段	充电量	放电量
00:00–04:00	5833.33	0.00	04:00–08:00	4833.33	6060.24
08:00–12:00	5907.57	2579.76	12:00–16:00	4932.08	140.12
16:00–20:00	0.00	6973.33	20:00–24:00	0.00	1666.67
0:00 储电量	1200.00		24:00 储电量	1200.00	

表 **33**　**2025.12.21** 储能设备在指定时间段的充放电量及 **0:00** 和 **24:00** 的储电量

时间段	充电量	放电量	时间段	充电量	放电量
00:00–04:00	9000.00	0.00	04:00–08:00	1666.67	833.33
08:00–12:00	5833.33	7806.67	12:00–16:00	7500.00	2695.30
16:00–20:00	660.87	5275.65	20:00–24:00	0.00	3364.35
0:00 储电量	1200.00		24:00 储电量	1200.00	

表 **34**　微网在指定日期的紧急购电量

2025.3.20		2025.6.21		2025.9.23		2025.12.21	
时间段	购电量	时间段	购电量	时间段	购电量	时间段	购电量
00:20–00:30	19.84	00:00–00:30	85.44	05:00–05:10	2.37	03:00–03:10	40.17
01:00–01:30	15.95	01:20–01:40	39.32	05:40–05:50	0.16	03:30–03:50	79.16
03:10–03:30	10.22	02:00–02:20	15.39	06:50–07:10	5.73	05:10–05:20	5.57
04:50–05:10	7.72	02:50–03:00	3.76	09:30–09:50	30.65	10:20–11:40	608.79
05:40–05:50	10.16	03:20–03:30	3.70	10:00–10:10	22.22	12:50–13:10	46.57
09:00–10:00	195.10	05:50–06:20	42.33	10:50–11:00	27.63	15:20–15:30	6.95
10:50–12:00	616.10	17:30–17:50	4.97	14:20–15:30	462.47	17:20–17:30	21.30
12:30–12:50	177.69	18:50–19:10	86.14	16:00–16:20	53.52	19:50–20:00	34.91
14:10–14:20	6.87	20:20–20:40	44.73	16:30–16:40	1.67	22:50–23:00	20.49
14:40–15:00	15.60	21:20–21:30	1.47	17:00–17:10	9.35	23:40–00:00	11.91
15:10–17:10	327.28	21:50–23:00	97.65	18:30–18:40	2.86		

表 **34**　微网在指定日期的紧急购电量（续）

2025.3.20		2025.6.21		2025.9.23		2025.12.21	
时间段	购电量	时间段	购电量	时间段	购电量	时间段	购电量
18:40–19:10	32.59			20:10–20:30	5.84		
20:00–20:20	36.03						
21:20–21:30	7.01						
23:30–23:40	28.08						
合计	1506.23	合计	424.89	合计	707.03	合计	875.82

表 **35**　**2025.3.20** 微网在指定时间段的计划购电量及全天的购电量和购电费

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	560.43	14:00–14:10	0.00
16:00–16:10	663.99	18:00–18:10	0.00	20:00–20:10	0.00
全天购电量	67887.93	全天购电费	41997.87		

表 **36**　**2025.6.21** 微网在指定时间段的计划购电量及全天的购电量和购电费

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	0.00	14:00–14:10	0.00
16:00–16:10	111.20	18:00–18:10	0.00	20:00–20:10	0.00
全天购电量	33099.44	全天购电费	15620.14		

表 **37**　**2025.9.23** 微网在指定时间段的计划购电量及全天的购电量和购电费

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	371.59	14:00–14:10	0.00
16:00–16:10	636.70	18:00–18:10	0.00	20:00–20:10	827.18
全天购电量	68433.17	全天购电费	44112.85		

表 **38**　**2025.12.21** 微网在指定时间段的计划购电量及全天的购电量和购电费

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	979.54	14:00–14:10	0.00
16:00–16:10	799.69	18:00–18:10	715.41	20:00–20:10	0.00
全天购电量	93474.87	全天购电费	69824.09		

表 **39**　**2025.3.20** 储能设备在指定时间段的充放电量及 **0:00** 和 **24:00** 的储电量

时间段	充电量	放电量	时间段	充电量	放电量
00:00–04:00	6666.67	0.00	04:00–08:00	4000.00	6149.96
08:00–12:00	4106.95	2313.92	12:00–16:00	6651.73	250.65
16:00–20:00	0.00	6412.35	20:00–24:00	833.33	2902.65
0:00 储电量	1200.00		24:00 储电量	1200.00	

表 **40**　**2025.6.21** 储能设备在指定时间段的充放电量及 **0:00** 和 **24:00** 的储电量

时间段	充电量	放电量	时间段	充电量	放电量
00:00–04:00	1666.67	1350.00	04:00–08:00	2242.74	1816.62
08:00–12:00	1812.59	0.00	12:00–16:00	8839.57	0.00
16:00–20:00	14.50	6154.10	20:00–24:00	748.18	3091.92
0:00 储电量	1200.00		24:00 储电量	1200.00	

表 **41**　**2025.9.23** 储能设备在指定时间段的充放电量及 **0:00** 和 **24:00** 的储电量

时间段	充电量	放电量	时间段	充电量	放电量
00:00–04:00	5833.33	0.00	04:00–08:00	4833.33	6435.18
08:00–12:00	5130.10	1920.52	12:00–16:00	5575.59	315.91
16:00–20:00	0.00	7021.77	20:00–24:00	0.00	1618.23
0:00 储电量	1200.00		24:00 储电量	1200.00	

表 **42**　**2025.12.21** 储能设备在指定时间段的充放电量及 **0:00** 和 **24:00** 的储电量

时间段	充电量	放电量	时间段	充电量	放电量
00:00–04:00	9000.00	0.00	04:00–08:00	1666.67	835.79
08:00–12:00	5833.33	7547.60	12:00–16:00	7500.00	2432.26
16:00–20:00	19.32	5171.86	20:00–24:00	0.00	3468.14
0:00 储电量	1200.00		24:00 储电量	1200.00	

表 **43**　微网在指定日期的紧急购电量

2025.3.20		2025.6.21		2025.9.23		2025.12.21	
时间段	购电量	时间段	购电量	时间段	购电量	时间段	购电量
06:00–12:00	2497.25	04:00–04:50	2.34	06:00–06:50	692.04	07:00–08:10	67.18
12:30–14:20	545.00	05:00–05:30	12.59	07:10–08:00	61.88	08:50–12:00	829.59
14:40–15:10	35.16	06:00–06:10	154.67	08:10–09:00	64.49	13:00–13:10	2.28
15:20–16:20	119.81	06:20–07:20	343.81	09:50–10:20	46.53	16:30–16:50	13.69
16:50–17:50	77.14	13:00–13:10	1.48	10:40–11:00	48.04		
		16:00–16:20	5.78	11:40–12:00	57.68		
		17:20–17:30	0.50	12:50–14:00	121.33		
		19:00–19:50	2.58	14:20–15:00	92.64		
				15:50–16:30	24.37		
				17:00–17:50	64.93		

表 **44**　波动电价下指定日期的购电费用

日期	问题二购电费	问题三总费用	紧急购电费
2025.3.20	47694.05	54192.97	13666.00
2025.6.21	18367.12	17963.88	1618.00
2025.9.23	49038.12	49152.77	5928.00
2025.12.21	78033.40	75009.33	4147.00

波动电价下四季典型日的购电功率策略如图29所示。与固定电价情形相比，购电功率对电价波动的响应更加灵敏，夜间低电价时段的购电与充电功率明显增大，白天高电价时段的购电被进一步压缩，体现了波动电价下更为积极的低充高放套利行为。

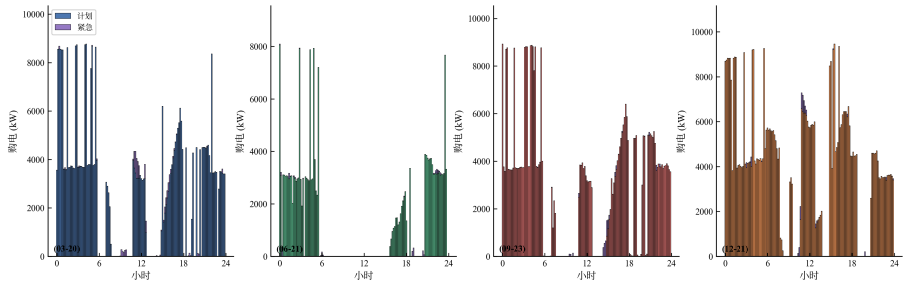


图 **29**　波动电价下四季典型日购电策略

波动电价下典型日的计划购电与滚动调整购电对比如图30所示。调整策略依然能够在光伏预报更新的时点及时修正购电功率，但由于电价本身波动，调整幅度与固定电价情形存在差异，总体上仍以减小供电缺口、降低高价时段购电为目标。

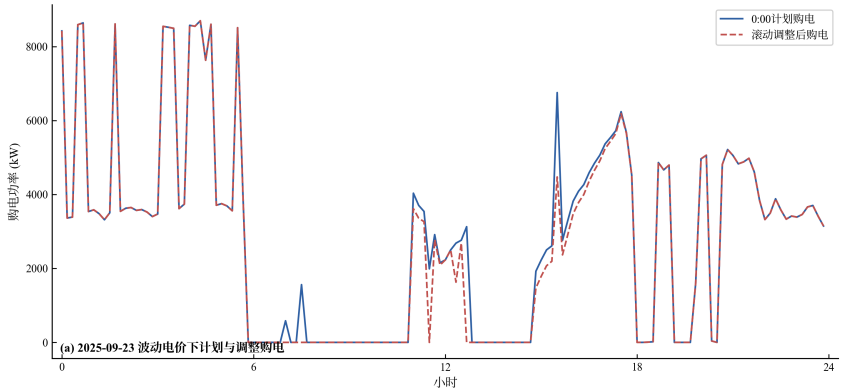


图 **30**　波动电价下典型日计划与调整购电对比

波动电价下典型日的储能荷电状态轨迹如图31所示。储能荷电状态在一天内完成多次充放切换，以适应波动电价的峰谷变化，且全程满足容量约束，说明储能能够有效利用波动电价的峰谷价差获取套利收益。

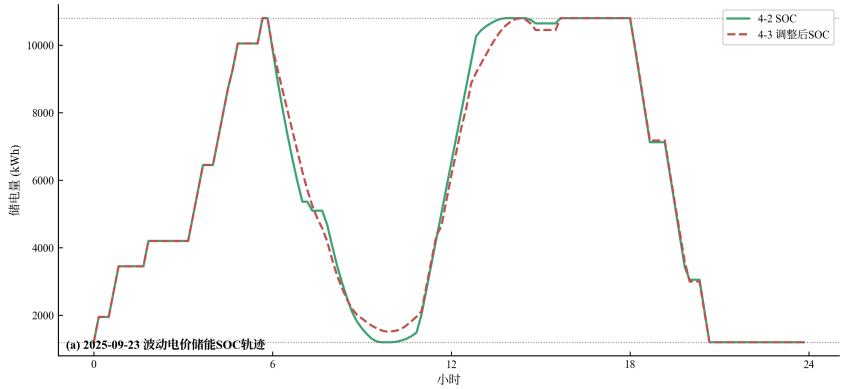


图 **31**　波动电价下典型日储能荷电状态轨迹

波动电价下 2.1—12.31 逐日紧急购电量如图32所示。紧急购电量整体保持低位，仅在个别预报偏差较大日期出现尖峰，与固定电价情形的规律一致，说明滚动调整在波动电价下同样有效。

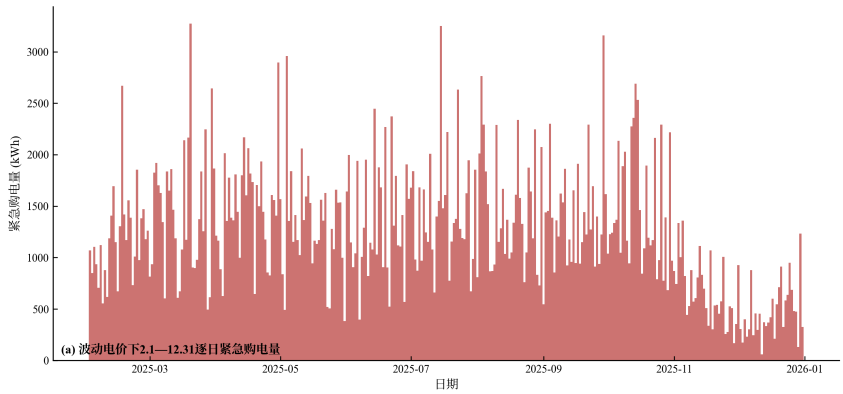


图 **32**　波动电价下 **2.1—12.31** 逐日紧急购电量

固定电价与波动电价下的费用对比如图33所示。波动电价下问题二与问题三的费用均高于固定电价情形，其中问题二由 14382892.99 元升至 15077131.66 元，问题三由 14337999.15 元升至 14956526.18 元，问题三较问题二节省 120605.48 元（约 0.80%），说明实时波动的电价增加了微网购电的不确定性与成本，储能套利与滚动调整只能部分对冲电价波动的影响。

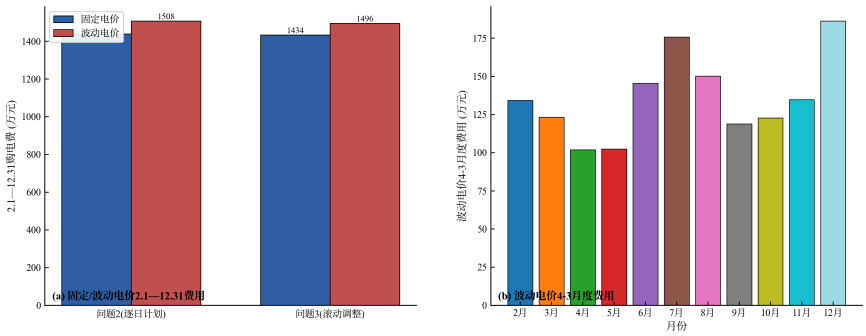


图 **33**　固定电价与波动电价费用对比（**2.1—12.31**）

综合以上结果，问题四在波动电价下重新计算了问题二与问题三的购电策略，两套模型均能稳定运行，储能低充高放的套利行为在波动电价下更为活跃，但购电费用较固定电价情形有所上升，结论合理可靠。

六、 模型检验与分析

为验证本文构建的模型的有效性和稳健性，进行以下检验。本文模型为线性规划优化模型，其最优性由求解器严格保证 [4]，因此检验重点在于约束满足程度、能量守恒精度与参数灵敏度。

6.1 误差分析

对问题一的最优解逐项核验其约束满足情况与能量守恒精度，结果如表45所示。

表 **45**　模型误差分析结果

检验项	检验结果	说明
功率平衡最大残差	1.36×10^{-12}	能量守恒严格满足
储电量越界次数	0	全程处于 $[1200, 10800]$ kWh
充放电功率越界次数	0	始终不超过 5000 kW
储电初值约束	满足	$E_0 = 6000$ kWh，与附录 1 给定一致
日循环约束残差	0	$E_0 = E_{144} = 6000$ kWh

如表45所示，最优解的功率平衡残差处于 10^{-12} 量级，储能荷电状态与充放电功率全程处于约束区间，日循环约束严格成立，验证了模型求解的正确性与数值精度。

6.2 灵敏度分析

为检验模型对关键参数的稳健性，分别调整储能充放电效率、最大充放电功率、储电量下限与负荷水平，观察全天购电费的变化，结果如表46所示。

表 **46**　灵敏度分析结果

参数变化	输出变化	灵敏度评估
$\eta = 0.85$	36603.41 元	费用上升 4.20%
$\eta = 0.95$	33767.03 元	费用下降 3.87%
$P_{\max} = 4000$	35247.87 元	费用上升 0.34%
$P_{\max} = 6000$	35083.29 元	费用下降 0.12%
$E_{\min} = 1500$	35329.91 元	费用上升 0.58%
负荷 $+5\%$	38834.17 元	费用上升 10.55%
负荷 -5%	31552.18 元	费用下降 10.18%

如表46所示，购电费对充放电效率最为敏感，效率每提高 5% 可使费用下降约 4%，说明储能效率是影响经济性的关键因素；充放电功率与储电量下限的小幅变化对费用的影响均不足 1%，模型稳健；负荷水平按比例影响费用，符合能量平衡的线性规律。综上，在上述参数摄动范围内模型表现稳定。

6.3 稳健性检验与适用边界

误差分析与灵敏度分析表明，模型在给定数据下数值可靠，对储能参数的摄动也不敏感；但这两项检验都建立在同一套数据与同一比较基准之上，尚不足以说明结论是否依赖于特定数据样本、求解算法以及未加说明的建模前提。为此，本节从数据摄动、算法替换、预报展开口径、无储能对照与逐日收益方向五个角度补充检验，并明确模型的适用边界。

1. 数据摄动检验

为检验结果是否依赖特定数据样本，对问题二全年 334 天的负荷与光伏数据施加乘性高斯噪声 $\varepsilon \sim N(0, \sigma^2)$，即令 $\tilde{L}_{d,k} = L_{d,k}(1 + \varepsilon^{L}_{d,k})$、$\tilde{S}_{d,k} = S_{d,k}(1 + \varepsilon^{S}_{d,k})$，在噪声强度 $\sigma = 5\%$ 与 $\sigma = 10\%$ 下分别随机重复 6 次与 3 次，重新求解全年逐日滚动模型，统计 2 月 1 日—12 月 31 日购电费的分布，结果如表47所示。

表 **47**　数据摄动检验结果

噪声强度	购电费均值（元）	标准差（元）	相对基准最大偏差
$\sigma = 0$（基准）	14382892.99	—	—
$\sigma = 5\%$	15472366.63	19240.15	$[+7.31\%, +7.71\%]$
$\sigma = 10\%$	17567357.05	36709.48	$[+21.82\%, +22.44\%]$

如表47所示,负荷与光伏同时受到 5% 的随机摄动时,全年购电费均值为 15472366.63 元，较基准上浮约 7.5%，6 次重复相对基准的最大偏差落在 $7.31\% \sim 7.71\%$；摄动强度增至 10% 时，均值升至 17567357.05 元、上浮约 22.1%，3 次重复的最大偏差升至 $21.82\% \sim 22.44\%$。可见在三段式结构下购电费对输入误差是敏感的：摄动直接作用于结算所用的真实功率序列，偏差越大，随机规划为规避 5 倍紧急电价而预留的裕度越保守，实际支出越接近最坏情形。这一方向与“预测不确定性有价”的直觉一致，本文如实证报，不作平滑处理；相应地，提高预测精度对降低总费用具有直接价值。

2. 算法一致性检验

为排除单一求解器误差对结论的影响，将线性规划（LP）求解器替换为基于储电量网格离散化的动态规划（DP）算法 [17]，独立求解问题一的单日调度问题。DP 将储电量区间 $[E_{\min}, E_{\max}]$ 按步长 Δ 离散化后逐时段递推最优费用，与 LP 属于完全不同的求解范式，不同网格步长下的全天购电费如表48所示。

表 **48**　算法一致性检验结果

求解算法	全天购电费（元）	相对 LP 偏差
LP（连续最优）	35126.95	—
DP（$\Delta = 80$ kWh）	35565.93	$+1.25\%$
DP（$\Delta = 40$ kWh）	35365.72	$+0.68\%$
DP（$\Delta = 20$ kWh）	35222.69	$+0.27\%$

如表48所示，随着网格步长由 80 kWh 加密至 20 kWh，DP 解单调收敛于 LP 最优解，步长为 20 kWh 时相对偏差仅 0.27%，验证了本文 LP 最优解的正确性；两种独立算法所得结果高度一致，排除了单一求解器误差对结论的影响。数据摄动与算法一致性两项检验的对比如图34所示。

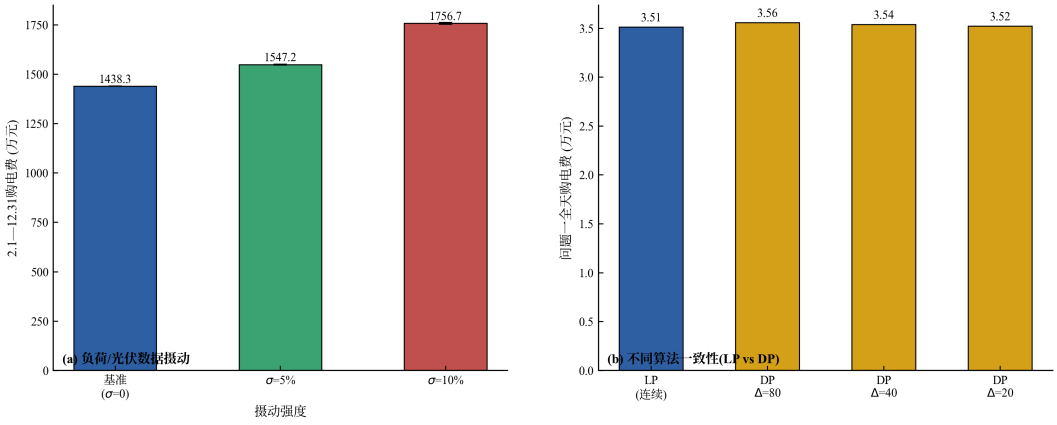


图 **34**　稳健性检验：数据摄动与算法一致性对比

3. 预报展开口径的敏感性

5.3.2 节指出，附件三只给出每个预报时刻的整点预报值，将其展开到 10 分钟栅格存在多种口径。正文采用线性插值；作为对照，若改用“阶梯保持”口径（把每个整点预报值直接保持到下一个整点），在同一模型、同一费用公式下重新求解，2.1—12.31 的结果如表49所示。

表 **49**　预报展开口径对结果的影响（**2.1—12.31**）

展开口径	含调整总费用（元）	无调整总费用（元）	紧急购电量（kWh）	紧急购电费（元）
线性插值（本文采用）	14337999.15	14782072.51	416613.13	1713169.76
阶梯保持（对照）	17948773.82	17884096.43	1223271.04	5391470.75

表49显示，两种口径给出的整点预报值完全相同，2.1—12.31 的总费用却相差 3610774.67 元。紧急购电量由 416613.13 kWh 变为 1223271.04 kWh，两者之比约为 1 : 2.9。在线性插值下引入滚动调整可节省 444073.36 元，而在阶梯保持下调整反而比不调整多支出 64677.39 元，结论发生反转。差别并不来自预报在整点处的精度：两口径的预报值在所有整点处逐点相同，全年逐时预报的平均绝对误差为 234.91 kW、均方根误差为 432.51 kW、平均偏差为 +63.54 kW，且各预报时刻在其 6 小时有效窗口内的分时段误差也完全一致（0:00、6:00、12:00、18:00 发布的预报对应的平均绝对误差分别为 0、535.34、297.85、106.45 kW）。真正的差别在于约束的时段归属：阶梯保持等价于额外假设“整点之间的出力保持不变”，该假设在附件二、附件三中均无依据，却把整点值强加给其后的每一个 10 分钟间隔，人为制造出 10 分钟尺度的瞬时供需缺口，进

而放大紧急购电需求。

据此，本文采用线性插值口径：它不改变任何整点处的预报取值，也不引入预报时刻之外的新信息，仅在相邻整点之间构造连续过渡，因而不具备系统性美化或恶化预报的机制。需要说明的是，这一结论只针对“整点值之间如何过渡”，不涉及预报本身的精度；附件三只给出整点值，10 分钟尺度的过渡方式无法由现有数据判定，本文将 3610774.67 元的量级差如实报告，以便在获得更细分辨率预报数据时复核。

4. 无储能对照

为检验储能的实际价值，取消储能后重新求解同一模型（储电量恒为 0，全部用电均需从电网购入）：问题一全天购电费由 35126.95 元升至 48052.05 元，上升 36.80%；问题二 2.1—12.31 的总费用由 14382892.99 元升至 16951241.92 元，上升 17.86%。两项检验都表明，储能削峰填谷是本文经济性提升的主要来源，而非求解误差或口径选择；问题二的改善幅度小于问题一，是因为在三段式结构下它还要承担预测偏差引致的紧急购电，储能的套利空间被部分占用。

5. 逐日收益的方向性

总费用为累计量，会掩盖逐日差异。以“仅执行 0:00 计划”为基准逐日比较（2.1—12.31 共 334 天），有 155 天调整后的费用反而高于基准，最差的一天为 2025 年 9 月 29 日，多支出 9676.94 元；题目给出的四个代表日中，3 月 20 日节省 201.33 元，而 6 月 21 日、9 月 23 日、12 月 21 日分别多支出 764.72 元、3277.99 元与 707.19 元（逐日的计划与调整购电量见表25，全年逐日紧急购电量见图22）。原因是调整购电量与实际需求的偏差要按 1.5 倍惩罚计价，当预报误差方向与偏差方向不利时，调整反而增加支出，这与 5.3.4 节边际分析中“6:00 预报单独引入时边际收益为 -3.74 万元”相互印证。全年累计仍净节省 444073.36 元，即 3.00%，故滚动调整在期望意义上有效，但不宜宣称逐日均受益。

6. 适用边界

（1）附件三仅给出整点预报值，其物理含义（瞬时值或小时平均值）题目未作说明。本文按瞬时值处理并对相邻整点作线性插值；若其实际含义为小时平均值，过渡段取值将发生改变，表49已给出该不确定性对总费用的影响范围。（2）本文结论依赖“预报误差方向随机”这一隐含前提。逐日比较中出现的 155 天负收益说明，当预报出现系统性偏差（持续高估或低估）时，滚动调整的净节省会被侵蚀，甚至由正转负，此时应降低调整幅度或提高预报精度，而非单纯增加预报时刻；5.3.4 节的结果也表明，在有真实预报的前提下提高预报精度所能带来的收益远大于加密预报时刻。（3）模型未考虑电价的随机性与储能寿命成本，这两者会在长期运行中影响策略的经济性，可作为后续改进方向。

七、 模型的评价

7.1 模型的优点

- **优点 1：**线性规划模型结构简洁、约束刻画完整，所有决策变量均为连续变量，求解器可严格保证全局最优，避免了启发式算法陷入局部最优的风险。
- **优点 2：**滚动调整模型利用上境变量将分段惩罚费用精确线性化，在不引入整数变量的前提下完整刻画违约与超额的经济代价，计算效率高。
- **优点 3：**逐日滚动与储电量传递机制将全年大规模优化分解为数百个日尺度小问题，既降低了计算复杂度，又保持了全年储电量的连续性。
- **优点 4：**模型具有高度可迁移性，只需替换电价、负载与光伏序列即可适应不同季节、不同地区乃至波动电价场景，并通过灵敏度分析验证了稳健性。

7.2 模型的缺点

- **缺点 1：**模型假设各 10 分钟间隔内功率恒定，忽略了更短时间尺度上的功率波动对储能实际运行的影响。
- **缺点 2：**滚动调整模型假定负荷完全可预知、储能严格按计划执行，未考虑储能动态响应光伏实时偏差的闭环控制过程。

八、 模型的改进与推广

8.1 模型的改进

本文模型的不足主要体现在三个方面。在方法层面，可将光伏预报误差建模为随机变量，引入两阶段随机规划或鲁棒优化，在目标函数中显式权衡期望购电费用与违约风险，使调整策略更具前瞻性。在参数层面，可将储能充放电效率进一步细分为充电效率与放电效率、考虑电池循环寿命损耗与自放电率，并引入储电量分段折旧成本，使经济模型更贴近实际设备。在数据层面，可结合历史气象数据与数值天气预报，对光伏预报误差的分布进行估计，据此优化各预报时刻的储能安全裕度，进一步降低紧急购电费用。

8.2 模型的推广

本文建立的微网购电与储能调控模型具有较强的通用性，可直接推广到更广泛的工程与政策场景。在工程领域，该模型可用于工业园区、商业综合体与电动汽车充电站的储能容量配置与日前调度，也可嵌入虚拟电厂的能量管理系统，实现多主体分布式资源的协同优化；在政策领域，该模型可用于评估分时电价、尖峰电价等价格机制的引导效

果，为需求响应与储能补贴政策的制定提供量化依据；在行业领域，其滚动调整与惩罚机制思想可迁移到电力现货市场交易、天然气购销合同与物流库存管理等存在预测更新与违约成本的场景，具有广泛的适用价值。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于代码编写与调试、数据处理的辅助实现以及论文文字的语言润色，详细使用情况见支撑材料。

参考文献

[1] Lasseter R H. Microgrids[C]//IEEE Power Engineering Society Winter Meeting. New York: IEEE, 2002: 305-308.

[2] Kanchev H, Lu D, Colas F, et al. Energy management and operational planning of a microgrid with a PV-based active generator for smart grid applications[J]. IEEE Transactions on Industrial Electronics, 2011, 58(10): 4583-4592.

[3] Dantzig G B. Linear programming and extensions[M]. Princeton: Princeton University Press, 1963.

[4] Boyd S, Vandenberghe L. Convex optimization[M]. Cambridge: Cambridge University Press, 2004.

[5] Birge J R, Louveaux F. Introduction to stochastic programming[M]. 2nd ed. New York: Springer, 2011.

[6] Parisio A, Rikos E, Glielmo L. A model predictive control approach to microgrid operation optimization[J]. IEEE Transactions on Control Systems Technology, 2014, 22(5): 1813-1827.

[7] Rawlings J B, Mayne D Q, Diehl M. Model predictive control: theory, computation, and design[M]. 2nd ed. Santa Barbara: Nob Hill Publishing, 2017.

[8] Antonanzas J, Osorio N, Escobar R, et al. Review of photovoltaic power forecasting[J]. Solar Energy, 2016, 136: 78-111.

[9] Sioshansi R, Denholm P, Jenkin T, et al. Estimating the value of electricity storage in PJM: arbitrage and some welfare effects[J]. Energy Economics, 2010, 31(2): 269-277.

[10] Camacho E F, Bordons C. Model predictive control[M]. 2nd ed. London: Springer, 2013.

[11] 鲁宗相, 王彩霞, 闵勇, 等. 微电网研究综述 [J]. 电力系统自动化, 2007, 31(19): 100-107.

[12] 吴雄, 王秀丽, 刘世民, 等. 微电网能量管理系统研究综述 [J]. 电力自动化设备, 2014, 34(10): 7-14.

[13] 周林, 黄勇, 郭珂, 等. 微电网储能技术研究综述 [J]. 电力系统保护与控制, 2011, 39(7): 147-152.

[14] 桑丙玉, 王德顺, 杨波, 等. 平滑新能源输出波动的储能优化配置方法 [J]. 中国电机工程学报, 2014, 34(22): 3700-3706.

[15] 张钦, 王锡凡, 付敏, 等. 电力市场下需求响应研究综述 [J]. 电力系统自动化, 2008, 32(3): 97-106.

[16] 刘卫亮, 刘长良, 林永君, 等. 计及雾霾影响因素的光伏发电超短期功率预测 [J]. 中国电机工程学报, 2018, 38(14): 4086-4095.

[17] 司守奎, 孙玺菁. 数学建模算法与应用 [M]. 北京: 国防工业出版社, 2011.

附录 A　程序与结果文件说明

表 **50**　程序与结果文件说明表

文件	说明
求解/_models.py	数据读取、小时预报到 10 分钟预报展开、线性规划建模公共模块
求解/_common.py	绘图与结果导出公共模块
求解/问题一/问题一.py	问题一：单日计划购电线性规划求解程序
求解/问题二/问题二.py	问题二：全年逐日“预测—随机规划—结算”三段式求解程序
求解/问题三/问题三.py	问题三：滚动预报—偏差调整两阶段求解程序
求解/问题四/问题四.py	问题四：波动电价情形下的重算程序
求解/预处理数据.csv	由附件 1～附件 4 清洗得到的 10 分钟级建模数据
求解/稳健性检验.py	第六章稳健性检验程序（数据摄动检验、线性规划与动态规划算法一致性检验）
求解/问题 X/图片/	各问题结果图，共 28 幅（问题一 6 幅、问题二 9 幅、问题三 7 幅、问题四 6 幅）
求解/图片/图 6-1_ 稳健性检验.png	第六章稳健性检验结果图（上述 28 幅之外另 1 幅）
求解/结果/result1.xlsx	问题一计划购电策略结果文件
求解/结果/result2.xlsx	问题二全年购电策略结果文件
求解/结果/result3.xlsx	问题三含紧急购电的结果文件
求解/结果/result4-2.xlsx	问题四波动电价下问题二结果文件
求解/结果/result4-3.xlsx	问题四波动电价下问题三结果文件

附录 B　程序源代码

B.1　公共模块 _models.py（数据读取与优化建模）

```python
# 公共求解模块：微网购电调度 LP 建模与数据读取
# 供 求解/问题X/*.py 通过 `from _models import *` 使用
import os
import numpy as np
import pandas as pd
from scipy.optimize import linprog
from scipy.sparse import csr_matrix

# ---- 常量 ----
DT = 1.0 / 6.0          # 10分钟 = 1/6 小时
ETA = 0.9               # 充/放电效率 90%
EMIN, EMAX = 1200.0, 10800.0   # 储电量上下界 (kWh)
PMAX = 5000.0           # 最大充放电功率 (kW)
E0_INIT = 6000.0        # 2025-01-01 0:00 初始储电量 (kWh)
N_INT = 144             # 一天 144 个 10 分钟间隔

# ---- 数据读取 ----
def _data_dir():
    """按候选顺序探测附件目录（兼容 求解/ 与 求解/问题X/ 两种脚本位置）。"""
    import glob as _glb
    here = os.path.dirname(os.path.abspath(__file__))
    cands = []
    for n in (1, 2, 3):
        up = os.path.normpath(os.path.join(here, *['..'] * n))
        for name in ('附件', '数据'):
            cands.append(os.path.join(up, name))
            cands += sorted(_glb.glob(os.path.join(up, '*', name)))
    for c in cands:
        if os.path.isfile(os.path.join(c, '附件1.xlsx')):
            return c
    raise FileNotFoundError('未找到附件目录，已尝试：' + '; '.join(cands))

def read_附件1():
    """返回 (price[144], load[144], pv[144])，附件1 单日电价/负载/光伏预测"""
    p = os.path.join(_data_dir(), '附件1.xlsx')
    df = pd.read_excel(p, sheet_name='Sheet1')
```

```
    price = pd.to_numeric(df['电价'], errors='coerce').to_numpy(dtype=float)
    load = pd.to_numeric(df['小区负载'], errors='coerce').to_numpy(dtype=float)
    pv = pd.to_numeric(df['光伏发电预测功率'], errors='coerce').to_numpy(dtype=float)
    assert len(price) == N_INT, f'附件1 行数 {len(price)} != 144'
    return price, load, pv

def read_附件2():
    """返回 (dates[365], load[365,144], pv[365,144]) 附件2 实际负载与光伏"""
    p = os.path.join(_data_dir(), '附件2.xlsx')
    load_df = pd.read_excel(p, sheet_name='小区负载')
    pv_df = pd.read_excel(p, sheet_name='光伏发电实际功率')
    dates = pd.to_datetime(load_df.iloc[:, 0]).dt.to_pydatetime()
    load = load_df.iloc[:, 1:145].to_numpy(dtype=float)
    pv = pv_df.iloc[:, 1:145].to_numpy(dtype=float)
    return dates, load, pv

def read_附件4():
    """返回 (dates[365], price[365,144]) 附件4 波动电价"""
    p = os.path.join(_data_dir(), '附件4.xlsx')
    df = pd.read_excel(p, sheet_name='Sheet1')
    dates = pd.to_datetime(df.iloc[:, 0]).dt.to_pydatetime()
    price = df.iloc[:, 1:145].to_numpy(dtype=float)
    return dates, price

def read_附件3():
    """返回 dict: {date: {T: f[24]}}, T in {0,6,12,18}, f=未来24小时整点光伏预报(kW)"""
    p = os.path.join(_data_dir(), '附件3.xlsx')
    df = pd.read_excel(p, sheet_name='Sheet1')
    df.columns = [str(c).strip() for c in df.columns]
    df['日期'] = df['日期'].ffill()
    df['预报时刻'] = df['预报时刻'].astype(str).str.strip()
    cols = [c for c in df.columns if '预报' in c and '小时' in c]
    T_map = {'0:00': 0, '6:00': 6, '12:00': 12, '18:00': 18}
    out = {}
    for _, row in df.iterrows():
        d = str(row['日期']).strip()
        T = T_map.get(row['预报时刻'])
```

```
        if T is None:
            continue
        f = pd.to_numeric(row[cols], errors='coerce').to_numpy(dtype=float)
        out.setdefault(d, {})[T] = f
    return out

# ---- 光伏预报小时值 -> 10分钟间隔功率 ----
def forecast_10min(T, f, k0=0, k1=N_INT - 1):
    """按时刻T发布的预报 f[24]（f[i]=T+(i+1)小时的整点功率），
    线性插值到间隔 k in [k0,k1] 的功率(kW)。间隔k对应时间(k+1)/6小时（间隔末端）。"""
    hours = np.arange(T, T + 25, dtype=float)          # T, T+1, ..., T+24
    vals = np.concatenate(([f[0]], f))                  # 当前整点用预报1小时近似
    ts = (np.arange(k0, k1 + 1, dtype=float) + 1.0) / 6.0   # 间隔末端时刻(小时)
    pv = np.interp(ts, hours, vals)
    return np.maximum(pv, 0.0)

def _forecast_day10min(fc, T, k0, k1):
    """把附件3某日某时刻预报插值到该日间隔 k0..k1（绝对日间隔序号）"""
    return forecast_10min(T, fc[T], k0, k1)

# ---- 单日 LP 求解 ----
def solve_daily(L, S, price, e0=None, soc_cycle=False, k0=0, k1=N_INT - 1):
    """求解间隔 k0..k1 的购电/充放电调度。
    L,S,price 长度均为 (k1-k0+1)（或全144，取 k0:k1+1 切片）。
    e0: 初始储电量(固定)；soc_cycle: 问题一——固定 E[k0]=e0（附录1给定，默认6000）
    并施加日循环 E[k0]=E[k1+1]。
    返回 (g, ch, dch, E, cost): g/ch/dch(kW, 长n), E(kWh, 长n+1), cost(元)。"""
    L = np.asarray(L, dtype=float)[k0:k1 + 1]
    S = np.asarray(S, dtype=float)[k0:k1 + 1]
    price = np.asarray(price, dtype=float)[k0:k1 + 1]
    n = k1 - k0 + 1
    # 变量顺序: g(0..n-1), ch(n..2n-1), dch(2n..3n-1), E(3n..4n)
    nv = 4 * n + 1
    c = np.zeros(nv)
    c[:n] = price * DT
    # 不等式: 供电≥负载（允许弃光） g + dch - ch >= L - S  ->  -g - dch + ch <= S - L
```

```
    Aub = np.zeros((n, nv))
    bub = np.zeros(n)
    for i in range(n):
        Aub[i, i] = -1.0            # -g
        Aub[i, 2 * n + i] = -1.0    # -dch
        Aub[i, n + i] = 1.0         # +ch
        bub[i] = S[i] - L[i]
    # 等式约束: SOC动态(n) (+ 初始/周期)
    Aeq = np.zeros((n, nv))
    beq = np.zeros(n)
    for i in range(n):
        Aeq[i, 3 * n + i + 1] = 1.0      # E[i+1]
        Aeq[i, 3 * n + i] = -1.0         # -E[i]
        Aeq[i, n + i] = -ETA * DT        # -eta*dt*ch
        Aeq[i, 2 * n + i] = DT / ETA     # +dt/eta*dch
        beq[i] = 0.0
    # 边界
    lb = np.zeros(nv)
    ub = np.full(nv, np.inf)
    ub[n:3 * n] = PMAX                   # ch, dch <= 5000
    lb[3 * n:] = EMIN
    ub[3 * n:] = EMAX
    # 初始/周期约束
    if soc_cycle:
        # 附录1给定 0:00 储电量: E[k0] = e0（默认 6000 kWh），不再作为可优化变量
        e0v = float(E0_INIT if e0 is None else e0)
        row = np.zeros(nv)
        row[3 * n] = 1.0
        Aeq = np.vstack([Aeq, row])
        beq = np.append(beq, e0v)
        # 日循环约束: E[k0] - E[k1+1] = 0
        row = np.zeros(nv)
        row[3 * n] = 1.0
        row[4 * n] = -1.0
        Aeq = np.vstack([Aeq, row])
        beq = np.append(beq, 0.0)
    else:
        # E[k0] = e0
        row = np.zeros(nv)
        row[3 * n] = 1.0
        Aeq = np.vstack([Aeq, row])
```

```
151         beq = np.append(beq, float(e0))
152     res = linprog(c, A_ub=Aub, b_ub=bub, A_eq=Aeq, b_eq=beq, bounds=list(
    zip(lb, ub)), method='highs')
153     if not res.success:
154         # 回退到 highs-ds / highs-ipm
155         res = linprog(c, A_ub=Aub, b_ub=bub, A_eq=Aeq, b_eq=beq, bounds=
    list(zip(lb, ub)), method='highs-ds')
156     if not res.success:
157         raise RuntimeError(f'LP失败: {res.message}')
158     x = res.x
159     g = x[:n]
160     ch = x[n:2 * n]
161     dch = x[2 * n:3 * n]
162     E = x[3 * n:4 * n + 1]
163     cost = float(res.fun)
164     return g, ch, dch, E, cost
165 
166 # ================= 问题二: 预测 — 决策 — 结算 三段式 =================
167 VALLEY_WEEKDAYS = (4, 5)      # 实测工作日/休息日负荷形态在此处分档（周五、
    周六）
168 
169 def _norm_shape(x):
170     """归一化日形状: 逐点除以日均值, 均值为 0 时原样返回。"""
171     x = np.asarray(x, dtype=float)
172     m = x.mean()
173     return x / m if m > 0 else x.copy()
174 
175 def forecast_load(hist_load, hist_dates, target, k_shape=8, k_level=2):
176     """负载预测: 同类日 EWMA 水平 × 近 k_shape 个同类日归一化形状均值。
177 
178     hist_load  长 (T,144) 历史负载; hist_dates 长 T 的日期序列; target 目标
    日。
179     只使用 target 之前的数据, 同类日按 VALLEY_WEEKDAYS 分档。
180     返回长 144 的预测负载 (kW)。
181     """
182     hist_load = np.asarray(hist_load, dtype=float)
183     hist_dates = list(hist_dates)
184     tgt_cls = target.weekday() in VALLEY_WEEKDAYS
185     idx = [i for i, d in enumerate(hist_dates)
186            if (d.weekday() in VALLEY_WEEKDAYS) == tgt_cls]
187     if len(idx) == 0:
188         idx = list(range(len(hist_dates)))
```

```
    idx = idx[-max(k_shape, k_level):]
    lvls = hist_load[idx].mean(axis=1)
    k_l = min(k_level, len(idx))
    w = np.linspace(1.0, 2.0, k_l)
    level = float(np.average(lvls[-k_l:], weights=w))
    shape = np.mean([_norm_shape(hist_load[i]) for i in idx[-k_shape:]],
    axis=0)
    return np.maximum(level * shape, 0.0)

def forecast_pv(hist_pv, hist_dates=None, target=None, k_shape=4, k_level
    =4):
    """光伏预测：近 k_shape 天归一化形状均值 × 近 k_level 天日均水平的 EWMA
    。

    光伏形态对星期不敏感，跨星期取最近若干天。返回长 144 的预测光伏 (kW)。
    """
    hist_pv = np.asarray(hist_pv, dtype=float)
    k_s = min(k_shape, len(hist_pv))
    idx = list(range(len(hist_pv) - k_s, len(hist_pv)))
    shape = np.mean([_norm_shape(hist_pv[i]) for i in idx], axis=0)
    lvls = hist_pv[idx].mean(axis=1)
    k_l = min(k_level, len(idx))
    w = np.linspace(1.0, 2.0, k_l)
    level = float(np.average(lvls[-k_l:], weights=w))
    return np.maximum(level * shape, 0.0)

def pred_residual(F, A, thr=1.0):
    """逐点相对预测残差 (A-F)/max(F,thr)，展平返回。"""
    F = np.asarray(F, dtype=float)
    A = np.asarray(A, dtype=float)
    return ((A - F) / np.maximum(F, thr)).ravel()

def scenario_factors(res_load, res_pv, q=(0.1, 0.5, 0.9), fallback=(0.95,
    1.0, 1.05)):
    """由历史预测残差的经验分位数标定情景因子。

    因子 f = 1 + 残差分位数；样本不足 50 时退回先验 fallback。
    返回 (load_fac, pv_fac) 各长 3，并做安全裁剪。
    """
    out = []
    for res, lo, hi in ((res_load, 0.7, 1.3), (res_pv, 0.5, 1.5)):
        res = np.asarray(res, dtype=float)
```

```
        res = res[np.isfinite(res)]
        if res.size < 50:
            f = np.asarray(fallback, dtype=float)
        else:
            f = 1.0 + np.percentile(res, np.asarray(q, dtype=float) *
    100.0)
        out.append(np.clip(f, lo, hi))
    return out[0], out[1]

def solve_daily_stochastic(L_fc, S_fc, price, load_fac, pv_fac, pi, e0,
                           emerg_mult=5.0, k0=0, k1=N_INT - 1):
    """两阶段随机 LP: 0:00 一次性决定计划购电与充放电，紧急购电事后兜底。

    第一阶段 g/ch/dch/E 不依赖情景；第二阶段各情景的紧急购电 δ_s 按 5 倍电
    价计入。
    目标 min Σ p_k g_k dt + Σ_s π_s Σ_k 5 p_k δ_{s,k} dt
    约束 δ_{s,k} ≥ L^_k f^L_s − Ŝ_k f^PV_s − g_k − dch_k + ch_k（逐情景独
    立）
    SOC 递推对全情景唯一，δ ≥ 0、ch/dch ≤ P_max、E 在 [E_min,E_max] 内。
    返回 (g, ch, dch, E, exp_cost)。
    """
    L_fc = np.asarray(L_fc, dtype=float)[k0:k1 + 1]
    S_fc = np.asarray(S_fc, dtype=float)[k0:k1 + 1]
    price = np.asarray(price, dtype=float)[k0:k1 + 1]
    load_fac = np.asarray(load_fac, dtype=float)
    pv_fac = np.asarray(pv_fac, dtype=float)
    pi = np.asarray(pi, dtype=float)
    n = k1 - k0 + 1
    ns = len(load_fac)
    off_d = 4 * n + 1                    # δ_s 的起始列
    nv = off_d + ns * n
    # 变量顺序: g, ch, dch, E(0..4n), δ(off_d..)
    c = np.zeros(nv)
    c[:n] = price * DT
    for s in range(ns):
        c[off_d + s * n: off_d + (s + 1) * n] = pi[s] * emerg_mult * price
    * DT
    # 不等式 δ_{s,k} + g_k + dch_k - ch_k >= L^_k f^L_s - Ŝ_k f^SV_s
    nr = ns * n
    ri = np.repeat(np.arange(nr), 4)
    base = np.tile(np.arange(n), ns)
    ci = np.empty(nr * 4, dtype=np.int64)
```

```
    ci[0::4] = base
    ci[1::4] = base + n
    ci[2::4] = base + 2 * n
    ci[3::4] = np.arange(nr) + off_d
    va = np.empty(nr * 4)
    va[0::4] = -1.0
    va[1::4] = 1.0
    va[2::4] = -1.0
    va[3::4] = -1.0
    Aub = csr_matrix((va, (ri, ci)), shape=(nr, nv))
    bub = (S_fc[None, :] * pv_fac[:, None] - L_fc[None, :] * load_fac[:, None]).ravel()
    # SOC 等式: E_{k+1} - E_k - eta*dt*ch_k + dt/eta*dch_k = 0; 首行 E_0 = e0
    er = np.repeat(np.arange(n), 4)
    ec = np.empty(4 * n, dtype=np.int64)
    ec[0::4] = 3 * n + np.arange(n) + 1
    ec[1::4] = 3 * n + np.arange(n)
    ec[2::4] = n + np.arange(n)
    ec[3::4] = 2 * n + np.arange(n)
    ev = np.empty(4 * n)
    ev[0::4] = 1.0
    ev[1::4] = -1.0
    ev[2::4] = -ETA * DT
    ev[3::4] = DT / ETA
    Aeq = csr_matrix((np.append(ev, 1.0), (np.append(er, n), np.append(ec, 3 * n))),
                     shape=(n + 1, nv))
    beq = np.zeros(n + 1)
    beq[n] = float(e0)
    lb = np.zeros(nv)
    ub = np.full(nv, np.inf)
    ub[n:3 * n] = PMAX
    lb[3 * n:4 * n + 1] = EMIN
    ub[3 * n:4 * n + 1] = EMAX
    res = linprog(c, A_ub=Aub, b_ub=bub, A_eq=Aeq, b_eq=beq,
                  bounds=list(zip(lb, ub)), method='highs')
    if not res.success:
        raise RuntimeError(f'随机LP失败: {res.message}')
    x = res.x
    return x[:n], x[n:2 * n], x[2 * n:3 * n], x[3 * n:4 * n + 1], float(res.fun)
```

```python

def settle_emergency(L_act, S_act, price, g_plan, ch_plan, dch_plan,
                     emerg_mult=5.0, k0=0, k1=N_INT - 1):
    """实际运行结算：按计划充放电，缺口用 5 倍电价紧急购电补足。

    返回 (delta, delta_energy, plan_cost, emerg_cost):
    delta(kW)、delta_energy(kWh)、计划购电费（全间隔按计划量计价）、紧急购电费。
    """
    L_act = np.asarray(L_act, dtype=float)[k0:k1 + 1]
    S_act = np.asarray(S_act, dtype=float)[k0:k1 + 1]
    price = np.asarray(price, dtype=float)[k0:k1 + 1]
    g_plan = np.asarray(g_plan, dtype=float)[k0:k1 + 1]
    ch_plan = np.asarray(ch_plan, dtype=float)[k0:k1 + 1]
    dch_plan = np.asarray(dch_plan, dtype=float)[k0:k1 + 1]
    delta = np.maximum(0.0, L_act - S_act - g_plan - dch_plan + ch_plan)
    delta_energy = delta * DT
    plan_cost = float(np.sum(price * g_plan) * DT)
    emerg_cost = float(np.sum(emerg_mult * price * delta) * DT)
    return delta, delta_energy, plan_cost, emerg_cost

# ---- 带违约/超额惩罚的调整 LP ----
def solve_adjust(L, S, price, plan, e0, k0=0, k1=N_INT - 1):
    """在间隔 k0..k1 重优化调整购电，费用=min(plan,adj)+0.5*max(plan-adj,0)
    +1.5*max(adj-plan,0)。
    plan 为全144的0:00计划购电(kW)；e0 为 E[k0]。
    返回 (adj, ch, dch, E, cost)。"""
    L = np.asarray(L, dtype=float)[k0:k1 + 1]
    S = np.asarray(S, dtype=float)[k0:k1 + 1]
    price = np.asarray(price, dtype=float)[k0:k1 + 1]
    plan = np.asarray(plan, dtype=float)[k0:k1 + 1]
    n = k1 - k0 + 1
    # 变量: g(0..n-1), ch(n..2n-1), dch(2n..3n-1), E(3n..4n), w(4n+1..5n)
    nv = 5 * n + 1
    c = np.zeros(nv)
    c[4 * n + 1:5 * n + 1] = 1.0          # min sum w
    # 等式: SOC动态(n) + 初始(1)
    Aeq = np.zeros((n + 1, nv))
    beq = np.zeros(n + 1)
    for i in range(n):
        Aeq[i, 3 * n + i + 1] = 1.0
        Aeq[i, 3 * n + i] = -1.0
```

```
        Aeq[i, n + i] = -ETA * DT
        Aeq[i, 2 * n + i] = DT / ETA
        beq[i] = 0.0
    Aeq[n, 3 * n] = 1.0
    beq[n] = float(e0)
    # 不等式1: 供电≥负载（允许弃光） -g - dch + ch <= S - L
    # 不等式2: 分段费用上境 w >= f(adj)
    Aub = np.zeros((3 * n, nv))
    bub = np.zeros(3 * n)
    for i in range(n):
        Aub[i, i] = -1.0
        Aub[i, 2 * n + i] = -1.0
        Aub[i, n + i] = 1.0
        bub[i] = S[i] - L[i]
    for i in range(n):
        p = price[i]
        # w - 0.5*dt*p*g >= 0.5*dt*p*plan  ->  -w + 0.5dtp g <= -0.5dtp plan
        Aub[n + i, 4 * n + 1 + i] = -1.0
        Aub[n + i, i] = 0.5 * DT * p
        bub[n + i] = -0.5 * DT * p * plan[i]
        # w - 1.5*dt*p*g >= -0.5*dt*p*plan ->  -w + 1.5dtp g <= 0.5dtp plan
        Aub[2 * n + i, 4 * n + 1 + i] = -1.0
        Aub[2 * n + i, i] = 1.5 * DT * p
        bub[2 * n + i] = 0.5 * DT * p * plan[i]
    lb = np.zeros(nv)
    ub = np.full(nv, np.inf)
    ub[n:3 * n] = PMAX
    lb[3 * n:4 * n + 1] = EMIN
    ub[3 * n:4 * n + 1] = EMAX
    lb[4 * n + 1:5 * n + 1] = 0.0
    res = linprog(c, A_ub=Aub, b_ub=bub, A_eq=Aeq, b_eq=beq,
                  bounds=list(zip(lb, ub)), method='highs')
    if not res.success:
        res = linprog(c, A_ub=Aub, b_ub=bub, A_eq=Aeq, b_eq=beq,
                      bounds=list(zip(lb, ub)), method='highs-ds')
    if not res.success:
        raise RuntimeError(f'调整LP失败: {res.message}')
    x = res.x
    adj = x[:n]
    ch = x[n:2 * n]
    dch = x[2 * n:3 * n]
```

```
    E = x[3 * n:4 * n + 1]
    return adj, ch, dch, E, float(res.fun)

# ---- 10分钟间隔标签 ----
def interval_labels(k0=0, k1=N_INT - 1):
    """间隔 k=[k/6,(k+1)/6] 的 'HH:MM-HH:MM' 标签（用末端时刻对齐，跨24点加+1）"""
    labs = []
    for k in range(k0, k1 + 1):
        a = k * 10
        b = (k + 1) * 10
        s = _hhmm(a) + '-' + _hhmm(b)
        if b >= 1440:
            s = _hhmm(a) + '-' + _hhmm(b - 1440) + '+1'
        labs.append(s)
    return labs

def _hhmm(mins):
    h = (mins // 60) % 24
    m = mins % 60
    return f'{h:02d}:{m:02d}'
```

B.2 公共模块 _common.py（绘图与结果导出）

```
# 公共代码模板：求解/_common.py（系统自动播种，每个任务只写一次，不要覆写覆盖函数）
import sys, os
# 问题目录 = 运行脚本所在目录（兼容 cd 求解/问题X 与 求解/ 根运行两种方式）
_script_dir = os.path.dirname(os.path.abspath(sys.argv[0]))
if not _script_dir:
    _script_dir = os.getcwd()
BASE_DIR = _script_dir

import pandas as pd
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import matplotlib.font_manager as _fm
import warnings, logging
warnings.filterwarnings('ignore')
logging.getLogger('matplotlib').setLevel(logging.ERROR)  # 抑制findfont警告
```

```
logging.getLogger('matplotlib.font_manager').setLevel(logging.ERROR)  # 抑制字体管理器警告

# * 统一绘图类导入中枢：一次性提供常用 matplotlib 类/子模块，
#   使脚本只写 `from _common import *` 即可直接用，无需各自重复 import（防止遗漏导致 NameError）。
import matplotlib.ticker as mticker
import scipy.stats as stats
from matplotlib.patches import (Rectangle, FancyBboxPatch, FancyArrowPatch, FancyArrow,
                                Circle, RegularPolygon, Ellipse, Wedge, Polygon, Arrow, Arc)
from matplotlib.collections import PatchCollection, LineCollection, PolyCollection
from matplotlib.path import Path
from matplotlib.spines import Spine
try:
    from matplotlib.projections.polar import PolarAxes
    from matplotlib.projections import register_projection
    from mpl_toolkits.axes_grid1.inset_locator import inset_axes, mark_inset
except Exception:
    pass

# 字体：仅首次运行时重建缓存（marker 防止每次都全量扫描字体拖慢）
def _find_font_dir():
    starts = [BASE_DIR, os.getcwd()]
    for start in starts:
        d = start
        for _ in range(6):
            candidate = os.path.join(d, 'fonts')
            if os.path.isdir(candidate) and os.path.isfile(os.path.join(candidate, 'SourceHanSerifCN-Regular.otf')):
                return candidate
            parent = os.path.dirname(d)
            if parent == d: break
            d = parent
    return None
_SOLVE_DIR = os.path.dirname(BASE_DIR)
# 求解/ 根目录（脚本所在 问题X 的上一级），跨问题共享中间文件统一放这里
SOLVE_DIR = _SOLVE_DIR
_FONT_MARKER = os.path.join(_SOLVE_DIR, '_fonts_ready')
```

```
if not os.path.exists(_FONT_MARKER):
    try:
        import glob as _glb
        for _fc in _glb.glob(os.path.join(matplotlib.get_cachedir(), 'font*
    ')):
            try: os.remove(_fc)
            except: pass
        _fm._load_fontmanager(try_read_cache=False)
        open(_FONT_MARKER, 'w').close()
    except Exception:
        pass
_FONT_DIR = _find_font_dir()
if _FONT_DIR:
    for _f in ['SourceHanSerifCN-Regular.otf', 'SourceHanSerifCN-Bold.otf'
    ]:
        _p = os.path.join(_FONT_DIR, _f)
        if os.path.exists(_p): _fm.fontManager.addfont(_p)
_SYS_FONT = os.path.join(os.environ.get('WINDIR', 'C:\\Windows'), 'Fonts')
for _f in ['times.ttf', 'timesbd.ttf', 'timesi.ttf', 'simsun.ttc']:
    _p = os.path.join(_SYS_FONT, _f)
    if os.path.exists(_p): _fm.fontManager.addfont(_p)
plt.rcParams['font.family'] = ['Times New Roman', 'Source Han Serif CN', '
    SimSun', 'serif']
plt.rcParams['font.weight'] = 'normal'
plt.rcParams['axes.unicode_minus'] = False
plt.rcParams['figure.dpi'] = 150
plt.rcParams['savefig.dpi'] = 220
plt.rcParams['savefig.bbox'] = 'tight'
plt.rcParams['pdf.fonttype'] = 42

# * 兼容层：monkey-patch 旧 API，防止 agent 用 plt.cm.get_cmap() 报错
if not hasattr(matplotlib.cm, 'get_cmap'):
    matplotlib.cm.get_cmap = lambda name, n=None: plt.colormaps[name].
    resampled(n) if n else plt.colormaps[name]
try:
    from mpl_toolkits.mplot3d import Axes3D as _Axes3D
except ImportError:
    pass

FIG_DIR = os.path.join(BASE_DIR, '图片')
OUT_DIR = os.path.join(BASE_DIR, '结果')
os.makedirs(FIG_DIR, exist_ok=True)
```

```
os.makedirs(OUT_DIR, exist_ok=True)

# * 数据目录：在脚本目录的若干级祖先及其一级子目录中探测含 附件1.xlsx 的 附件/ 或 数据/
def _find_data_dir():
    import glob as _glb
    _cands = []
    for _n in (1, 2, 3, 4):
        _up = os.path.normpath(os.path.join(BASE_DIR, *['..'] * _n))
        for _name in ('附件', '数据'):
            _cands.append(os.path.join(_up, _name))
            _cands += sorted(_glb.glob(os.path.join(_up, '*', _name)))
    for _c in _cands:
        _c = os.path.normpath(_c)
        if os.path.isfile(os.path.join(_c, '附件1.xlsx')):
            return _c
    return os.path.normpath(os.path.join(BASE_DIR, '..', '..', '附件'))

DATA_DIR = _find_data_dir()

# * 结果总目录：求解/结果/（跨问题汇总，供论文附录与交付使用；不写入原始附件目录）
ROOT_OUT = os.path.join(SOLVE_DIR, '结果')
os.makedirs(ROOT_OUT, exist_ok=True)

def despine(ax):
    """只保留左侧和底侧边框，移除上侧和右侧"""
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

def save_fig(fig, name_cn, dpi=300):
    """保存单张图片，自动加.png后缀（默认 300dpi 高清）"""
    path = os.path.join(FIG_DIR, name_cn + '.png')
    fig.savefig(path, dpi=dpi, bbox_inches='tight', facecolor='white')
    plt.close(fig)
    print(f'  已保存: {name_cn}.png')

def save_dual(fig, name_cn, dpi=300, bw=False):
    """保存图片（默认 300dpi 高清）。绘图规则只输出全彩，默认不再生成黑白版（bw=False，仅保存彩色）。
    如需临时验证黑白效果可手动传 bw=True，但最终产出图一律不得黑白。"""
    from PIL import Image, ImageOps
```

```
    color_path = os.path.join(FIG_DIR, name_cn + '.png')
    fig.savefig(color_path, dpi=dpi, bbox_inches='tight', facecolor='white'
)
    print(f'  已保存: {name_cn}.png')
    if bw:
        bw_path = os.path.join(FIG_DIR, name_cn + '_bw.png')
        try:
            img = Image.open(color_path)
            img_gray = ImageOps.grayscale(img)
            img_gray.save(bw_path)
            img.close()
        except Exception:
            fig.savefig(bw_path, dpi=dpi, bbox_inches='tight', facecolor='
white')
        print(f'  已保存: {name_cn}_bw.png')
        print(f'  * 双版本完成: {name_cn}')
    plt.close(fig)

def save_csv(df, name_cn):
    df.to_csv(OUT_DIR + '/' + name_cn, index=False, encoding='utf-8-sig')

def shared_path(name):
    """跨问题共享中间文件的统一路径（存到 求解/ 根，供各问题脚本互相读
取）。
    例: 问题一存 shared_path('预处理数据.csv'), 问题二用 pd.read_csv(
shared_path('预处理数据.csv')) 读取。"""
    return os.path.join(SOLVE_DIR, name)

def is_ax_empty(ax):
    """检测坐标轴是否无数据（空图）"""
    return (len(ax.lines) == 0 and len(ax.collections) == 0
            and len(ax.patches) == 0 and len(ax.images) == 0)

def is_flat_line(x, y):
    """折线是否为直线（完全无变化→不画）: 所有y相同或所有x相同"""
    if x is None or y is None or len(x) < 2 or len(y) < 2:
        return False
    y_arr = np.asarray(y, dtype=float)
    x_arr = np.asarray(x, dtype=float)
    y_flat = np.allclose(y_arr, y_arr[0])
    x_flat = np.allclose(x_arr, x_arr[0])
    return y_flat or x_flat
```

```python

def validate_subplots(nrows, ncols):
    """
    子图布局校验，不合法返回错误原因字符串，合法返回None。
    规则（硬性）：单图1×1；一行多列1×N(N≥2)；两行2×N且N≥4；其余禁止。
    禁止：三行及以上；两行列数≤3；2×2四宫格。
    """
    if nrows >= 3:
        return f"禁止超过两行（{nrows}行），行列最多两行"
    if nrows == 2 and ncols <= 3:
        return f"两行列数必须>3（当前{ncols}列），换2×{max(4, ncols)}或改单行布局"
    return None

def get_colors(cmap_name, n):
    """安全获取colormap的n个颜色（替代plt.cm.get_cmap，已废弃会报错）。"""
    cmap = plt.colormaps[cmap_name].resampled(n)
    return [cmap(i) for i in range(n)]

def safe_cmap(cmap_name):
    """安全获取colormap对象（不带resampled）。"""
    return plt.colormaps[cmap_name]

def safe_boxplot(ax, data, labels, **kwargs):
    """安全的boxplot（替代ax.boxplot(labels=...)，labels参数已废弃会报错）。
    内部强制 patch_artist=True，使 boxes 为 Patch 对象，AI 可直接用 bp['boxes'][i].set_facecolor(...) 填色；
    否则 boxes 是 Line2D，set_facecolor 会抛 AttributeError。"""
    kwargs.setdefault('patch_artist', True)
    bp = ax.boxplot(data, **kwargs)
    ax.set_xticklabels(labels, fontsize=11)
    return bp

def safe_heatmap(ax, data, row_labels=None, col_labels=None, cmap='RdBu_r',
                 linewidth=0.3, edgecolor='black', fontsize=11, **kwargs):
    """
    强制带黑色细边框的热力图（替代裸 imshow/heatmap 的纯色块）。
    用 pcolormesh 绘制，每个格子边线统一为黑色细框（edgecolor=black, linewidth=linewidth），
    避免"色块连成一片"的观感。NaN 单元格填白色。默认 vmin/vmax 取数据范围。
    用法：
```

```
        safe_heatmap(ax, corr.values, row_labels=corr.index, col_labels=corr.columns, cmap='RdBu_r')
    返回 pcolormesh 的 QuadMesh（可用其 .set_clim 调色标范围）。
    """
    data = np.asarray(data, dtype=float)
    rows, cols = data.shape
    cmap_obj = plt.colormaps[cmap]
    cmap_obj.set_bad(color='#FFFFFF')          # NaN 为白，避免黑洞
    mesh = ax.pcolormesh(np.ma.masked_invalid(data), cmap=cmap_obj,
                         edgecolors=edgecolor, linewidth=linewidth, **kwargs)
    if row_labels is not None:
        ax.set_yticks(np.arange(rows) + 0.5)
        ax.set_yticklabels(list(row_labels), fontsize=fontsize)
    if col_labels is not None:
        ax.set_xticks(np.arange(cols) + 0.5)
        ax.set_xticklabels(list(col_labels), rotation=45, ha='right',
                           rotation_mode='anchor', fontsize=fontsize)
    ax.set_xlim(0, cols)
    ax.set_ylim(0, rows)
    ax.set_aspect('auto')
    return mesh

def png_wh(png_path):
    """读取PNG实际宽高（字节，读16-24字节IHDR）"""
    import struct
    with open(png_path, 'rb') as f:
        f.read(16)
        w, h = struct.unpack('>II', f.read(8))
    return w, h

def latex_includegraphics(png_path, caption, label, pair=False, float_pos='ht'):
    r"""
    按高宽比生成匹配档位的 \includegraphics LaTeX 片段（直接粘论文）。
    pair=True：同一张figure里两张并排minipage，给半宽。float_pos默认'ht'，禁止'H'锁死。
    """
    if float_pos.upper() == 'H':
        raise ValueError("float_pos禁止'H'强制定位，默认用'ht'允许浮动")
    w, h = png_wh(png_path)
    aspect = h / w
```

```
    if pair:
        width = '0.48\\textwidth'
    elif aspect >= 0.9:
        width = '0.60\\textwidth'
    elif aspect >= 0.6:
        width = '0.65\\textwidth'
    else:
        width = '0.70\\textwidth'
    rel = png_path.replace('\\', '/')
    return (
        f"\\begin{{figure}}[{float_pos}]\n"
        f"  \\centering\n"
        f"  \\includegraphics[width={width}]{{{rel}}}\n"
        f"  \\caption{{{caption}}}\n"
        f"  \\label{{fig:{label}}}\n"
        f"\\end{{figure}}"
    )

# * 数据清洗辅助（避免 "--" 等非数值导致 min/max/idxmax 报错）
def numeric_clean(s, default=None):
    """将列安全转为数值：非数值元素→NaN；default 不为 None 时用 default 填充NaN。
    用法：df['评分'] = numeric_clean(df['评分'])   # 先转数值（NaN保留）
          df['评分'] = numeric_clean(df['评分'], default=0.0)   # 转数值并填0"""
    s = pd.to_numeric(s, errors='coerce')
    if default is not None:
        s = s.fillna(default)
    return s

def safe_idxmax(s, default_idx=None):
    """安全的 idxmax：空序列/全NA 返回 default_idx，否则返回最大值索引。"""
    s = pd.to_numeric(s, errors='coerce')
    s = s.dropna()
    if s.empty:
        return default_idx
    return s.idxmax()

def safe_idxmin(s, default_idx=None):
    """安全的 idxmin：空序列/全NA 返回 default_idx，否则返回最小值索引。"""
    s = pd.to_numeric(s, errors='coerce')
    s = s.dropna()
```

```
    if s.empty:
        return default_idx
    return s.idxmin()

def has_na(df):
    """检测DataFrame是否含NaN/空值（用于空值防线）。"""
    return bool(df.isna().any().any())
```

B.3 问题一求解程序 问题一.py

```
# 问题一：每天电价与负载相同 + 光伏预测已知，储能0:00与24:00储电量相同
# 单日计划购电策略 LP
import sys, os
HERE = os.path.dirname(os.path.abspath(__file__))
SOLVE = os.path.dirname(HERE)
sys.path.insert(0, SOLVE)
from _common import *
from _models import *

# ---------- 1. 读数据 & 预处理 ----------
price, load, pv = read_附件1()
print('附件1 列核对：电价/小区负载/光伏发电预测功率，行数=', len(price))
# 预处理：数值化(已由read完成)、时间边界'0:00+1'→24:00、144点映射为144个10分钟间隔
# 异常值检测(3σ)：记录异常数（用于预处理前后对比说明）
def outlier_mask(x, k=3.0):
    mu, sd = x.mean(), x.std()
    return (np.abs(x - mu) > k * sd)
for name, arr in [('电价', price), ('负载', load), ('光伏', pv)]:
    print(f'  {name}: min={arr.min():.4f} max={arr.max():.4f} 异常点数(3σ)={outlier_mask(arr).sum()}')

# 预处理后数据（间隔对齐：第k个间隔=[k*10,(k+1)*10]分钟，功率=间隔k的平均功率）
proc = pd.DataFrame({
    '间隔序号': np.arange(N_INT),
    '间隔标签': interval_labels(),
    '电价(元/kWh)': price,
    '小区负载(kW)': load,
    '光伏预测(kW)': pv,
})
```

```
proc.to_csv(shared_path('预处理数据.csv'), index=False, encoding='utf-8-sig')
print('已保存 求解/预处理数据.csv')

# ---------- 2. LP 求解 ----------
# 附录1给定 2025-01-01 0:00 储电量 E0=6000 kWh，故固定 E0=6000 并施加日循环 E0=E144
g, ch, dch, E, cost = solve_daily(load, pv, price, e0=E0_INIT, soc_cycle=True)
E0, E24 = E[0], E[-1]
gE = g * DT          # 购电量 kWh/间隔
chE = ch * DT
dchE = dch * DT
print(f'\n问题一结果: 全天购电量={gE.sum():.4f} kWh, 全天购电费={cost:.4f}元')
print(f'  0:00储电量={E0:.4f}, 24:00储电量={E24:.4f} kWh, SOC范围=[{E.min():.2f},{E.max():.2f}]')
print(f'  全天充电量={chE.sum():.4f}, 全天放电量={dchE.sum():.4f} kWh')

# 表1 指定时间段
spec = [(10, 0), (12, 0), (14, 0), (16, 0), (18, 0), (20, 0)]  # (时,分)
print('\n表1 指定时间段购电量:')
for hh, mm in spec:
    k = (hh * 60 + mm) // 10
    print(f'  {hh:02d}:{mm:02d}-{hh:02d}:{mm + 10:02d}  购电量={gE[k]:.4f} kWh')

# 表2 6个4小时段充放电量
print('\n表2 储能充放电量:')
blocks = [(0, 4), (4, 8), (8, 12), (12, 16), (16, 20), (20, 24)]
block_rec = []
for a, b in blocks:
    k0, k1 = a * 6, b * 6 - 1
    cq = chE[k0:k1 + 1].sum()
    dq = dchE[k0:k1 + 1].sum()
    block_rec.append((f'{a:02d}:00-{b:02d}:00', cq, dq))
    print(f'  {a:02d}:00-{b:02d}:00  充电量={cq:.4f}  放电量={dq:.4f}')
print(f'  0:00储电量={E0:.4f} kWh, 24:00储电量={E24:.4f} kWh')

# ---------- 3. 出图 ----------
t = np.arange(N_INT) / 6.0   # 小时
C1, C2, C3, C4, C5 = '#2E5FA3', '#E07B39', '#3BA272', '#C0504D', '#8A6BBE'
```

```
plt.rcParams.update({'figure.dpi': 150, 'savefig.dpi': 300, 'axes.unicode_minus': False})

def style_ax(ax, xlabel=None, ylabel=None):
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)
    ax.tick_params(direction='in', labelsize=9)
    for s in ['bottom', 'left']:
        ax.spines[s].set_linewidth(0.9)
    if xlabel:
        ax.set_xlabel(xlabel, fontsize=10)
    if ylabel:
        ax.set_ylabel(ylabel, fontsize=10)

# 图1-1 预处理前后对比（原始时间点序列 vs 间隔对齐，整合为单张双轴图，不再拆成三个小图）
fig, ax = plt.subplots(figsize=(11.5, 5.4))
t_raw = np.arange(N_INT + 1) / 6.0   # 原始时间点序列（含'0:00+1'→24:00边界点）

# 左轴：负载、光伏（单位均为 kW，量级接近，共轴对比）
ax.plot(t_raw, np.append(load, load[0]), '--', color='#9AA5B1', lw=1.1, label='负载·原始(时间点)')
ax.plot(t, load, '-', color=C2, lw=1.8, label='负载·间隔对齐')
ax.plot(t_raw, np.append(pv, pv[0]), '--', color='#B8C6D9', lw=1.1, label='光伏·原始(时间点)')
ax.plot(t, pv, '-', color=C3, lw=1.8, label='光伏·间隔对齐')
style_ax(ax, '小时', '负载 / 光伏 (kW)')
ax.set_xticks(np.arange(0, 25, 6))
ax.set_xlim(-0.3, 24.3)
ax.set_ylim(0, max(load.max(), pv.max()) * 1.12)

# 右轴：电价（元/kWh，量级与功率不同，单独成轴）
ax2 = ax.twinx()
ax2.plot(t_raw, np.append(price, price[0]), '--', color='#CBB7A5', lw=1.1, label='电价·原始(时间点)')
ax2.plot(t, price, '-', color=C1, lw=1.8, label='电价·间隔对齐')
ax2.set_ylabel('电价 (元/kWh)', fontsize=10)
ax2.set_ylim(0, price.max() * 1.12)
ax2.tick_params(direction='in', labelsize=9)
ax2.spines['top'].set_visible(False)
ax2.spines['bottom'].set_visible(False)
```

```
ax2.spines['right'].set_linewidth(0.9)

# 合并双轴图例（单图单图例，说明三条序列预处理前后均吻合）
h1, l1 = ax.get_legend_handles_labels()
h2, l2 = ax2.get_legend_handles_labels()
ax.legend(h1 + h2, l1 + l2, fontsize=8, loc='upper center', bbox_to_anchor
    =(0.5, -0.14), ncol=3, frameon=False)
ax.text(0.01, 0.02, '(a) 数据预处理前后对比', transform=ax.transAxes,
    fontsize=10, fontweight='bold')
save_fig(fig, '图1-1_预处理前后对比')

# 图1-2 分时电价与最优购电功率（双轴）
fig, ax = plt.subplots(figsize=(11.5, 5.2))
ax.bar(t, g, width=0.12, color=C1, alpha=0.85, edgecolor='black', lw=0.6,
    label='购电功率')
style_ax(ax, '小时', '购电功率 (kW)')
ax.set_xticks(np.arange(0, 25, 6))
ax.set_xlim(-0.3, 24.3)
ax2 = ax.twinx()
ax2.plot(t, price, color=C4, lw=1.6, label='电价')
ax2.set_ylabel('电价 (元/kWh)', fontsize=10)
ax2.tick_params(direction='in', labelsize=9)
ax2.spines['top'].set_visible(False)
ax2.spines['bottom'].set_visible(False)
ax2.spines['right'].set_linewidth(0.9)
h1, l1 = ax.get_legend_handles_labels()
h2, l2 = ax2.get_legend_handles_labels()
ax.legend(h1 + h2, l1 + l2, fontsize=9, loc='upper left', framealpha=0.9)
ax.text(0.01, 0.02, '(a) 分时电价与购电功率', transform=ax.transAxes,
    fontsize=10, fontweight='bold')
save_fig(fig, '图1-2_电价与购电策略')

# 图1-3 负载/光伏/净负荷与储能充放电
fig, ax = plt.subplots(figsize=(11.5, 5.2))
net = load - pv
ax.plot(t, load, color=C2, lw=1.6, label='小区负载')
ax.plot(t, pv, color=C3, lw=1.6, label='光伏功率')
ax.plot(t, net, color=C4, lw=1.3, ls='--', label='净负荷(负载-光伏)')
ax.bar(t, ch, width=0.1, color=C5, alpha=0.7, edgecolor='black', lw=0.4,
    label='充电')
ax.bar(t, -dch, width=0.1, color=C1, alpha=0.7, edgecolor='black', lw=0.4,
    label='放电')
```

```
style_ax(ax, '小时', '功率 (kW)')
ax.axhline(0, color='black', lw=0.8)
ax.set_xticks(np.arange(0, 25, 6)); ax.set_xlim(-0.3, 24.3)
ax.legend(fontsize=8, loc='upper right', framealpha=0.9, ncol=2)
ax.text(0.01, 0.02, '(a) 负载-光伏-净负荷与储能充放电', transform=ax.
    transAxes, fontsize=10, fontweight='bold')
save_fig(fig, '图1-3_负荷光伏净负荷与充放电')

# 图1-4 储能SOC轨迹与充放电功率（双子图，共享x）
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(11.5, 6.4), sharex=True)
ax1.plot(t, E[:144], color=C1, lw=1.8)
ax1.fill_between(t, EMIN, E[:144], color=C1, alpha=0.12)
ax1.axhline(EMIN, color='#7A7A7A', lw=0.8, ls=':')
ax1.axhline(EMAX, color='#7A7A7A', lw=0.8, ls=':')
ax1.set_ylim(EMIN - 400, EMAX + 400)
style_ax(ax1, None, '储电量 (kWh)')
ax1.set_xticks(np.arange(0, 25, 6))
ax2.bar(t, ch, width=0.12, color=C5, edgecolor='black', lw=0.5, label='充电
    ')
ax2.bar(t, -dch, width=0.12, color=C4, edgecolor='black', lw=0.5, label='放
    电')
ax2.axhline(0, color='black', lw=0.8)
style_ax(ax2, '小时', '充/放电功率 (kW)')
ax2.legend(fontsize=9, loc='upper right', framealpha=0.9)
ax1.text(0.01, 0.02, '(a) SOC轨迹', transform=ax1.transAxes, fontsize=10,
    fontweight='bold')
ax2.text(0.01, 0.02, '(b) 充放电功率', transform=ax2.transAxes, fontsize
    =10, fontweight='bold')
save_fig(fig, '图1-4_储能SOC与充放电功率')

# 图1-5 能量平衡（24小时堆叠面积：购电+光伏+放电 = 负载+充电）
fig, ax = plt.subplots(figsize=(11.5, 5.2))
supply = np.vstack([g, pv, dch])
ax.stackplot(t, supply, labels=['购电', '光伏', '储能放电'], colors=[C1, C3
    , C4],
             alpha=0.82, edgecolor='black', linewidth=0.5)
ax.plot(t, load, color='black', lw=1.6, label='小区负载')
ax.plot(t, load + ch, color='#5A5A5A', lw=1.2, ls='--', label='负载+充电')
style_ax(ax, '小时', '功率 (kW)')
ax.set_xticks(np.arange(0, 25, 6)); ax.set_xlim(-0.3, 24.3)
ax.legend(fontsize=8, loc='upper right', framealpha=0.9)
ax.text(0.01, 0.02, '(a) 能量平衡关系', transform=ax.transAxes, fontsize
```

```
    =10, fontweight='bold')
save_fig(fig, '图1-5_能量平衡')

# 图1-6 灵敏度：购电费随0:00储电量初值变化（固定 E0 并满足日循环 E24=E0）
e0s = np.linspace(EMIN, EMAX, 21)
costs = []
for e0 in e0s:
    # 固定 E0=e0 且要求 E24=e0（周期），观察购电费
    _, _, _, _, c_ = solve_daily(load, pv, price, e0=float(e0), soc_cycle=
    True)
    costs.append(c_)
costs = np.array(costs)
cost_at_given = costs[np.argmin(np.abs(e0s - E0_INIT))]
fig, ax = plt.subplots(figsize=(11.5, 5.2))
ax.plot(e0s, costs, '-o', color=C1, lw=1.6, ms=4, markeredgecolor='black')
ax.axvline(E0_INIT, color=C4, lw=1.0, ls='--')
ax.plot(E0_INIT, cost_at_given, 'o', color=C4, ms=6)
ax.annotate(f'附录1给定 E0=6000 kWh\n对应购电费={cost_at_given:.0f} 元',
            xy=(E0_INIT, cost_at_given),
            xytext=(E0_INIT + 500, costs.min() + (costs.max() - costs.min()
    ) * 0.55), fontsize=9,
            arrowprops=dict(arrowstyle='->', lw=0.9))
style_ax(ax, '0:00储电量 E0 (kWh)', '全天购电费 (元)')
ax.text(0.02, 0.02, '(a) 购电费对储能初值灵敏度', transform=ax.transAxes,
    fontsize=10, fontweight='bold')
save_fig(fig, '图1-6_购电费灵敏度')

# ---------- 4. 写 result1.xlsx（完整的计划购电策略） ----------
import openpyxl
from openpyxl import Workbook
from openpyxl.styles import Font
from openpyxl.utils import get_column_letter

labs = interval_labels()
# 每个4小时段的购电量（用于充放电量表，与表2对齐）
gE_blocks = []
for a, b in blocks:
    k0, k1 = a * 6, b * 6 - 1
    gE_blocks.append(gE[k0:k1 + 1].sum())

wb = Workbook()

```

```
# Sheet1 完整计划购电策略：144个间隔的购电功率/购电量 + 储能充放电功率 + 储电量(SOC)
ws1 = wb.active
ws1.title = '计划购电策略'
ws1.append(['间隔序号', '时间段', '电价(元/kWh)', '计划购电功率(kW)', '计划购电量(kWh)',
            '充电功率(kW)', '放电功率(kW)', '储电量(kWh)'])
for k in range(N_INT):
    ws1.append([
        k, labs[k],
        round(float(price[k]), 4),
        round(float(g[k]), 4),
        round(float(gE[k]), 4),
        round(float(ch[k]), 4),
        round(float(dch[k]), 4),
        round(float(E[k + 1]), 4),
    ])

# Sheet2 6个4小时段的充放电量与购电量汇总
ws2 = wb.create_sheet('充放电量')
ws2.append(['时间段', '充电量', '放电量', '购电量', '时刻', '储电量'])
for i, (a, b) in enumerate(blocks):
    lab = f'{a:02d}:00-{b:02d}:00'
    _, cq, dq = block_rec[i]
    if i == 0:
        ws2.append([lab, round(cq, 4), round(dq, 4), round(gE_blocks[i], 4), '0:00', round(float(E0), 4)])
    elif i == 1:
        ws2.append([lab, round(cq, 4), round(dq, 4), round(gE_blocks[i], 4), '24:00', round(float(E24), 4)])
    else:
        ws2.append([lab, round(cq, 4), round(dq, 4), round(gE_blocks[i], 4), None, None])

# Sheet3 汇总
ws3 = wb.create_sheet('汇总')
ws3.append(['指标', '数值'])
ws3.append(['全天购电量(kWh)', round(float(gE.sum()), 4)])
ws3.append(['全天购电费(元)', round(float(cost), 4)])
ws3.append(['全天充电量(kWh)', round(float(chE.sum()), 4)])
ws3.append(['全天放电量(kWh)', round(float(dchE.sum()), 4)])
ws3.append(['0:00储电量(kWh)', round(float(E0), 4)])
```

```
ws3.append(['24:00储电量(kWh)', round(float(E24), 4)])

# 样式：表头加粗、冻结首行、设置列宽
hdr_font = Font(bold=True)
for ws in (ws1, ws2, ws3):
    for cell in ws[1]:
        cell.font = hdr_font
    ws.freeze_panes = 'A2'
for i, w in enumerate([10, 14, 13, 18, 16, 14, 14, 14], 1):
    ws1.column_dimensions[get_column_letter(i)].width = w
for i, w in enumerate([14, 14, 14, 14, 10, 12], 1):
    ws2.column_dimensions[get_column_letter(i)].width = w
ws3.column_dimensions['A'].width = 22
ws3.column_dimensions['B'].width = 18

out1 = os.path.join(ROOT_OUT, 'result1.xlsx')
wb.save(out1)
wb.save(os.path.join(OUT_DIR, 'result1.xlsx'))
print('\n已保存 result1.xlsx（完整计划购电策略）→ 求解/结果/ 与 问题一/结果/')
print('全部输出图完成')

# ---------- 5. 参数灵敏度（供第六章灵敏度分析，固定 E0=6000） ----------
import _models as M
print('\n第六章灵敏度分析（固定 E0=6000）:')
print(f'  基准购电费={cost:.2f} 元')

def _sens_cost(**kwargs):
    saved = {k: getattr(M, k) for k in kwargs}
    for k, v in kwargs.items():
        setattr(M, k, v)
    _, _, _, _, c_ = solve_daily(load, pv, price, e0=E0_INIT, soc_cycle=True)
    for k, v in saved.items():
        setattr(M, k, v)
    return c_

for label, kw in [
    ('ETA=0.85', {'ETA': 0.85}),
    ('ETA=0.95', {'ETA': 0.95}),
    ('PMAX=4000', {'PMAX': 4000.0}),
    ('PMAX=6000', {'PMAX': 6000.0}),
```

```
    ('EMIN=1500', {'EMIN': 1500.0}),
]:
    c_ = _sens_cost(**kw)
    print(f'  {label}: {c_:.2f} 元, 相对基准 {100*(c_-cost)/cost:+.2f}%')
for label, scale in [('load+5%', 1.05), ('load-5%', 0.95)]:
    _, _, _, _, c_ = solve_daily(load * scale, pv, price, e0=E0_INIT,
    soc_cycle=True)
    print(f'  {label}: {c_:.2f} 元, 相对基准 {100*(c_-cost)/cost:+.2f}%')
```

B.4 问题二求解程序 问题二.py

```
# 问题二: 每天电价相同, 负载与光伏随时间变化; 0:00 预测 — 决策 — 结算 三段式
# ① 预测段: 0:00 用历史数据滚动预测当日负载与光伏 (绝不用当日真值)
# ② 决策段: 不确定下的两阶段随机 LP 定计划购电量与充放电计划
# ③ 结算段: 按计划执行, 实际缺口以 5 倍电价紧急购电补足, 富余弃电
import sys, os
HERE = os.path.dirname(os.path.abspath(__file__))
SOLVE = os.path.dirname(HERE)
sys.path.insert(0, SOLVE)
from _common import *
from _models import *

price_daily, _, _ = read_附件1()          # 每日相同电价 (附件1, 日内随时段波动)
dates, load_all, pv_all = read_附件2()
N = len(dates)
S = 31                                    # 2025-02-01 索引; 1 月作预测模型预热与情景标定窗口
SPEC = ['2025-03-20', '2025-06-21', '2025-09-23', '2025-12-21']
idx_map = {}
for i, dd in enumerate(dates):
    ds = f'{dd.year:04d}-{dd.month:02d}-{dd.day:02d}'
    if ds in SPEC:
        idx_map[ds] = i
assert len(idx_map) == 4, f'指定日期缺失: {idx_map}'
print('附件2 日期范围:', dates[0].date(), '→', dates[-1].date(), '天数', N)
print(f'统计口径: 2025-02-01 → 2025-12-31 共 {N - S} 天; 1 月为预热期')

# ---------- 三段式逐日滚动 ----------
g_all = np.zeros((N, N_INT)); ch_all = np.zeros((N, N_INT)); dch_all = np.zeros((N, N_INT))
```

```
dl_all = np.zeros((N, N_INT)); su_all = np.zeros((N, N_INT))          # 紧急购电(kW)、弃电(kWh)
E_all = np.zeros((N, N_INT + 1))
plan_all = np.zeros(N); emg_all = np.zeros(N)                        # 日计划费、日紧急费(元)
Lf_all = np.full((N, N_INT), np.nan); Sf_all = np.full((N, N_INT), np.nan)
fL_all = np.full((N, 3), np.nan); fP_all = np.full((N, 3), np.nan)
res_pool_L, res_pool_P = [], []
PI = np.outer([0.2, 0.6, 0.2], [0.2, 0.6, 0.2]).ravel()              # 9 情景 = 3 负载 × 3 光伏
carry = E0_INIT
for d in range(N):
    if d == 0:
        # 1/1 0:00 尚无任何历史数据，计划量置零，储电量全天保持 6000 kWh
        E_all[d] = np.full(N_INT + 1, E0_INIT)
        continue
    L_fc = forecast_load(load_all[:d], dates[:d], dates[d])
    S_fc = forecast_pv(pv_all[:d])
    fL, fP = scenario_factors(np.concatenate(res_pool_L) if res_pool_L else np.array([]),
                              np.concatenate(res_pool_P) if res_pool_P else np.array([]))
    LF = np.repeat(fL, 3); SF = np.tile(fP, 3)
    g, ch, dch, E, _ = solve_daily_stochastic(L_fc, S_fc, price_daily, LF, SF, PI, e0=carry)
    delta, _, plan_cost, emerg_cost = settle_emergency(load_all[d], pv_all[d], price_daily, g, ch, dch)
    g_all[d], ch_all[d], dch_all[d], E_all[d] = g, ch, dch, E
    dl_all[d] = delta; plan_all[d] = plan_cost; emg_all[d] = emerg_cost
    su_all[d] = np.maximum(0.0, pv_all[d] + g + dch - ch - load_all[d]) * DT
    Lf_all[d], Sf_all[d], fL_all[d], fP_all[d] = L_fc, S_fc, fL, fP
    carry = float(E[-1])
    # 把当日"事前预测—事后实际"的残差并入标定池，供次日及以后使用
    res_pool_L.append(pred_residual(L_fc, load_all[d]))
    m = S_fc > 50.0
    if m.any():
        res_pool_P.append(((pv_all[d] - S_fc) / np.maximum(S_fc, 1.0))[m])

# ---------- 汇总指标（2.1—12.31） ----------
gE = g_all * DT; chE = ch_all * DT; dchE = dch_all * DT
W = slice(S, N); nW = N - S
```

```
plan_kwh = float(gE[W].sum()); plan_fee = float(plan_all[W].sum())
emg_kwh = float(dl_all[W].sum() * DT); emg_fee = float(emg_all[W].sum())
tot_fee = plan_fee + emg_fee
n_emg_int = int((dl_all[W].reshape(-1) > 1e-6).sum())
n_emg_day = int((dl_all[W].sum(axis=1) > 1e-6).sum())
net_kwh = float((load_all[W] - pv_all[W]).sum() * DT)
print(f'\n[计划段] 计划购电量={plan_kwh:.2f} kWh, 计划购电费={plan_fee:.2f}
    元')
print(f'[紧急段] 紧急购电量={emg_kwh:.2f} kWh, 紧急购电费={emg_fee:.2f} 元'
    )
print(f'[合计]   总购电费={tot_fee:.2f} 元, 日均={tot_fee / nW:.2f} 元')
print(f'[紧急覆盖面] 非零间隔={n_emg_int} 个 ({n_emg_int / (nW * N_INT) *
    100:.2f}%), '
      f'发生天数={n_emg_day} 天 ({n_emg_day / nW * 100:.2f}%)')
print(f'[紧急强度] 最大功率={dl_all[W].max():.2f} kW, 单日最大电量={float((
    dl_all[W].sum(axis=1) * DT).max()):.2f} kWh')
print(f'[储能] 2/1 0:00 SOC={E_all[S, 0]:.2f}, 12/31 24:00 SOC={E_all[N -
    1, -1]:.2f}, '
      f'SOC 最小={E_all[W, :].min():.2f}, SOC 最大={E_all[W, :].max():.2f}'
    )
print(f'[能量流] 净负荷={net_kwh:.2f} kWh, 放电量={float(dchE[W].sum()):.2f
    } kWh, '
      f'充电量={float(chE[W].sum()):.2f} kWh, 弃电={float(su_all[W].sum())
    :.2f} kWh')
bal = net_kwh - plan_kwh - float(dchE[W].sum()) + float(chE[W].sum())
print(f'[自洽校验] 净负荷-计划-放电+充电={bal:.2f} = 紧急电量({emg_kwh:.2f
    }) - 弃电({float(su_all[W].sum()):.2f})')

# ---------- 预测精度（2.1—12.31 窗口内 out-of-sample） ----------
err_L = (load_all - Lf_all)[W].reshape(-1); err_P = (pv_all - Sf_all)[W].
    reshape(-1)
rel_L = ((load_all - Lf_all) / np.maximum(Lf_all, 1.0))[W].reshape(-1)
maskP = (Sf_all[W] > 50.0).reshape(-1)
rel_P = ((pv_all - Sf_all) / np.maximum(Sf_all, 1.0))[W].reshape(-1)[maskP]
q = np.arange(0, 101, 10)
print(f'\n[预测精度] 负载 MAE={np.mean(np.abs(err_L)):.2f} kW, RMSE={np.
    sqrt(np.mean(err_L ** 2)):.2f} kW')
print(f'           光伏 MAE={np.mean(np.abs(err_P)):.2f} kW, RMSE={np.sqrt(
    np.mean(err_P ** 2)):.2f} kW')
print(f'           负载相对误差分位(%)=[{", ".join(f"{v:.2f}" for v in np.
    percentile(rel_L, q) * 100)}], '
      f'std={np.std(rel_L) * 100:.2f}%')
```

```
print(f'            光伏相对误差分位(%)=[{", ".join(f"{v:.2f}" for v in np.percentile(rel_P, q) * 100)}] '
      f'(样本 {maskP.sum()}), std={np.std(rel_P) * 100:.2f}%')
for nm, arr in (('负载', fL_all[S:]), ('光伏', fP_all[S:])):
    print(f'[情景因子] {nm} 低/中/高 均值=[{", ".join(f"{v:.4f}" for v in arr.mean(axis=0))}], '
          f'低档 min/median/max={arr[:, 0].min():.4f}/{np.median(arr[:, 0]):.4f}/{arr[:, 0].max():.4f}, '
          f'高档 min/median/max={arr[:, 2].min():.4f}/{np.median(arr[:, 2]):.4f}/{arr[:, 2].max():.4f}')

# ---------- 表1 / 表2 / 表3: 四个指定日期 ----------
def merge_runs(delta, thr=1e-6):
    """把非零紧急购电的连续间隔合并为 (起时刻, 止时刻, 电量kWh)。"""
    labs = interval_labels(); out = []; i = 0; n = len(delta)
    while i < n:
        if delta[i] > thr:
            j = i
            while j + 1 < n and delta[j + 1] > thr:
                j += 1
            out.append((labs[i].split('-')[0], labs[j].split('-')[1].replace('+1', ''),
                        float(np.sum(delta[i:j + 1]) * DT)))
            i = j + 1
        else:
            i += 1
    return out

print('\n===== 表1 微网在指定时间段的购电量及全天的购电量和购电费（计划）=====')
print('日期        10:00-10:10 12:00-12:10 14:00-14:10 16:00-16:10 18:00-18:10 20:00-20:10 全天购电量 全天购电费')
tab1 = {}
for sd in SPEC:
    d = idx_map[sd]; gd = gE[d]
    vals = [float(gd[h * 6]) for h in (10, 12, 14, 16, 18, 20)]
    tot = float(gd.sum())
    tab1[sd] = (vals, tot, float(plan_all[d]), float(emg_all[d]), float(dl_all[d].sum() * DT))
    print(f'{sd}  ' + ' '.join(f'{v:10.2f}' for v in vals) + f' {tot:9.2f} {plan_all[d]:10.2f}'
          f'  [紧急 {float(dl_all[d].sum() * DT):.2f} kWh / {emg_all[d]:.2f
```

```
    } 元]')

print('\n===== 表2 储能设备在指定时间段的充放电量及0:00和24:00的储电量 =====')
tab2 = {}
for sd in SPEC:
    d = idx_map[sd]; ce = chE[d]; de = dchE[d]
    segs = []
    for a, b in [(0, 4), (4, 8), (8, 12), (12, 16), (16, 20), (20, 24)]:
        k0, k1 = a * 6, b * 6 - 1
        segs.append((f'{a:02d}:00-{b:02d}:00', float(ce[k0:k1 + 1].sum()), float(de[k0:k1 + 1].sum())))
    tab2[sd] = (segs, float(E_all[d, 0]), float(E_all[d, -1]))
    print(f'  {sd}  0:00储电量={E_all[d, 0]:.2f}  24:00储电量={E_all[d, -1]:.2f}')
    for nm, c, dn in segs:
        print(f'    {nm}  充电量={c:9.2f}  放电量={dn:9.2f}')

print('\n===== 表3 微网在指定日期的紧急购电量 =====')
tab3 = {}
for sd in SPEC:
    d = idx_map[sd]
    runs = merge_runs(dl_all[d])
    tab3[sd] = runs
    tot = float(dl_all[d].sum() * DT)
    if not runs:
        print(f'  {sd}: 无紧急购电')
    else:
        print(f'  {sd}: 合计 {tot:.2f} kWh, 共 {len(runs)} 段')
        for a, b, e in runs:
            print(f'    {a}-{b}  {e:.2f} kWh')

# ---------- 预处理: 缺失/异常检查 ----------
miss = int(np.isnan(load_all).sum() + np.isnan(pv_all).sum())
print(f'\n附件2 缺失值总数={miss}, 负载范围=[{load_all.min():.1f},{load_all.max():.1f}] kW, '
      f'光伏范围=[{pv_all.min():.1f},{pv_all.max():.1f}] kW')

# ---------- 出图 ----------
t = np.arange(N_INT) / 6.0
C1, C2, C3, C4, C5 = '#2E5FA3', '#E07B39', '#3BA272', '#C0504D', '#8A6BBE'
dw = np.array([dates[d] for d in range(S, N)])
```

```
161
162 def style_ax(ax, xlabel=None, ylabel=None):
163     ax.spines['top'].set_visible(False); ax.spines['right'].set_visible(
    False)
164     ax.tick_params(direction='in', labelsize=9)
165     for s in ['bottom', 'left']:
166         ax.spines[s].set_linewidth(0.9)
167     if xlabel: ax.set_xlabel(xlabel, fontsize=10)
168     if ylabel: ax.set_ylabel(ylabel, fontsize=10)
169
170 def tag(ax, s, bold=True):
171     ax.text(0.02, 0.02, s, transform=ax.transAxes, fontsize=9,
172             fontweight='bold' if bold else 'normal')
173
174 # 图2-1 预处理前后对比（附件2 全年完整性 + 样本日对齐）
175 fig, axes = plt.subplots(1, 3, figsize=(15.6, 5.0))
176 d0 = idx_map['2025-03-20']
177 ax = axes[0]
178 ax.plot(dates, load_all.max(axis=1), '.', ms=3, color=C2, alpha=0.5, label=
    '负载峰值')
179 ax.plot(dates, pv_all.max(axis=1), '.', ms=3, color=C3, alpha=0.5, label='
    光伏峰值')
180 style_ax(ax, '日期', '功率峰值 (kW)')
181 ax.legend(fontsize=8, loc='upper right', framealpha=0.9)
182 ax = axes[1]
183 daily_na = np.isnan(load_all).sum(axis=1) + np.isnan(pv_all).sum(axis=1)
184 ax.bar(np.arange(12), [int(daily_na[m::12].sum()) for m in range(12)],
    color=C1, edgecolor='black', lw=0.5)
185 ax.set_xticks(np.arange(12)); ax.set_xticklabels([f'{m + 1}月' for m in
    range(12)], fontsize=8)
186 style_ax(ax, '月份', '缺失值计数')
187 ax = axes[2]
188 ax.plot(np.arange(N_INT + 1) / 6.0, np.append(load_all[d0], load_all[d0,
    0]), '-', color='#9AA5B1', lw=1.2, label='原始时间点')
189 ax.plot(t, load_all[d0], '-', color=C2, lw=1.8, label='间隔对齐')
190 style_ax(ax, '小时', '负载 (kW)')
191 ax.legend(fontsize=8, loc='upper right', framealpha=0.9)
192 for ax_, tg, name in zip(axes, 'abc', ['全年数据完整性', '缺失值月度分布',
    '样本日负载对齐']):
193     tag(ax_, f'({tg}) {name}')
194 save_fig(fig, '图2-1_预处理前后对比')
195
```

```
# 图2-2 2.1—12.31 逐日购电费（计划 + 紧急 叠加柱）
fig, ax = plt.subplots(figsize=(11.5, 5.2))
pv_e = plan_all[S:]; ev_e = emg_all[S:]
ax.bar(dw, pv_e, width=0.85, color=C1, edgecolor='black', lw=0.15, label='计划购电费')
ax.bar(dw, ev_e, bottom=pv_e, width=0.85, color=C4, edgecolor='black', lw=0.15, label='紧急购电费')
ax.plot(dw, pd.Series(pv_e + ev_e).rolling(7, center=True).mean(), color='black', lw=1.6, label='总费用7日滑动平均')
style_ax(ax, '日期', '购电费（元）')
ax.legend(fontsize=9, loc='upper left', framealpha=0.9)
tag(ax, '(a) 2.1—12.31逐日购电费构成')
save_fig(fig, '图2-2_全年逐日购电费')

# 图2-3 四季典型日购电功率（计划 + 紧急 叠加柱）
fig, axes = plt.subplots(1, 4, figsize=(15.6, 4.6))
for ax, sd, col in zip(axes, SPEC, [C1, C3, C4, C2]):
    d = idx_map[sd]
    ax.bar(t, g_all[d], width=0.14, color=col, alpha=0.85, edgecolor='black', lw=0.4, label='计划')
    ax.bar(t, dl_all[d], bottom=g_all[d], width=0.14, color=C5, alpha=0.9, edgecolor='black', lw=0.4, label='紧急')
    ax.set_xticks(np.arange(0, 25, 6)); style_ax(ax, '小时', '购电（kW）')
    top = max(g_all[d].max(), (g_all[d] + dl_all[d]).max()) * 1.18
    ax.set_ylim(0, top)
    tag(ax, f'({sd[5:]})')
axes[0].legend(fontsize=8, loc='upper left', framealpha=0.9)
save_fig(fig, '图2-3_四季典型日购电策略')

# 图2-4 四季典型日SOC轨迹
fig, axes = plt.subplots(1, 4, figsize=(15.6, 4.6))
for ax, sd, col in zip(axes, SPEC, [C1, C3, C4, C2]):
    d = idx_map[sd]
    ax.plot(t, E_all[d, :N_INT], color=col, lw=1.8)
    ax.fill_between(t, EMIN, E_all[d, :N_INT], color=col, alpha=0.12)
    ax.axhline(EMIN, color='#7A7A7A', lw=0.8, ls=':'); ax.axhline(EMAX, color='#7A7A7A', lw=0.8, ls=':')
    ax.set_xticks(np.arange(0, 25, 6)); style_ax(ax, '小时', '储电量（kWh）')
    ax.set_ylim(EMIN - 400, EMAX + 400)
    tag(ax, f'({sd[5:]})')
save_fig(fig, '图2-4_四季典型日SOC轨迹')
```

```
231
232 # 图2-5 储能SOC月度分布（2—12月，箱线）
233 fig, ax = plt.subplots(figsize=(11.5, 5.2))
234 monthly = [[E_all[d, 0] for d in range(S, N) if dates[d].month == m] for m
    in range(2, 13)]
235 bp = safe_boxplot(ax, monthly, [f'{m}月' for m in range(2, 13)])
236 cols = get_colors('tab20', 11)
237 for i, b in enumerate(bp['boxes']):
238     b.set_facecolor(cols[i]); b.set_edgecolor('black'); b.set_linewidth
    (0.9)
239 for med in bp['medians']:
240     med.set_color('black'); med.set_linewidth(1.2)
241 style_ax(ax, '月份', '每日0:00储电量 (kWh)')
242 ax.axhline(EMIN, color='#7A7A7A', lw=0.8, ls=':')
243 tag(ax, '(a) 2—12月0:00储电量月度分布')
244 save_fig(fig, '图2-5_全年储能SOC月度分布')
245
246 # 图2-6 购电量与净负荷关系 + 月度购电费构成
247 fig, axes = plt.subplots(1, 2, figsize=(15.6, 5.4))
248 ax = axes[0]
249 net_all = (load_all - pv_all).mean(axis=1)
250 ax.scatter(net_all[S:], g_all.mean(axis=1)[S:], s=10, c=(plan_all + emg_all
    )[S:], cmap='RdYlGn_r',
251            edgecolor='black', lw=0.3)
252 style_ax(ax, '日均净负荷 (kW)', '日均计划购电功率 (kW)')
253 cb = fig.colorbar(ax.collections[0], ax=ax, fraction=0.046, pad=0.04)
254 cb.set_label('购电费 (元)', fontsize=9); cb.ax.tick_params(labelsize=8)
255 ax = axes[1]
256 months = np.arange(2, 13)
257 mp = np.array([plan_all[[d for d in range(S, N) if dates[d].month == m]].
    sum() / 1e4 for m in months])
258 me = np.array([emg_all[[d for d in range(S, N) if dates[d].month == m]].sum
    () / 1e4 for m in months])
259 ax.bar(months - 0.18, mp, width=0.36, color=C1, edgecolor='black', lw=0.5,
    label='计划购电费')
260 ax.bar(months + 0.18, me, width=0.36, color=C4, edgecolor='black', lw=0.5,
    label='紧急购电费')
261 ax.set_xticks(months); ax.set_xticklabels([f'{m}月' for m in months],
    fontsize=9)
262 style_ax(ax, '月份', '月度购电费 (万元)')
263 ax.legend(fontsize=9, loc='upper left', framealpha=0.9)
264 tag(axes[0], '(a) 购电量-净负荷关系'); tag(ax, '(b) 月度购电费构成')
```

```
save_fig(fig, '图2-6_购电量与净负荷关系')

# 图2-7 预测与实际对比（上排负载、下排光伏，四个指定日）
fig, axes = plt.subplots(2, 4, figsize=(15.6, 7.6))
for j, sd in enumerate(SPEC):
    d = idx_map[sd]
    ax = axes[0, j]
    ax.plot(t, load_all[d], color=C2, lw=1.6, label='实际')
    ax.plot(t, Lf_all[d], color=C1, lw=1.4, ls='--', label='0:00预测')
    ax.set_xticks(np.arange(0, 25, 6)); style_ax(ax, '小时', '负载 (kW)' if
    j == 0 else None)
    tag(ax, f'({chr(97 + j)}) {sd[5:]} 负载')
    if j == 0: ax.legend(fontsize=8, loc='upper right', framealpha=0.9)
    ax = axes[1, j]
    ax.plot(t, pv_all[d], color=C3, lw=1.6, label='实际')
    ax.plot(t, Sf_all[d], color=C1, lw=1.4, ls='--', label='0:00预测')
    ax.set_xticks(np.arange(0, 25, 6)); style_ax(ax, '小时', '光伏 (kW)' if
    j == 0 else None)
    tag(ax, f'({chr(101 + j)}) {sd[5:]} 光伏')
save_fig(fig, '图2-7_预测与实际对比')

# 图2-8 预测误差分布
fig, axes = plt.subplots(1, 2, figsize=(15.6, 5.2))
ax = axes[0]
bp = safe_boxplot(ax, [rel_L * 100, rel_P * 100], ['负载', '光伏'], whis
   =(5, 95), showfliers=False)
for b, c in zip(bp['boxes'], [C1, C3]):
    b.set_facecolor(c); b.set_edgecolor('black'); b.set_linewidth(0.9)
for med in bp['medians']:
    med.set_color('black'); med.set_linewidth(1.2)
ax.axhline(0, color='#7A7A7A', lw=0.8, ls=':')
style_ax(ax, '预测量', '相对预测误差 (%)')
tag(ax, '(a) 相对误差箱线（5—95分位）')
ax = axes[1]
ax.plot(q, np.percentile(rel_L, q) * 100, '-o', ms=3, color=C1, lw=1.6,
   label='负载')
ax.plot(q, np.percentile(rel_P, q) * 100, '-s', ms=3, color=C3, lw=1.6,
   label='光伏')
ax.axhline(0, color='#7A7A7A', lw=0.8, ls=':')
style_ax(ax, '分位数 (%)', '相对误差 (%)')
ax.legend(fontsize=9, loc='upper left', framealpha=0.9)
tag(ax, '(b) 相对误差分位曲线')
```

```
save_fig(fig, '图2-8_预测误差分布')

# 图2-9 紧急购电分布
fig, axes = plt.subplots(1, 2, figsize=(15.6, 5.4))
ax = axes[0]
ax.bar(dw, dl_all[S:].sum(axis=1) * DT, width=0.85, color=C4, edgecolor='black', lw=0.15)
style_ax(ax, '日期', '紧急购电量 (kWh/日)')
tag(ax, '(a) 逐日紧急购电量')
ax = axes[1]
hm = np.zeros((12, 24))
for d in range(S, N):
    for k in range(N_INT):
        hm[dates[d].month - 1, k // 6] += dl_all[d, k] * DT
mesh = safe_heatmap(ax, hm, row_labels=[f'{m}月' for m in range(1, 13)],
                    col_labels=[f'{h}时' for h in range(24)], cmap='YlOrRd', fontsize=7)
cb = fig.colorbar(mesh, ax=ax, fraction=0.046, pad=0.04)
cb.set_label('紧急购电量 (kWh)', fontsize=9); cb.ax.tick_params(labelsize=8)
style_ax(ax, '时段', '月份')
tag(ax, '(b) 月份-时段紧急购电量', bold=False)
save_fig(fig, '图2-9_紧急购电分布')

# ---------- 写 result2.xlsx ----------
from openpyxl import Workbook
wb = Workbook()
ws = wb.active; ws.title = '计划购电量'
labs = interval_labels()
ws.append(['日期\\时间'] + labs + ['全天购电量', '全天购电费'])
for d in range(S, N):
    ws.append([dates[d]] + [round(float(x), 4) for x in gE[d]]
              + [round(float(gE[d].sum()), 4), round(float(plan_all[d]), 4)])
ws2 = wb.create_sheet('充放电量')
ws2.append(['日期', '时间段', '充电量', '放电量', '时刻', '储电量'])
for d in range(S, N):
    for bi, (a, b) in enumerate([(0, 4), (4, 8), (8, 12), (12, 16), (16, 20), (20, 24)]):
        k0, k1 = a * 6, b * 6 - 1
        cq = round(float(chE[d, k0:k1 + 1].sum()), 4); dq = round(float(dchE[d, k0:k1 + 1].sum()), 4)
```

```
        if bi == 0:
            ws2.append([dates[d], f'{a:02d}:00-{b:02d}:00', cq, dq, '00:00', round(float(E_all[d, 0]), 4)])
        elif bi == 1:
            ws2.append([None, f'{a:02d}:00-{b:02d}:00', cq, dq, '24:00', round(float(E_all[d, -1]), 4)])
        else:
            ws2.append([None, f'{a:02d}:00-{b:02d}:00', cq, dq, None, None])
ws3 = wb.create_sheet('紧急购电量')
ws3.append(['日期', '紧急购电时间段', '紧急购电量'])
for d in range(S, N):
    runs = merge_runs(dl_all[d])
    for i, (a, b, e) in enumerate(runs):
        ws3.append([dates[d] if i == 0 else None, f'{a}-{b}', round(e, 4)])
for path in (os.path.join(ROOT_OUT, 'result2.xlsx'), os.path.join(OUT_DIR, 'result2.xlsx')):
    wb.save(path)
print(f'\n已保存 result2.xlsx（{nW} 天，3 张工作表）→ 求解/结果/ 与 问题二/结果/')
```

B.5 问题三求解程序 问题三.py

```
# 问题三：0/6/12/18时滚动预报 + 调整购电（违约0.5倍/超额1.5倍/紧急5倍）
import sys, os
HERE = os.path.dirname(os.path.abspath(__file__))
SOLVE = os.path.dirname(HERE)
sys.path.insert(0, SOLVE)
from _common import *
from _models import *

price_daily, _, _ = read_附件1()
dates, load_all, pv_actual_all = read_附件2()
fc = read_附件3()
print('附件3 预报天数:', len(fc))

def date_key(d):
    return f'{d.year}-{d.month}-{d.day}'

def run_day(d, verbose=False):
    """滚动求解一天，返回 dict"""
    L = load_all[d]
```

```
    S_act = pv_actual_all[d]
    key = date_key(dates[d])
    f = fc[key]
    # 各时刻预报 -> 全天10分钟（k0=0..143，未用到的前段由插值外延但不参与计算）
    S0 = forecast_10min(0, f[0], 0, 143)
    S6 = forecast_10min(6, f[6], 0, 143)
    S12 = forecast_10min(12, f[12], 0, 143)
    S18 = forecast_10min(18, f[18], 0, 143)
    # 0:00 计划
    g0, ch0, dch0, E0, c_plan = solve_daily(L, S0, price_daily, e0=carry[d])
    # 6:00 调整
    g6, ch6, dch6, E6, _ = solve_adjust(L, S6, price_daily, g0, e0=float(E0[36]), k0=36)
    # 12:00 调整
    g12, ch12, dch12, E12, _ = solve_adjust(L, S12, price_daily, g0, e0=float(E6[36]), k0=72)
    # 18:00 调整
    g18, ch18, dch18, E18, _ = solve_adjust(L, S18, price_daily, g0, e0=float(E12[36]), k0=108)
    # 拼接最终
    blocks = [0, 36, 72, 108, 144]
    sols = [(g0, ch0, dch0, E0), (g6, ch6, dch6, E6), (g12, ch12, dch12, E12), (g18, ch18, dch18, E18)]
    adj = np.zeros(144); chf = np.zeros(144); dchf = np.zeros(144); Ef = np.empty(145)
    Ef[0] = E0[0]
    for i in range(4):
        a, b = blocks[i], blocks[i + 1]
        g_, c_, d_, E_ = sols[i]
        adj[a:b] = g_[0:b - a]; chf[a:b] = c_[0:b - a]; dchf[a:b] = d_[0:b - a]
        Ef[a + 1:b + 1] = E_[1:b - a + 1]
    # 费用
    plan = g0
    base = np.minimum(plan, adj) * DT * price_daily
    down = np.maximum(plan - adj, 0) * DT * price_daily * 0.5
    up = np.maximum(adj - plan, 0) * DT * price_daily * 1.5
    plan_commit = (plan * DT * price_daily).sum()
    purchase_total = (base + down + up).sum()
    # 紧急购电 = 实际供电缺口：delta_k = max{L - (g_k + S^r_k + d_k - c_k),
```

```
    0}
    # 其中 g_k 为调整后最终购电、d_k/c_k 为最终放电/充电，S^r_k 为实际光伏。
    gap = np.maximum(L - (adj + S_act + dchf - chf), 0.0)
    emer_kwh = gap * DT
    emer_cost = (emer_kwh * 5.0 * price_daily).sum()
    total = purchase_total + emer_cost
    # 无调整基线：仅执行 0:00 计划，缺口按相同定义核算
    gap0 = np.maximum(L - (g0 + S_act + dch0 - ch0), 0.0)
    emer0_kwh = gap0 * DT
    total0 = plan_commit + (emer0_kwh * 5.0 * price_daily).sum()
    return dict(g0=g0, adj=adj, chf=chf, dchf=dchf, Ef=Ef, S0=S0, S6=S6, S12=S12, S18=S18,
                S_act=S_act, plan_commit=plan_commit, purchase_total=purchase_total,
                emer_kwh=emer_kwh, emer_cost=emer_cost, total=total, total0=total0)

def _fc_for(T, key, S_act, mode):
    """返回按时刻T发布的10分钟光伏预报（全144）。
    T=0 恒用 0:00 真实预报；mode='perfect' 时 T>0 全部用实际光伏（完全预见上界）；
    mode='real' 时附件3有预报的时刻用真实预报，其余时刻用实际光伏近似。"""
    f = fc[key]
    if T == 0:
        return forecast_10min(0, f[0], 0, 143)
    if mode == 'perfect' or T not in f:
        return S_act
    return forecast_10min(T, f[T], 0, 143)

def run_day_times(d, times, e0_start, mode='real'):
    """按给定调整时刻序列 times(小时,含0) 滚动求解一天，返回(当天总费用，次日初始储电量)。
    计划购电恒由 0:00 制定；后续每个时刻在剩余时段 [T,24:00] 重优化调整购电，
    购电费用=min(plan,adj)+0.5*max(plan-adj,0)+1.5*max(adj-plan,0)，缺口按5倍紧急购电。"""
    L = load_all[d]
    S_act = pv_actual_all[d]
    key = date_key(dates[d])
    S_list = [_fc_for(T, key, S_act, mode) for T in times]
    g0, ch0, dch0, E0, _ = solve_daily(L, S_list[0], price_daily, e0=
```

```
    e0_start)
    sols = [(g0, ch0, dch0, E0)]
    prev_k0, prev_E = 0, E0
    for i in range(1, len(times)):
        T = times[i]
        k0 = T * 6
        e0 = float(prev_E[k0 - prev_k0])
        adj, ch, dch, E, _ = solve_adjust(L, S_list[i], price_daily, g0, e0
    =e0, k0=k0)
        sols.append((adj, ch, dch, E))
        prev_k0, prev_E = k0, E
    ks = [0] + [T * 6 for T in times[1:]] + [144]
    adj = np.zeros(144); chf = np.zeros(144); dchf = np.zeros(144); Ef = np
    .empty(145)
    Ef[0] = E0[0]
    for i in range(len(times)):
        a, b = ks[i], ks[i + 1]
        g_, c_, d_, E_ = sols[i]
        adj[a:b] = g_[0:b - a]; chf[a:b] = c_[0:b - a]; dchf[a:b] = d_[0:b
    - a]
        Ef[a + 1:b + 1] = E_[1:b - a + 1]
    plan = g0
    base = np.minimum(plan, adj) * DT * price_daily
    down = np.maximum(plan - adj, 0) * DT * price_daily * 0.5
    up = np.maximum(adj - plan, 0) * DT * price_daily * 1.5
    purchase_total = (base + down + up).sum()
    gap = np.maximum(L - (adj + S_act + dchf - chf), 0.0)
    emer_cost = (gap * DT * 5.0 * price_daily).sum()
    return purchase_total + emer_cost, float(Ef[-1])

def run_year(times, mode='real', start=0):
    """全年滚动求总费用（carry 储电量逐日传递）。
    start 为汇总起始日索引（附件5要求仅统计 2.1—12.31，1月作为预热期，故
    start=31）。"""
    carry2 = [E0_INIT] * N
    tot = 0.0
    for d in range(N):
        t, e_end = run_day_times(d, times, carry2[d], mode)
        if d >= start:
            tot += t
        if d + 1 < N:
            carry2[d + 1] = e_end
```

```
    return tot

# 全年滚动（carry：储电量传递）
N = len(dates)
S = 31   # 2025-02-01 索引；附件5仅要求保存 2.1—12.31，1月作为预热期，不纳入汇总统计
carry = [E0_INIT] * N
rec = [None] * N
totals = np.zeros(N); totals0 = np.zeros(N)
for d in range(N):
    r = run_day(d)
    rec[d] = r
    totals[d] = r['total']; totals0[d] = r['total0']
    if d + 1 < N:
        carry[d + 1] = float(r['Ef'][-1])
print(f'全年(365天,含1月预热)含调整总费用={totals.sum():.2f} 元，无调整={totals0.sum():.2f} 元')
print(f'[汇总口径] 2.1—12.31 含调整={totals[S:].sum():.2f} 元，无调整={totals0[S:].sum():.2f} 元，'
      f'节省={totals0[S:].sum()-totals[S:].sum():.2f} 元，降幅={100*(totals0[S:].sum()-totals[S:].sum())/totals0[S:].sum():.3f}%')

# 指定四天详细结果
spec_days = ['2025-03-20', '2025-06-21', '2025-09-23', '2025-12-21']
idx = {}
for d in range(N):
    dd = pd.Timestamp(dates[d]).date()
    for sd in spec_days:
        if dd == pd.Timestamp(sd).date():
            idx[sd] = d
print('\n表1/表2/表3 指定日期:')
for sd in spec_days:
    d = idx[sd]; r = rec[d]
    gE = r['g0'] * DT; aE = r['adj'] * DT
    print(f'  {sd}: 计划购电量={gE.sum():.1f}，调整购电量={aE.sum():.1f}，'
          f'总费用={r["total"]:.2f}(计划{r["plan_commit"]:.0f}+调整{r["purchase_total"]-r["plan_commit"]:.0f}+紧急{r["emer_cost"]:.0f})')
    for hh in [10, 12, 14, 16, 18, 20]:
        k = hh * 6
        print(f'     {hh:02d}:00-{hh:02d}:10  计划={gE[k]:.2f}  调整={aE[k]:.2f}')

```

```python
# ---------- 出图 ----------
t = np.arange(144) / 6.0
C1, C2, C3, C4, C5, C6 = '#2E5FA3', '#E07B39', '#3BA272', '#C0504D', '#8A6BBE', '#D4A017'
plt.rcParams.update({'figure.dpi': 150, 'savefig.dpi': 300, 'axes.unicode_minus': False})

def style_ax(ax, xlabel=None, ylabel=None):
    ax.spines['top'].set_visible(False); ax.spines['right'].set_visible(False)
    ax.tick_params(direction='in', labelsize=9)
    for s in ['bottom', 'left']:
        ax.spines[s].set_linewidth(0.9)
    if xlabel: ax.set_xlabel(xlabel, fontsize=10)
    if ylabel: ax.set_ylabel(ylabel, fontsize=10)

sd = '2025-09-23'; d0 = idx[sd]; r0 = rec[d0]

# 图3-1 各时刻预报与实际光伏
fig, ax = plt.subplots(figsize=(11.5, 5.2))
ax.plot(t, r0['S_act'], color='black', lw=2.0, label='实际光伏')
ax.plot(t, r0['S0'], color=C1, lw=1.2, label='0:00预报')
ax.plot(t[36:], r0['S6'][36:], color=C2, lw=1.2, label='6:00预报')
ax.plot(t[72:], r0['S12'][72:], color=C3, lw=1.2, label='12:00预报')
ax.plot(t[108:], r0['S18'][108:], color=C4, lw=1.2, label='18:00预报')
style_ax(ax, '小时', '光伏功率 (kW)')
ax.set_xticks(np.arange(0, 25, 6)); ax.set_xlim(-0.3, 24.3)
ax.legend(fontsize=9, loc='upper right', framealpha=0.9)
ax.text(0.01, 0.02, f'(a) {sd} 各时刻预报与实际光伏', transform=ax.transAxes, fontsize=10, fontweight='bold')
save_fig(fig, '图3-1_预报与实际光伏对比')

# 图3-2 计划与最终调整购电量对比
fig, ax = plt.subplots(figsize=(11.5, 5.2))
ax.plot(t, r0['g0'], color=C1, lw=1.4, label='0:00计划购电')
ax.plot(t, r0['adj'], color=C4, lw=1.4, ls='--', label='滚动调整后购电')
style_ax(ax, '小时', '购电功率 (kW)')
ax.set_xticks(np.arange(0, 25, 6)); ax.set_xlim(-0.3, 24.3)
ax.legend(fontsize=9, loc='upper right', framealpha=0.9)
ax.text(0.01, 0.02, f'(a) {sd} 计划购电与调整购电对比', transform=ax.transAxes, fontsize=10, fontweight='bold')
save_fig(fig, '图3-2_计划与调整购电量对比')
```

```
197
198 # 图3-3 2.1—12.31 逐日紧急购电量
199 fig, ax = plt.subplots(figsize=(11.5, 5.2))
200 emer_daily = np.array([rec[d]['emer_kwh'].sum() for d in range(N)])
201 ax.bar(dates[S:], emer_daily[S:], width=1.0, color=C4, alpha=0.8, edgecolor
    ='none')
202 style_ax(ax, '日期', '紧急购电量 (kWh)')
203 ax.text(0.01, 0.02, '(a) 2.1—12.31逐日紧急购电量', transform=ax.transAxes,
     fontsize=10, fontweight='bold')
204 save_fig(fig, '图3-3_全年紧急购电量')
205
206 # 图3-4 有无调整费用对比 + 月度节省（2—12月）
207 fig, axes = plt.subplots(1, 2, figsize=(13.6, 5.0))
208 ax = axes[0]
209 ax.bar([0, 1], [totals0[S:].sum() / 1e4, totals[S:].sum() / 1e4], width
    =0.5,
210        color=[C1, C3], edgecolor='black', lw=0.8)
211 for i, v in enumerate([totals0[S:].sum() / 1e4, totals[S:].sum() / 1e4]):
212     ax.text(i, v + 15, f'{v:.1f}万元', ha='center', fontsize=10)
213 ax.set_xticks([0, 1]); ax.set_xticklabels(['仅0:00计划', '滚动调整'],
    fontsize=10)
214 style_ax(ax, None, '2.1—12.31购电费 (万元)')
215 ax.text(0.03, 0.02, '(a) 有无调整2.1—12.31费用', transform=ax.transAxes,
    fontsize=10, fontweight='bold')
216 ax = axes[1]
217 msave = []
218 for m in range(1, 12):
219     dd = [d for d in range(N) if dates[d].month == m + 1]
220     msave.append((totals0[dd].sum() - totals[dd].sum()) / 1e4)
221 ax.bar(np.arange(11), msave, color=get_colors('tab20', 11), edgecolor='
    black', lw=0.6)
222 ax.set_xticks(np.arange(11)); ax.set_xticklabels([f'{m+1}月' for m in range
    (1, 12)], fontsize=9)
223 style_ax(ax, '月份', '调整节省费用 (万元)')
224 ax.text(0.02, 0.02, '(b) 月度调整节省', transform=ax.transAxes, fontsize
    =10, fontweight='bold')
225 save_fig(fig, '图3-4_有无调整费用对比')
226
227 # 图3-5 典型日SOC：计划 vs 调整后
228 fig, ax = plt.subplots(figsize=(11.5, 5.2))
229 # 重新求解0:00计划SOC（用于对比）
230 g0_, ch0_, dch0_, E0p_, _ = solve_daily(load_all[d0], rec[d0]['S0'],
```

```
    price_daily, e0=carry[d0])
ax.plot(t, E0p_[:144], color=C1, lw=1.3, ls='--', label='0:00计划SOC')
ax.plot(t, rec[d0]['Ef'][:144], color=C4, lw=1.8, label='滚动调整后SOC')
ax.axhline(EMIN, color='#7A7A7A', lw=0.8, ls=':'); ax.axhline(EMAX, color='
    #7A7A7A', lw=0.8, ls=':')
style_ax(ax, '小时', '储电量 (kWh)')
ax.set_xticks(np.arange(0, 25, 6)); ax.set_ylim(EMIN - 400, EMAX + 400)
ax.legend(fontsize=9, loc='upper right', framealpha=0.9)
ax.text(0.01, 0.02, f'(a) {sd} 储电量：计划 vs 调整后', transform=ax.
    transAxes, fontsize=10, fontweight='bold')
save_fig(fig, '图3-5_计划与调整后SOC对比')

# 图3-6 费用构成（2—12月，月度堆叠）
fig, ax = plt.subplots(figsize=(11.5, 5.2))
plan_m = []; adj_m = []; emer_m = []
for m in range(1, 12):
    dd = [d for d in range(N) if dates[d].month == m + 1]
    plan_m.append(sum(rec[d]['plan_commit'] for d in dd) / 1e4)
    adj_m.append(sum(rec[d]['purchase_total'] - rec[d]['plan_commit'] for d
     in dd) / 1e4)
    emer_m.append(sum(rec[d]['emer_cost'] for d in dd) / 1e4)
x = np.arange(11)
ax.bar(x, plan_m, width=0.6, color=C1, edgecolor='black', lw=0.5, label='计
    划购电费')
ax.bar(x, adj_m, width=0.6, bottom=plan_m, color=C2, edgecolor='black', lw
    =0.5, label='调整相关费')
ax.bar(x, emer_m, width=0.6, bottom=np.array(plan_m) + np.array(adj_m),
    color=C4, edgecolor='black', lw=0.5, label='紧急购电费')
ax.set_xticks(x); ax.set_xticklabels([f'{m+1}月' for m in range(1, 12)],
    fontsize=9)
style_ax(ax, '月份', '月度费用 (万元)')
ax.legend(fontsize=9, loc='upper left', framealpha=0.9)
ax.text(0.02, 0.02, '(a) 2—12月费用构成(月度堆叠)', transform=ax.transAxes
    , fontsize=10, fontweight='bold')
save_fig(fig, '图3-6_全年费用构成')

# ---------- 写 result3.xlsx ----------
from openpyxl import Workbook
out = os.path.join(ROOT_OUT, 'result3.xlsx')
wb = Workbook()
labs = interval_labels()
ws = wb.active; ws.title = '计划购电量'
```

```
ws.append(['日期\\时间'] + labs + ['全天购电量', '全天购电费'])
start = S
for d in range(start, N):
    r = rec[d]
    gE = r['g0'] * DT
    ws.append([dates[d]] + [round(float(x), 4) for x in gE] + [round(float(gE.sum()), 4), round(float(r['plan_commit']), 4)])
ws2 = wb.create_sheet('调整购电量')
ws2.append(['日期\\时间'] + labs + ['全天购电量', '全天购电费'])
for d in range(start, N):
    r = rec[d]
    aE = r['adj'] * DT
    ws2.append([dates[d]] + [round(float(x), 4) for x in aE] + [round(float(aE.sum()), 4), round(float(r['purchase_total']), 4)])
ws3 = wb.create_sheet('充放电量')
ws3.append(['日期', '时间段', '充电量', '放电量', '时刻', '储电量'])
for d in range(start, N):
    r = rec[d]
    chE = r['chf'] * DT; dchE = r['dchf'] * DT
    for bi, (a, b) in enumerate([(0, 4), (4, 8), (8, 12), (12, 16), (16, 20), (20, 24)]):
        k0, k1 = a * 6, b * 6 - 1
        cq = round(float(chE[k0:k1 + 1].sum()), 4); dq = round(float(dchE[k0:k1 + 1].sum()), 4)
        if bi == 0:
            ws3.append([dates[d], f'{a:02d}:00-{b:02d}:00', cq, dq, '00:00:00', round(float(r['Ef'][0]), 4)])
        elif bi == 1:
            ws3.append([None, f'{a:02d}:00-{b:02d}:00', cq, dq, '24:00', round(float(r['Ef'][-1]), 4)])
        else:
            ws3.append([None, f'{a:02d}:00-{b:02d}:00', cq, dq, None, None])
ws4 = wb.create_sheet('紧急购电量')
ws4.append(['日期', '购电时间段', '购电量'])
for d in range(start, N):
    r = rec[d]
    em = r['emer_kwh']
    segs = []
    i = 0
    while i < 144:
        if em[i] > 1e-3:
```

```
            j = i
            while j < 144 and em[j] > 1e-3:
                j += 1
            segs.append((i, j, em[i:j].sum()))
            i = j
        else:
            i += 1
    if not segs:
        ws4.append([dates[d], None, 0.0])
    else:
        for si, (a, b, amt) in enumerate(segs):
            period = f'{a*10//60:02d}:{a*10%60:02d}-{b*10//60:02d}:{b*10%60:02d}'
            if si == 0:
                ws4.append([dates[d], period, round(float(amt), 4)])
            else:
                ws4.append([None, period, round(float(amt), 4)])
wb.save(out)
wb.save(os.path.join(OUT_DIR, 'result3.xlsx'))
print(f'已保存 result3.xlsx（{N - start} 天）→ 求解/结果/ 与 问题三/结果/')

# ---------- 边际收益分析：是否需要引入更多时刻的预报 ----------
print('\n========== 其他时刻预报的边际收益分析 ==========')
base0 = totals0[S:].sum()                    # 仅0:00（无任何调整），2.1—12.31 口径
cur = totals[S:].sum()                       # 当前方案 0/6/12/18（真实预报），2.1—12.31 口径
# 真实可用预报逐次累加（频率与精度均为题目给定水平）
s_real = [
    ('仅0:00',    base0),
    ('0/6',       run_year([0, 6], start=S)),
    ('0/6/12',    run_year([0, 6, 12], start=S)),
    ('0/6/12/18', cur),
]
# 完全预见（相同预报质量下）的频率效应：更密/更多时刻是否有额外收益
s_perf = [
    ('每6h(完全预见)',      run_year([0, 6, 12, 18], mode='perfect', start=S)),
    ('每6h+9/15(完全预见)', run_year([0, 6, 9, 12, 15, 18], mode='perfect', start=S)),
    ('每3h(完全预见)',      run_year([0, 3, 6, 9, 12, 15, 18, 21], mode='perfect', start=S)),
```

```
    ('每小时(完全预见)',    run_year(list(range(24)), mode='perfect', start=S)),
]
for nm, c in s_real + s_perf:
    print(f'  {nm}: {c:,.2f} 元, 较仅0:00节省 {base0-c:,.2f} 元 ({100*(base0-c)/base0:.3f}%)')
r = {nm: c for nm, c in s_real + s_perf}
m6 = base0 - r['0/6']
m12 = r['0/6'] - r['0/6/12']
m18 = r['0/6/12'] - r['0/6/12/18']
m915 = r['每6h(完全预见)'] - r['每6h+9/15(完全预见)']
m3h = r['每6h(完全预见)'] - r['每3h(完全预见)']
m1h = r['每3h(完全预见)'] - r['每小时(完全预见)']
acc = cur - r['每6h(完全预见)']
print(f'  真实预报边际节省: 6:00={m6:,.2f} 元, 12:00={m12:,.2f} 元, 18:00={m18:,.2f} 元')
print(f'  完全预见下频率增量: +9/15={m915:,.2f} 元, 6h→3h={m3h:,.2f} 元, 3h→1h={m1h:,.2f} 元')
print(f'  精度效应(6h频率下): 真实→完全预见 再省 {acc:,.2f} 元')
print(f'  当前方案捕获比 = {100*(base0-cur)/(base0-r["每小时(完全预见)"]):.2f}% (相对每小时完全预见上界)')

# 图3-7 其他时刻预报的边际收益
fig, axes = plt.subplots(1, 2, figsize=(13.6, 5.0))
# (a) 全年总费用
ax = axes[0]
lab_a = ['仅0:00', '0/6', '0/6/12', '0/6/12/18', '每6h\n完全预见', '每3h\n完全预见', '每小时\n完全预见']
val_a = [s_real[i][1] for i in range(4)] + [s_perf[0][1], s_perf[2][1], s_perf[3][1]]
col_a = [C1, C1, C1, C3] + [C6, C6, C6]
bars_a = [v / 1e4 for v in val_a]
ax.bar(range(7), bars_a, width=0.6, color=col_a, edgecolor='black', lw=0.6)
for i, v in enumerate(bars_a):
    ax.text(i, v + 6, f'{v:.1f}', ha='center', fontsize=8)
ax.set_xticks(range(7)); ax.set_xticklabels(lab_a, fontsize=8.5)
# 动态 y 轴范围: 完全预见方案费用远低于真实预报（约1220万元, 低于原固定下限1380）,
# 固定 ylim 会把这三根柱子裁掉, 导致"空白"。改为按数据自动缩放并留出标签空间。
lo_a, hi_a = min(bars_a), max(bars_a)
pad_a = max((hi_a - lo_a) * 0.18, 6.0)
```

```
ax.set_ylim(lo_a - pad_a, hi_a + pad_a)
style_ax(ax, None, '2.1—12.31总费用（万元）')
ax.text(0.02, 0.02, '(a) 不同调整时刻方案2.1—12.31总费用', transform=ax.transAxes, fontsize=10, fontweight='bold')
# (b) 边际节省
ax = axes[1]
lab_b = ['6:00', '12:00', '18:00', '+9:00/\n15:00', '6h→3h', '3h→1h']
val_b = [m6, m12, m18, m915, m3h, m1h]
col_b = [C3, C3, C3, C6, C6, C6]
ax.bar(range(6), [v / 1e4 for v in val_b], width=0.6, color=col_b, edgecolor='black', lw=0.6)
for i, v in enumerate(val_b):
    ax.text(i, v / 1e4 + 0.8, f'{v/1e4:.2f}', ha='center', fontsize=8)
ax.set_xticks(range(6)); ax.set_xticklabels(lab_b, fontsize=8.5)
style_ax(ax, None, '边际节省（万元）')
import matplotlib.patches as mpatches
ax.legend(handles=[mpatches.Patch(color=C3, label='真实预报时刻'),
                   mpatches.Patch(color=C6, label='完全预见下加密时刻')],
          fontsize=8, loc='upper right', framealpha=0.9)
ax.text(0.02, 0.02, '(b) 各新增时刻的边际节省', transform=ax.transAxes, fontsize=10, fontweight='bold')
save_fig(fig, '图3-7_其他时刻预报边际收益')
```

B.6 问题四求解程序 问题四.py

```
# 问题四：波动电价下重算问题2、问题3
# 4-2 与问题二同口径：每天 0:00 预测 — 决策 — 结算 三段式
# 4-3 为滚动时域调整（日内 4 次预报更新）
import sys, os
HERE = os.path.dirname(os.path.abspath(__file__))
SOLVE = os.path.dirname(HERE)
sys.path.insert(0, SOLVE)
from _common import *
from _models import *

price_fixed, _, _ = read_附件1()
dates, load_all, pv_actual_all = read_附件2()
d4, price_all = read_附件4()
fc = read_附件3()

def date_key(d):
    return f'{d.year}-{d.month}-{d.day}'
```

```
18
19  def merge_runs(delta, thr=1e-6):
20      """把非零紧急购电的连续间隔合并为（起时刻，止时刻，电量kWh）。"""
21      labs = interval_labels(); out = []; i = 0; n = len(delta)
22      while i < n:
23          if delta[i] > thr:
24              j = i
25              while j + 1 < n and delta[j + 1] > thr:
26                  j += 1
27              out.append((labs[i].split('-')[0], labs[j].split('-')[1].
    replace('+1', ''),
28                          float(np.sum(delta[i:j + 1]) * DT)))
29              i = j + 1
30          else:
31              i += 1
32      return out
33
34  N = len(dates)
35  S = 31   # 2025-02-01 索引；附件5仅要求保存 2.1—12.31，1月作为预热期，不纳
    入汇总统计
36  # ---- 4-2: 三段式（预测—随机规划—结算，波动电价） ----
37  g42 = np.zeros((N, N_INT)); ch42 = np.zeros((N, N_INT)); dch42 = np.zeros((
    N, N_INT))
38  dl42 = np.zeros((N, N_INT)); su42 = np.zeros((N, N_INT))
39  E42 = np.zeros((N, N_INT + 1))
40  plan42 = np.zeros(N); emg42 = np.zeros(N)
41  Lf42 = np.full((N, N_INT), np.nan); Sf42 = np.full((N, N_INT), np.nan)
42  res_pool_L, res_pool_P = [], []
43  PI = np.outer([0.2, 0.6, 0.2], [0.2, 0.6, 0.2]).ravel()   # 9 情景 = 3 负载
    × 3 光伏
44  carry2 = E0_INIT
45  for d in range(N):
46      if d == 0:
47          # 1/1 0:00 尚无历史数据，计划量置零，储电量全天保持 6000 kWh
48          E42[d] = np.full(N_INT + 1, E0_INIT)
49          continue
50      L_fc = forecast_load(load_all[:d], dates[:d], dates[d])
51      S_fc = forecast_pv(pv_actual_all[:d])
52      fL, fP = scenario_factors(np.concatenate(res_pool_L) if res_pool_L else
    np.array([]),
53                                np.concatenate(res_pool_P) if res_pool_P else
    np.array([]))
```

```
    LF = np.repeat(fL, 3); SF = np.tile(fP, 3)
    g, ch, dch, E, _ = solve_daily_stochastic(L_fc, S_fc, price_all[d], LF, SF, PI, e0=carry2)
    delta, _, plan_cost, emerg_cost = settle_emergency(load_all[d], pv_actual_all[d], price_all[d], g, ch, dch)
    g42[d], ch42[d], dch42[d], E42[d] = g, ch, dch, E
    dl42[d] = delta; plan42[d] = plan_cost; emg42[d] = emerg_cost
    su42[d] = np.maximum(0.0, pv_actual_all[d] + g + dch - ch - load_all[d]) * DT
    Lf42[d], Sf42[d] = L_fc, S_fc
    carry2 = float(E[-1])
    res_pool_L.append(pred_residual(L_fc, load_all[d]))
    m = S_fc > 50.0
    if m.any():
        res_pool_P.append(((pv_actual_all[d] - S_fc) / np.maximum(S_fc, 1.0))[m])
cost42 = plan42 + emg42

gE42 = g42 * DT; chE42 = ch42 * DT; dchE42 = dch42 * DT
W = slice(S, N); nW = N - S
plan_kwh = float(gE42[W].sum()); plan_fee = float(plan42[W].sum())
emg_kwh = float(dl42[W].sum() * DT); emg_fee = float(emg42[W].sum())
tot_fee = plan_fee + emg_fee
n_emg_int = int((dl42[W].reshape(-1) > 1e-6).sum())
n_emg_day = int((dl42[W].sum(axis=1) > 1e-6).sum())
net_kwh = float((load_all[W] - pv_actual_all[W]).sum() * DT)
print(f'4-2 波动电价全年(365天)购电费={cost42.sum():.2f} 元')
print(f'[计划段] 计划购电量={plan_kwh:.2f} kWh, 计划购电费={plan_fee:.2f} 元')
print(f'[紧急段] 紧急购电量={emg_kwh:.2f} kWh, 紧急购电费={emg_fee:.2f} 元')
print(f'[合计]  4-2 2.1—12.31 总购电费={tot_fee:.2f} 元, 日均={tot_fee / nW:.2f} 元 '
      f'(固定电价问题2为 14382892.99 元)')
print(f'[紧急覆盖面] 非零间隔={n_emg_int} 个 ({n_emg_int / (nW * N_INT) * 100:.2f}%), '
      f'发生天数={n_emg_day} 天 ({n_emg_day / nW * 100:.2f}%)')
print(f'[紧急强度] 最大功率={dl42[W].max():.2f} kW, 单日最大电量={float((dl42[W].sum(axis=1) * DT).max()):.2f} kWh')
print(f'[储能] 2/1 0:00 SOC={E42[S, 0]:.2f}, 12/31 24:00 SOC={E42[N - 1, -1]:.2f}, '
      f'SOC 最小={E42[W, :].min():.2f}, SOC 最大={E42[W, :].max():.2f}')
```

```
print(f'[能量流] 净负荷={net_kwh:.2f} kWh, 放电量={float(dchE42[W].sum()):.2f} kWh, '
      f'充电量={float(chE42[W].sum()):.2f} kWh, 弃电={float(su42[W].sum()):.2f} kWh')
bal = net_kwh - plan_kwh - float(dchE42[W].sum()) + float(chE42[W].sum())
print(f'[自洽校验] 净负荷-计划-放电+充电={bal:.2f} = 紧急电量({emg_kwh:.2f}) - 弃电({float(su42[W].sum()):.2f})')

# ---- 4-3: 滚动（波动电价） ----
def run_day43(d):
    L = load_all[d]; S_act = pv_actual_all[d]; key = date_key(dates[d]); f = fc[key]
    pr = price_all[d]
    S0 = forecast_10min(0, f[0], 0, 143); S6 = forecast_10min(6, f[6], 0, 143)
    S12 = forecast_10min(12, f[12], 0, 143); S18 = forecast_10min(18, f[18], 0, 143)
    g0, ch0, dch0, E0, c_plan = solve_daily(L, S0, pr, e0=carry3[d])
    g6, ch6, dch6, E6, _ = solve_adjust(L, S6, pr, g0, e0=float(E0[36]), k0=36)
    g12, ch12, dch12, E12, _ = solve_adjust(L, S12, pr, g0, e0=float(E6[36]), k0=72)
    g18, ch18, dch18, E18, _ = solve_adjust(L, S18, pr, g0, e0=float(E12[36]), k0=108)
    blocks = [0, 36, 72, 108, 144]
    sols = [(g0, ch0, dch0, E0), (g6, ch6, dch6, E6), (g12, ch12, dch12, E12), (g18, ch18, dch18, E18)]
    adj = np.zeros(144); chf = np.zeros(144); dchf = np.zeros(144); Ef = np.empty(145); Ef[0] = E0[0]
    for i in range(4):
        a, b = blocks[i], blocks[i + 1]
        g_, c_, d_, E_ = sols[i]
        adj[a:b] = g_[0:b - a]; chf[a:b] = c_[0:b - a]; dchf[a:b] = d_[0:b - a]
        Ef[a + 1:b + 1] = E_[1:b - a + 1]
    plan = g0
    base = np.minimum(plan, adj) * DT * pr
    down = np.maximum(plan - adj, 0) * DT * pr * 0.5
    up = np.maximum(adj - plan, 0) * DT * pr * 1.5
    plan_commit = (plan * DT * pr).sum()
    purchase_total = (base + down + up).sum()
    # 紧急购电 = 实际供电缺口: delta_k = max{L - (g_k + S^r_k + d_k - c_k),
```

```
    0}
    gap = np.maximum(L - (adj + S_act + dchf - chf), 0.0)
    emer_kwh = gap * DT
    emer_cost = (emer_kwh * 5.0 * pr).sum()
    total = purchase_total + emer_cost
    return dict(g0=g0, adj=adj, chf=chf, dchf=dchf, Ef=Ef, emer_kwh=
    emer_kwh, emer_cost=emer_cost,
                plan_commit=plan_commit, purchase_total=purchase_total,
    total=total, pr=pr)

carry3 = [E0_INIT] * N
rec43 = [None] * N
cost43 = np.zeros(N)
for d in range(N):
    r = run_day43(d)
    rec43[d] = r; cost43[d] = r['total']
    if d + 1 < N:
        carry3[d + 1] = float(r['Ef'][-1])
print(f'4-3 波动电价滚动全年(365天)总费用={cost43.sum():.2f} 元')
print(f'[汇总口径] 4-3 2.1—12.31 总费用={cost43[S:].sum():.2f} 元（固定电
    价问题3为 14337999.15 元）')

# 指定四天
spec_days = ['2025-03-20', '2025-06-21', '2025-09-23', '2025-12-21']
idx = {}
for d in range(N):
    dd = pd.Timestamp(dates[d]).date()
    for sd in spec_days:
        if dd == pd.Timestamp(sd).date():
            idx[sd] = d
print('\n表1/表2/表3 指定日期(波动电价):')
for sd in spec_days:
    d = idx[sd]
    print(f'  {sd}: 4-2购电费={cost42[d]:.2f}, 4-3总费用={cost43[d]:.2f}(计
    划{rec43[d]["plan_commit"]:.0f}+紧急{rec43[d]["emer_cost"]:.0f})')

print('\n[表4-2a~d] 指定时段计划购电量(kWh)')
for sd in spec_days:
    d = idx[sd]; gd = gE42[d]
    vals = [float(gd[h * 6]) for h in (10, 12, 14, 16, 18, 20)]
    print(f'  {sd}  ' + ' '.join(f'{v:10.2f}' for v in vals)
          + f'  全天={gd.sum():.2f} kWh, 计划费={plan42[d]:.2f} 元, '
```

```
            f'紧急量={float(dl42[d].sum() * DT):.2f} kWh, 紧急费={emg42[d]:.2f} 元')

print('\n[表4-2e~h] 指定时段充放电量(kWh)与两端储电量')
for sd in spec_days:
    d = idx[sd]; ce = chE42[d]; de = dchE42[d]
    print(f'  {sd}  0:00储电量={E42[d, 0]:.2f}  24:00储电量={E42[d, -1]:.2f}')
    for a, b in [(0, 4), (4, 8), (8, 12), (12, 16), (16, 20), (20, 24)]:
        k0, k1 = a * 6, b * 6 - 1
        print(f'    {a:02d}:00-{b:02d}:00  充电量={ce[k0:k1 + 1].sum():9.2f}  放电量={de[k0:k1 + 1].sum():9.2f}')

print('\n[表4-2i] 指定日期紧急购电时段与电量')
for sd in spec_days:
    d = idx[sd]; runs = merge_runs(dl42[d])
    print(f'  {sd}: 合计 {float(dl42[d].sum() * DT):.2f} kWh, 共 {len(runs)} 段')
    for a, b, e in runs:
        print(f'    {a}-{b}  {e:.2f} kWh')

# ---------- 出图 ----------
t = np.arange(144) / 6.0
C1, C2, C3, C4, C5, C6 = '#2E5FA3', '#E07B39', '#3BA272', '#C0504D', '#8A6BBE', '#D4A017'
plt.rcParams.update({'figure.dpi': 150, 'savefig.dpi': 300, 'axes.unicode_minus': False})

def style_ax(ax, xlabel=None, ylabel=None):
    ax.spines['top'].set_visible(False); ax.spines['right'].set_visible(False)
    ax.tick_params(direction='in', labelsize=9)
    for s in ['bottom', 'left']:
        ax.spines[s].set_linewidth(0.9)
    if xlabel: ax.set_xlabel(xlabel, fontsize=10)
    if ylabel: ax.set_ylabel(ylabel, fontsize=10)

sd = '2025-09-23'; d0 = idx[sd]

# 图4-1 波动电价 vs 固定电价（典型日）
fig, ax = plt.subplots(figsize=(11.5, 5.2))
ax.plot(t, price_all[d0], color=C4, lw=1.6, label='波动电价(附件4)')
```

```
ax.plot(t, price_fixed, color=C1, lw=1.6, ls='--', label='固定电价(附件1)')
style_ax(ax, '小时', '电价 (元/kWh)')
ax.set_xticks(np.arange(0, 25, 6)); ax.set_xlim(-0.3, 24.3)
ax.legend(fontsize=9, loc='upper right', framealpha=0.9)
ax.text(0.01, 0.02, f'(a) {sd} 波动电价与固定电价对比', transform=ax.
    transAxes, fontsize=10, fontweight='bold')
save_fig(fig, '图4-1_波动与固定电价对比')

# 图4-2 波动电价四季典型日购电策略（4-2，计划+紧急 叠加）
fig, axes = plt.subplots(1, 4, figsize=(15.6, 4.6))
for ax, sd_, col in zip(axes, spec_days, [C1, C3, C4, C2]):
    d = idx[sd_]
    ax.bar(t, g42[d], width=0.14, color=col, alpha=0.85, edgecolor='black',
     lw=0.4, label='计划')
    ax.bar(t, dl42[d], bottom=g42[d], width=0.14, color=C5, alpha=0.9,
    edgecolor='black', lw=0.4, label='紧急')
    ax.set_xticks(np.arange(0, 25, 6)); style_ax(ax, '小时', '购电 (kW)')
    ax.set_ylim(0, max(g42[d].max(), (g42[d] + dl42[d]).max()) * 1.18)
    ax.text(0.03, 0.02, f'({sd_[5:]})', transform=ax.transAxes, fontsize=9,
     fontweight='bold')
axes[0].legend(fontsize=8, loc='upper left', framealpha=0.9)
save_fig(fig, '图4-2_波动电价四季典型日购电')

# 图4-3 波动电价滚动调整购电量（4-3）
fig, ax = plt.subplots(figsize=(11.5, 5.2))
ax.plot(t, rec43[d0]['g0'], color=C1, lw=1.4, label='0:00计划购电')
ax.plot(t, rec43[d0]['adj'], color=C4, lw=1.4, ls='--', label='滚动调整后购
    电')
style_ax(ax, '小时', '购电功率 (kW)')
ax.set_xticks(np.arange(0, 25, 6)); ax.set_xlim(-0.3, 24.3)
ax.legend(fontsize=9, loc='upper right', framealpha=0.9)
ax.text(0.01, 0.02, f'(a) {sd} 波动电价下计划与调整购电', transform=ax.
    transAxes, fontsize=10, fontweight='bold')
save_fig(fig, '图4-3_波动电价滚动调整购电')

# 图4-4 波动电价SOC轨迹（4-2典型日）
fig, ax = plt.subplots(figsize=(11.5, 5.2))
ax.plot(t, E42[d0, :144], color=C3, lw=1.8, label='4-2 SOC')
ax.plot(t, rec43[d0]['Ef'][:144], color=C4, lw=1.8, ls='--', label='4-3 调
    整后SOC')
ax.axhline(EMIN, color='#7A7A7A', lw=0.8, ls=':'); ax.axhline(EMAX, color='
    #7A7A7A', lw=0.8, ls=':')
```

```
style_ax(ax, '小时', '储电量 (kWh)')
ax.set_xticks(np.arange(0, 25, 6)); ax.set_ylim(EMIN - 400, EMAX + 400)
ax.legend(fontsize=9, loc='upper right', framealpha=0.9)
ax.text(0.01, 0.02, f'(a) {sd} 波动电价储能SOC轨迹', transform=ax.transAxes
    , fontsize=10, fontweight='bold')
save_fig(fig, '图4-4_波动电价SOC轨迹')

# 图4-5 波动电价紧急购电量（4-3，2.1—12.31）
fig, ax = plt.subplots(figsize=(11.5, 5.2))
emer_daily = np.array([rec43[d]['emer_kwh'].sum() for d in range(N)])
ax.bar(dates[S:], emer_daily[S:], width=1.0, color=C4, alpha=0.8, edgecolor
    ='none')
style_ax(ax, '日期', '紧急购电量 (kWh)')
ax.text(0.01, 0.02, '(a) 波动电价下2.1—12.31逐日紧急购电量', transform=ax.
    transAxes, fontsize=10, fontweight='bold')
save_fig(fig, '图4-5_波动电价紧急购电量')

# 图4-6 固定vs波动电价费用对比（2.1—12.31）
fig, axes = plt.subplots(1, 2, figsize=(13.6, 5.0))
ax = axes[0]
fixed = [14382892.99, 14337999.15]   # 固定电价下问题2/问题3 的 2.1—12.31
    汇总
fluc = [cost42[S:].sum(), cost43[S:].sum()]
x = np.arange(2)
w = 0.35
ax.bar(x - w / 2, [f / 1e4 for f in fixed], w, color=C1, edgecolor='black',
     lw=0.7, label='固定电价')
ax.bar(x + w / 2, [f / 1e4 for f in fluc], w, color=C4, edgecolor='black',
    lw=0.7, label='波动电价')
for i in range(2):
     ax.text(i - w / 2, fixed[i] / 1e4 + 10, f'{fixed[i]/1e4:.0f}', ha='
    center', fontsize=8)
     ax.text(i + w / 2, fluc[i] / 1e4 + 10, f'{fluc[i]/1e4:.0f}', ha='center
    ', fontsize=8)
ax.set_xticks(x); ax.set_xticklabels(['问题2(逐日计划)', '问题3(滚动调整)'
    ], fontsize=10)
style_ax(ax, None, '2.1—12.31购电费 (万元)')
ax.legend(fontsize=9, loc='upper left', framealpha=0.9)
ax.text(0.02, 0.02, '(a) 固定/波动电价2.1—12.31费用', transform=ax.
    transAxes, fontsize=10, fontweight='bold')
ax = axes[1]
mflu = []
```

```
for m in range(1, 12):
    dd = [d for d in range(N) if dates[d].month == m + 1]
    mflu.append(cost43[dd].sum() / 1e4)
ax.bar(np.arange(11), mflu, color=get_colors('tab20', 11), edgecolor='black', lw=0.6)
ax.set_xticks(np.arange(11)); ax.set_xticklabels([f'{m+1}月' for m in range(1, 12)], fontsize=9)
style_ax(ax, '月份', '波动电价4-3月度费用（万元）')
ax.text(0.02, 0.02, '(b) 波动电价4-3月度费用', transform=ax.transAxes, fontsize=10, fontweight='bold')
save_fig(fig, '图4-6_固定与波动电价费用对比')

# ---------- 写 result4-2.xlsx ----------
from openpyxl import Workbook
labs = interval_labels()
start = S
wb = Workbook()
ws = wb.active; ws.title = '计划购电量'
ws.append(['日期\\时间'] + labs + ['全天购电量', '全天购电费'])
for d in range(start, N):
    gE = g42[d] * DT
    ws.append([dates[d]] + [round(float(x), 4) for x in gE] + [round(float(gE.sum()), 4), round(float(plan42[d]), 4)])
ws2 = wb.create_sheet('充放电量')
ws2.append(['日期', '时间段', '充电量', '放电量', '时刻', '储电量'])
for d in range(start, N):
    chE = ch42[d] * DT; dchE = dch42[d] * DT
    for bi, (a, b) in enumerate([(0, 4), (4, 8), (8, 12), (12, 16), (16, 20), (20, 24)]):
        k0, k1 = a * 6, b * 6 - 1
        cq = round(float(chE[k0:k1 + 1].sum()), 4); dq = round(float(dchE[k0:k1 + 1].sum()), 4)
        if bi == 0:
            ws2.append([dates[d], f'{a:02d}:00-{b:02d}:00', cq, dq, '00:00:00', round(float(E42[d, 0]), 4)])
        elif bi == 1:
            ws2.append([None, f'{a:02d}:00-{b:02d}:00', cq, dq, '24:00', round(float(E42[d, -1]), 4)])
        else:
            ws2.append([None, f'{a:02d}:00-{b:02d}:00', cq, dq, None, None])
ws3 = wb.create_sheet('紧急购电量')
```

```
ws3.append(['日期', '紧急购电时间段', '紧急购电量'])
for d in range(start, N):
    runs = merge_runs(dl42[d])
    for i, (a, b, e) in enumerate(runs):
        ws3.append([dates[d] if i == 0 else None, f'{a}-{b}', round(e, 4)])
wb.save(os.path.join(ROOT_OUT, 'result4-2.xlsx'))
wb.save(os.path.join(OUT_DIR, 'result4-2.xlsx'))
print('已保存 result4-2.xlsx → 求解/结果/ 与 问题四/结果/')

# ---------- 写 result4-3.xlsx ----------
wb = Workbook()
ws = wb.active; ws.title = '计划购电量'
ws.append(['日期\\时间'] + labs + ['全天购电量', '全天购电费'])
for d in range(start, N):
    r = rec43[d]; gE = r['g0'] * DT
    ws.append([dates[d]] + [round(float(x), 4) for x in gE] + [round(float(gE.sum()), 4), round(float(r['plan_commit']), 4)])
ws2 = wb.create_sheet('调整购电量')
ws2.append(['日期\\时间'] + labs + ['全天购电量', '全天购电费'])
for d in range(start, N):
    r = rec43[d]; aE = r['adj'] * DT
    ws2.append([dates[d]] + [round(float(x), 4) for x in aE] + [round(float(aE.sum()), 4), round(float(r['purchase_total']), 4)])
ws3 = wb.create_sheet('充放电量')
ws3.append(['日期', '时间段', '充电量', '放电量', '时刻', '储电量'])
for d in range(start, N):
    r = rec43[d]; chE = r['chf'] * DT; dchE = r['dchf'] * DT
    for bi, (a, b) in enumerate([(0, 4), (4, 8), (8, 12), (12, 16), (16, 20), (20, 24)]):
        k0, k1 = a * 6, b * 6 - 1
        cq = round(float(chE[k0:k1 + 1].sum()), 4); dq = round(float(dchE[k0:k1 + 1].sum()), 4)
        if bi == 0:
            ws3.append([dates[d], f'{a:02d}:00-{b:02d}:00', cq, dq, '00:00:00', round(float(r['Ef'][0]), 4)])
        elif bi == 1:
            ws3.append([None, f'{a:02d}:00-{b:02d}:00', cq, dq, '24:00', round(float(r['Ef'][-1]), 4)])
        else:
            ws3.append([None, f'{a:02d}:00-{b:02d}:00', cq, dq, None, None])
ws4 = wb.create_sheet('紧急购电量')
```

```
ws4.append(['日期', '购电时间段', '购电量'])
for d in range(start, N):
    em = rec43[d]['emer_kwh']
    segs = []
    i = 0
    while i < 144:
        if em[i] > 1e-3:
            j = i
            while j < 144 and em[j] > 1e-3:
                j += 1
            segs.append((i, j, em[i:j].sum())); i = j
        else:
            i += 1
    if not segs:
        ws4.append([dates[d], None, 0.0])
    else:
        for si, (a, b, amt) in enumerate(segs):
            period = f'{a*10//60:02d}:{a*10%60:02d}-{b*10//60:02d}:{b*10%60:02d}'
            if si == 0:
                ws4.append([dates[d], period, round(float(amt), 4)])
            else:
                ws4.append([None, period, round(float(amt), 4)])
wb.save(os.path.join(ROOT_OUT, 'result4-3.xlsx'))
wb.save(os.path.join(OUT_DIR, 'result4-3.xlsx'))
print('已保存 result4-3.xlsx → 求解/结果/ 与 问题四/结果/')
```

B.7 稳健性检验程序 稳健性检验.py

```
# 第六章稳健性/鲁棒性检验：数据摄动 + 不同算法一致性
# 数据摄动：对问题二三段式（预测—随机LP—结算）全年滚动施加乘性高斯噪声，重算 2.1—12.31 购电费
# 算法一致性：离散动态规划(DP) vs 线性规划(LP) 求解问题一单日调度
import sys, os, time
HERE = os.path.dirname(os.path.abspath(__file__))
SOLVE = HERE
sys.path.insert(0, SOLVE)
from _common import *
from _models import *

price_daily, load1, pv1 = read_附件1()
dates, load_all, pv_all = read_附件2()
```

```
N = len(dates)
S = 31          # 2025-02-01 索引；1月为预热期，仅统计 2.1—12.31
PI = np.outer([0.2, 0.6, 0.2], [0.2, 0.6, 0.2]).ravel()   # 9 情景 = 3 负载 × 3 光伏


def run_year_q2(load, pv):
    """问题二三段式全年滚动。返回 (计划购电费, 紧急购电费)（仅累计 2.1—12.31）。
    与 问题二.py 主循环同口径：0:00 只用历史数据预测 → 9 情景两阶段随机 LP → 真值结算。"""
    pool_L, pool_P = [], []
    carry = E0_INIT
    plan_tot = 0.0
    emg_tot = 0.0
    for d in range(N):
        if d == 0:
            continue                                    # 1/1 无任何历史，计划置零
        L_fc = forecast_load(load[:d], dates[:d], dates[d])
        S_fc = forecast_pv(pv[:d])
        fL, fP = scenario_factors(np.concatenate(pool_L) if pool_L else np.array([]),
                                  np.concatenate(pool_P) if pool_P else np.array([]))
        LF = np.repeat(fL, 3); SF = np.tile(fP, 3)
        g, ch, dch, E, _ = solve_daily_stochastic(L_fc, S_fc, price_daily,
                                                  LF, SF, PI, e0=carry)
        _, _, plan_cost, emerg_cost = settle_emergency(load[d], pv[d], price_daily, g, ch, dch)
        if d >= S:
            plan_tot += plan_cost
            emg_tot += emerg_cost
        carry = float(E[-1])
        pool_L.append(pred_residual(L_fc, load[d]))
        m = S_fc > 50.0
        if m.any():
            pool_P.append(((pv[d] - S_fc) / np.maximum(S_fc, 1.0))[m])
    return plan_tot, emg_tot


def run_year_nostorage(load, pv):
```

```
    """无储能对照：时段解耦时随机最优计划量满足 P(净负荷>g)=0.2（因边际紧急电价 5 倍），
    g 取 9 情景净负荷的加权 80% 分位；再按真值以 5 倍电价结算缺口。返回总购电费。"""
    pool_L, pool_P = [], []
    tot = 0.0
    for d in range(N):
        if d == 0:
            continue
        L_fc = forecast_load(load[:d], dates[:d], dates[d])
        S_fc = forecast_pv(pv[:d])
        fL, fP = scenario_factors(np.concatenate(pool_L) if pool_L else np.array([]),
                                  np.concatenate(pool_P) if pool_P else np.array([]))
        LF = np.repeat(fL, 3); SF = np.tile(fP, 3)
        net = L_fc[None, :] * LF[:, None] - S_fc[None, :] * SF[:, None]   # (9,144)
        g = np.empty(N_INT)
        for k in range(N_INT):
            o = np.argsort(net[:, k]); w = np.cumsum(PI[o])
            g[k] = net[o[min(int(np.searchsorted(w, 0.8)), 8)], k]
        if d >= S:
            tot += float(np.sum(price_daily * g) * DT)
            gap = np.maximum(0.0, (load[d] - pv[d]) - g)
            tot += float(np.sum(5.0 * price_daily * gap) * DT)
        pool_L.append(pred_residual(L_fc, load[d]))
        m = S_fc > 50.0
        if m.any():
            pool_P.append(((pv[d] - S_fc) / np.maximum(S_fc, 1.0))[m])
    return tot


t0 = time.time()
plan_base, emg_base = run_year_q2(load_all, pv_all)
base = plan_base + emg_base
print(f'基准三段式 2.1—12.31: 计划购电费={plan_base:.2f} 紧急购电费={emg_base:.2f} '
      f'合计={base:.2f} 元（1次全年耗时 {time.time()-t0:.1f}s）')

t0 = time.time()
nostor = run_year_nostorage(load_all, pv_all)
```

```
print(f'无储能对照 2.1—12.31 购电费 = {nostor:.2f} 元（耗时 {time.time()-t0:.1f}s，'
      f'相对储能方案 {100*(nostor-base)/base:+.2f}%）')

# ---------- 数据摄动检验 ----------
def perturb_run(sigma, seed):
    r = np.random.default_rng(seed)
    L = np.maximum(load_all * (1.0 + r.normal(0.0, sigma, load_all.shape)), 0.0)
    P = np.maximum(pv_all * (1.0 + r.normal(0.0, sigma, pv_all.shape)), 0.0)
    p, e = run_year_q2(L, P)
    return p + e

pert = {}
for sigma, seeds in [(0.05, 6), (0.10, 3)]:
    vals = np.array([perturb_run(sigma, 1000 + i) for i in range(seeds)])
    pert[sigma] = vals
    print(f'sigma={sigma:.2f}: 均值={vals.mean():.2f} 标准差={vals.std():.2f} '
          f'范围=[{vals.min():.2f},{vals.max():.2f}] 最大偏差=[{100*(vals.min()-base)/base:+.3f}%, {100*(vals.max()-base)/base:+.3f}%]')

# ---------- 算法一致性：离散动态规划(DP) vs LP ----------
def solve_dp(load, pv, price, e0, e_end=None, de=20.0):
    """SOC 网格化动态规划求解单日调度。返回 (最优费用, 末值边界费用向量)。
    e_end=None 自由末值；否则固定末值（问题一的日循环 E0=E24）。"""
    L = np.asarray(load, dtype=float); S = np.asarray(pv, dtype=float)
    pr = np.asarray(price, dtype=float); n = len(L)
    grid = np.arange(EMIN, EMAX + de / 2.0, de)
    ng = len(grid)
    INF = 1e18
    dp = np.full(ng, INF)
    dp[int(round((e0 - EMIN) / de))] = 0.0
    gv = grid.reshape(ng, 1)
    for k in range(n):
        Delta = gv.T - gv                       # Delta[b,a] = SOC_b - SOC_a
        c = np.where(Delta > 0, Delta / (ETA * DT), 0.0)      # 充电功率
        d = np.where(Delta < 0, -Delta * ETA / DT, 0.0)       # 放电功率
        feas = (c <= PMAX + 1e-6) & (d <= PMAX + 1e-6)
        g = np.maximum(L[k] - S[k] - d + c, 0.0)              # 电网购电功率
        cost = g * pr[k] * DT
```

```
        cost[~feas] = INF
        dp = np.min(dp[None, :] + cost, axis=1)
    if e_end is None:
        return float(dp.min()), dp
    return float(dp[int(round((e_end - EMIN) / de))]), dp

# 问题一 LP 基准（日循环 E0=E24=6000）
_, _, _, _, lp_cost1 = solve_daily(load1, pv1, price_daily, e0=E0_INIT,
    soc_cycle=True)
print(f'问题一 LP(日循环) 购电费 = {lp_cost1:.2f} 元')
dp_results = {}
for de in [80.0, 40.0, 20.0]:
    c, _ = solve_dp(load1, pv1, price_daily, E0_INIT, e_end=E0_INIT, de=de)
    dp_results[de] = c
    print(f'问题一 DP(日循环, de={de:.0f}kWh) 购电费 = {c:.2f} 元, 相对LP偏
    差 {100*(c-lp_cost1)/lp_cost1:+.4f}%')

# 问题一自由末值 LP vs DP（同口径，验证算法一致性，不含日循环）
_, _, _, _, lp_free1 = solve_daily(load1, pv1, price_daily, e0=E0_INIT)
dp_free1, _ = solve_dp(load1, pv1, price_daily, E0_INIT, e_end=None, de
    =20.0)
print(f'问题一(自由末值): LP={lp_free1:.2f} 元, DP(de=20)={dp_free1:.2f} 元
    , 偏差 {100*(dp_free1-lp_free1)/lp_free1:+.4f}%')

# ---------- 出图 图6-1 稳健性检验 ----------
C1, C2, C3, C4, C6 = '#2E5FA3', '#E07B39', '#3BA272', '#C0504D', '#D4A017'
plt.rcParams.update({'figure.dpi': 150, 'savefig.dpi': 300, 'axes.
    unicode_minus': False})

def style_ax(ax, xlabel=None, ylabel=None):
    ax.spines['top'].set_visible(False); ax.spines['right'].set_visible(
    False)
    ax.tick_params(direction='in', labelsize=9)
    for s in ['bottom', 'left']:
        ax.spines[s].set_linewidth(0.9)
    if xlabel: ax.set_xlabel(xlabel, fontsize=10)
    if ylabel: ax.set_ylabel(ylabel, fontsize=10)

fig, axes = plt.subplots(1, 2, figsize=(13.6, 5.0))

# (a) 数据摄动
ax = axes[0]
```

```
means = [base, pert[0.05].mean(), pert[0.10].mean()]
stds = [0.0, pert[0.05].std(), pert[0.10].std()]
xs = np.arange(3)
ax.bar(xs, [m / 1e4 for m in means], width=0.5, yerr=[s / 1e4 for s in stds],
       capsize=5, color=[C1, C3, C4], edgecolor='black', lw=0.8, ecolor='black', error_kw=dict(lw=1.0))
for i, m in enumerate(means):
    ax.text(i, m / 1e4 + 18, f'{m/1e4:.1f}', ha='center', fontsize=9)
ax.set_xticks(xs); ax.set_xticklabels(['基准\n($\\sigma$=0)', '$\\sigma$=5%', '$\\sigma$=10%'], fontsize=9)
style_ax(ax, '摄动强度', '2.1—12.31购电费（万元）')
ax.text(0.02, 0.02, '(a) 负荷/光伏数据摄动', transform=ax.transAxes, fontsize=10, fontweight='bold')

# (b) 算法一致性：LP vs DP（不同网格）
ax = axes[1]
x2 = np.arange(4)
vals2 = [lp_cost1 / 1e4, dp_results[80.0] / 1e4, dp_results[40.0] / 1e4, dp_results[20.0] / 1e4]
col2 = [C1, C6, C6, C6]
ax.bar(x2, vals2, width=0.55, color=col2, edgecolor='black', lw=0.8)
for i, v in enumerate(vals2):
    ax.text(i, v + 0.06, f'{v:.2f}', ha='center', fontsize=9)
ax.set_xticks(x2)
ax.set_xticklabels(['LP\n(连续)', 'DP\n$\\Delta$=80', 'DP\n$\\Delta$=40', 'DP\n$\\Delta$=20'], fontsize=8.5)
style_ax(ax, None, '问题一全天购电费（万元）')
ax.text(0.02, 0.02, '(b) 不同算法一致性(LP vs DP)', transform=ax.transAxes, fontsize=10, fontweight='bold')

save_fig(fig, '图6-1_稳健性检验')
print('图6-1 已保存')

# 汇总打印（供 LaTeX 表取值）
print('\n===== 稳健性检验汇总 =====')
print(f'base={base:.2f} (计划={plan_base:.2f} 紧急={emg_base:.2f})')
print(f'nostorage={nostor:.2f} dev={100*(nostor-base)/base:+.3f}')
for sigma in [0.05, 0.10]:
    v = pert[sigma]
    print(f'sigma{sigma}: mean={v.mean():.2f} std={v.std():.2f} maxdev_up={100*(v.max()-base)/base:.3f} maxdev_dn={100*(v.min()-base)/base:.3f}')
```

```
print(f'lp_cost1={lp_cost1:.2f}')
for de in [80.0, 40.0, 20.0]:
    print(f'dp_de{de:.0f}={dp_results[de]:.2f} dev={100*(dp_results[de]-lp_cost1)/lp_cost1:.3f}')
print(f'lp_free1={lp_free1:.2f} dp_free1={dp_free1:.2f} dev={100*(dp_free1-lp_free1)/lp_free1:.3f}')
```